

## **ABSTRACT**

Distributed Denial-of-Service (DDoS) attacks continue to represent one of the most disruptive classes of threats to modern network infrastructure, exploiting the availability pillar of the Confidentiality, Integrity, and Availability (CIA) triad by overwhelming target systems with coordinated traffic from compromised hosts. Conventional signature-based Intrusion Detection Systems, while effective against previously catalogued attacks, are structurally unable to identify zero-day or polymorphic attack variants, motivating a shift toward anomaly-based detection paradigms that learn a representation of normal network behaviour and flag significant deviations from it.

This project undertakes a systematic, comparative evaluation of four anomaly detection paradigms — a classical univariate statistical method (Z-Score), an ensemble partition-based method (Isolation Forest), a kernel-based novelty detector (One-Class Support Vector Machine), and a density-based novelty detector (Local Outlier Factor) — for the detection of DDoS traffic within flow-level network data. The study is conducted on the Friday-WorkingHours-Afternoon-DDoS subset of the CICIDS2017 benchmark dataset, comprising network flows captured on a realistic testbed and processed through the CICFlowMeter tool into a rich set of flow-level statistical descriptors.

Prior to model development, the raw dataset was subjected to a structured preprocessing pipeline addressing missing and infinite values, duplicate flow records, ten constant and non-discriminatory features, and twenty-six features exhibiting pairwise correlation above 0.95, yielding a cleaned dataset described by a reduced set of forty-two independent numerical features. All features were standardized prior to modelling to ensure that no individual descriptor dominated distance- or deviation-based computations owing to differences in measurement scale. Exploratory data analysis was additionally conducted to characterise feature distributions, class-conditional separability, and the correlation structure of the dataset, directly informing the preprocessing strategy adopted.

Each of the four techniques was evaluated using a common methodology wherever the underlying learning paradigm permitted. Isolation Forest, One-Class SVM, and Local Outlier Factor were trained exclusively on benign traffic in a novelty detection configuration and evaluated on a held-out combination of unseen benign flows and the complete set of DDoS flows, while the Z-Score method was applied directly across the full labelled dataset as a fully unsupervised statistical

baseline. Each technique was subjected to a structured hyperparameter search, and model performance was assessed using Precision, Recall, F1-score, overall accuracy, the confusion matrix, the False Positive Rate, Benign Accuracy, and the DDoS Detection Rate, together with training and prediction times where recorded.

The results establish a clear performance hierarchy among the four techniques evaluated. The Z-Score baseline, constrained by its univariate, axis-aligned formulation, achieved only modest detection performance across all tested thresholds and was found unsuitable as a standalone detector, retaining value chiefly as a computationally negligible point of comparison. All three machine learning techniques substantially outperformed this baseline once systematically tuned, confirming that multivariate modelling of joint feature structure is necessary for effective DDoS detection on this dataset. Among the three, Local Outlier Factor achieved the strongest overall performance, combining the highest F1-score and DDoS Detection Rate with the lowest False Positive Rate among the machine learning methods evaluated, consistent with the expectation that flooding-style DDoS traffic forms a locally distinguishable population well suited to density-based detection. One-Class SVM achieved closely comparable detection accuracy at a considerably higher computational cost during training, while Isolation Forest offered the most favourable computational scalability at the expense of a markedly higher False Positive Rate.

Overall, the study demonstrates that density-based novelty detection, as exemplified by Local Outlier Factor, provides the most effective balance of detection accuracy and operational reliability for DDoS detection on flow-based network traffic at the scale examined, while also highlighting the computational and hyperparameter-sensitivity considerations that are relevant to practical, large-scale deployment of such systems.

## **ABBREVIATIONS**

The following table lists the principal abbreviations used throughout this dissertation, in the order in which most first appear or are introduced.

| <b>Abbreviation</b> | <b>Full Form</b>                                                               |
|---------------------|--------------------------------------------------------------------------------|
| ACK                 | Acknowledgement Flag (TCP control flag)                                        |
| AUC                 | Area Under the (ROC) Curve                                                     |
| CIA                 | Confidentiality, Integrity, Availability (security triad)                      |
| CICIDS2017          | Canadian Institute for Cybersecurity Intrusion Detection System 2017 (dataset) |
| CSV                 | Comma-Separated Values                                                         |
| CWE                 | CWE Flag Count (a constant CICFlowMeter-derived flow feature)                  |
| DDoS                | Distributed Denial of Service                                                  |
| DoS                 | Denial of Service                                                              |
| ECE                 | ECN-Echo Flag (TCP control flag)                                               |
| EDA                 | Exploratory Data Analysis                                                      |
| FIN                 | Finish Flag (TCP control flag)                                                 |
| FPR                 | False Positive Rate                                                            |
| FTP                 | File Transfer Protocol                                                         |
| HTTP / HTTPS        | HyperText Transfer Protocol / HyperText Transfer Protocol Secure               |
| IAT                 | Inter-Arrival Time                                                             |
| IDPS                | Intrusion Detection and Prevention System                                      |
| IDS                 | Intrusion Detection System                                                     |
| IF                  | Isolation Forest                                                               |
| IoT                 | Internet of Things                                                             |
| LOF                 | Local Outlier Factor                                                           |
| ML                  | Machine Learning                                                               |
| NIST                | National Institute of Standards and Technology                                 |
| OCSVM / OC-SVM      | One-Class Support Vector Machine                                               |
| PSH                 | Push Flag (TCP control flag)                                                   |
| RBF                 | Radial Basis Function                                                          |
| ROC                 | Receiver Operating Characteristic                                              |
| SSH                 | Secure Shell                                                                   |
| SVM                 | Support Vector Machine                                                         |

|     |                                                                 |
|-----|-----------------------------------------------------------------|
| SYN | Synchronize Flag (TCP control flag)                             |
| TCP | Transmission Control Protocol                                   |
| TPR | True Positive Rate (equivalent to Recall / DDoS Detection Rate) |
| UDP | User Datagram Protocol                                          |
| URG | Urgent Flag (TCP control flag)                                  |

## LIST OF FIGURES

**Figure 2.1:** A network flow as a bidirectional sequence of forward and backward packets between a client and server.

**Figure 2.2:** Z-score thresholding under the assumed Gaussian distribution of a feature. Observations falling in the shaded tail regions, beyond  $\pm t$  standard deviations from the mean, are flagged as anomalous.

**Figure 2.3:** Left – random axis-aligned partitioning of the feature space isolates a sparse anomaly (red star) quickly. Right – the corresponding isolation-tree path length is short for the anomaly and long for a normal point buried in a dense region.

**Figure 2.4:** Conceptual illustration of the general trade-off pattern between Precision, Recall and F1-score as the contamination parameter is increased. Curve shapes are illustrative only.

**Figure 2.5:** One-Class SVM learns a closed, non-linear decision boundary (via the RBF kernel) around the normal traffic cluster; points lying outside the boundary are flagged as anomalies.

**Figure 2.6:** Local Outlier Factor compares a point's local density to that of its neighbours. Points in the dense blue region and the sparser (but locally consistent) green region are normal, while the red point has a much larger k-distance relative to its neighbours and is flagged as an outlier.

**Figure 2.7:** Conceptual illustration of the general trade-off pattern between neighbourhood size ( $n\_neighbors$ ) and detection performance for LOF. Curve shape is illustrative only.

**Figure 2.8:** Conceptual ROC curve. The diagonal line represents random guessing ( $AUC = 0.5$ ); a curve bowed toward the top-left corner represents better separation between the normal and anomalous classes.

**Figure 4.1:** Confusion matrix for Z-Score anomaly detection at threshold = 3.0.

**Figure 4.2:** Comparison of Precision, Recall, and F1-Score across Z-Score thresholds.

**Figure 4.3:** Confusion matrix for Z-Score anomaly detection at threshold = 2.5.

**Figure 4.4:** Confusion matrix for Z-Score anomaly detection at threshold = 2.0.

**Figure 4.5:** Confusion matrix for the initial (untuned) One-Class SVM model ( $nu = 0.05$ ,  $gamma = 'scale'$ ).

**Figure 4.6:** Effect of  $nu$  on One-Class SVM Precision, Recall, and F1-Score.

**Figure 4.7:** Effect of  $gamma$  on One-Class SVM Precision, Recall, and F1-Score.

**Figure 4.8:** Confusion matrix for the optimized One-Class SVM model ( $nu = 0.05$ ,  $gamma = 3.0$ ).

**Figure 4.9:** Precision, Recall, and F1-Score by class for the optimized One-Class SVM model.

**Figure 4.10:** Confusion matrix for the initial (untuned) Isolation Forest model (contamination = 'auto').

**Figure 4.11:** Isolation Forest Precision, Recall, and F1-Score across evaluated contamination rates.

**Figure 4.12:** Line plot of Precision, Recall, and F1-Score against contamination rate for Isolation Forest.

**Figure 4.13:** Confusion matrix for the best contamination-only Isolation Forest model (contamination = 0.5, n\_estimators = 100).

**Figure 4.14:** Effect of n\_estimators on Isolation Forest Precision, Recall, and F1-Score (contamination = 0.5).

**Figure 4.15:** Confusion matrix for the final Isolation Forest model (n\_estimators = 200, contamination = 0.5).

**Figure 4.16:** Isolation Forest anomaly score distribution by class (histogram and boxplot).

**Figure 4.17:** Top 15 features by permutation importance for the final Isolation Forest model.

**Figure 4.18:** Effect of n\_neighbors on Local Outlier Factor F1-score, grouped by contamination.

**Figure 4.19:** Effect of contamination on Local Outlier Factor F1-score, grouped by n\_neighbors.

**Figure 4.20:** Heatmap of Local Outlier Factor F1-score across the full n\_neighbors × contamination grid.

**Figure 4.21:** Confusion matrix for the optimized Local Outlier Factor model (n\_neighbors = 20, contamination = 0.10).

## **LIST OF TABLES**

**Table 2.0:** Summary of mathematical notation used throughout this report.

**Table 2.1:** Theoretical tail probabilities of the standard normal distribution for common Z-score thresholds.

**Table 2.3:** Key Isolation Forest hyperparameters and their general effect on detection behaviour.

**Table 2.4:** Generic confusion-matrix template used to evaluate Isolation Forest predictions at a given contamination setting.

**Table 2.5:** Key One-Class SVM hyperparameters and their general effect on the learned decision boundary.

**Table 2.6:** Key Local Outlier Factor hyperparameters and their general effect on detection behaviour.

**Table 2.7:** Generic confusion-matrix template used to evaluate Local Outlier Factor predictions at a given hyperparameter configuration.

**Table 2.8:** Structure of the  $2 \times 2$  confusion matrix used for binary anomaly-detection evaluation.

**Table 2.9:** Hypothetical confusion matrix used purely to illustrate the metric formulas.

**Table 2.10:** Qualitative comparison of the three machine learning anomaly detection methods described in the report.

**Table 4.1:** Z-Score performance across evaluated thresholds.

**Table 4.2:** Effect of  $\nu$  on One-Class SVM performance ( $\gamma = \text{'scale'}$ , DDoS class metrics).

**Table 4.3:** Effect of  $\gamma$  on One-Class SVM performance ( $\nu = 0.05$ , DDoS class metrics).

**Table 4.4:** Effect of contamination on Isolation Forest performance ( $n_{\text{estimators}} = 100$ , DDoS class metrics).

**Table 4.5:** Effect of  $n_{\text{estimators}}$  on Isolation Forest performance ( $\text{contamination} = 0.5$ , DDoS class metrics).

**Table 4.6:** Selected Local Outlier Factor grid-search results across  $n_{\text{neighbors}}$  and contamination (DDoS class metrics; full grid comprised 30 configurations).

**Table 4.7:** Comparison of all four anomaly detection techniques (DDoS class metrics).

# **CHAPTER - 1**

## **Introduction**

### **1.1 Background**

The twenty-first century has been characterized by a rapid and unprecedented growth in Internet connectivity and digital communication. Modern society has shifted toward a paradigm of total dependence on connected actuators and sensors, with the number of connected devices reaching a peak of approximately 50 billion. This digital revolution has seen Internet traffic per capita surge from 10 GB in 2016 to an average of 30 GB, reflecting the integration of network technology into every facet of human life. Consequently, computer networks have become the backbone of critical global infrastructures, including banking, healthcare, education, government operations, and industrial automation. Organizations today rely so heavily on uninterrupted network services that malicious activities are estimated to cost more than \$2.9 million every minute, with sensitive data violations alone costing major businesses roughly \$25 per minute.

As connectivity increases, so does the complexity and frequency of cyber threats. The attack surface is expanding exponentially because every new service interface and connected IoT device provides a potential entry point for adversaries. Cybersecurity has therefore transitioned from a niche technical concern to one of the most critical areas in information technology. Maintaining network availability and security is essential for the survival of modern enterprises, as even a brief disruption can lead to massive record exposure, such as the 36 billion records exposed in the first three quarters of 2020 alone.

To defend these vital infrastructures, organizations deploy Intrusion Detection and Prevention Systems (IDPS). According to the NIST Guide to Intrusion Detection and Prevention Systems (SP 800-94), intrusion detection is the automated process of monitoring events occurring in a computer system or network and analysing them for signs of possible incidents, which are violations of security policies or standard practices. An IDS acts as a strategic security solution, offering a thorough defence against potential threats that target the network's integrity and data. While early versions of these tools were highly effective at identifying known threats through predefined patterns, they are increasingly challenged by the arrival of zero-day attacks and polymorphic malware—threats that have no known signature at the time of their deployment.

Traditional security relies heavily on Signature-Based Detection, which compares patterns of observed events against a database of known threats. While these systems demonstrate high accuracy and low false-positive rates for established attacks, they are inherently reactive and struggle to adapt to emerging or modified threats. This limitation has necessitated a shift toward Anomaly-Based Detection, which identifies significant deviations from a definition of "normal" behaviour. By monitoring typical network characteristics—such as bandwidth usage or session duration—these systems can detect previously unknown attacks that behave differently from established profiles.

Despite the potential of anomaly-based systems, they often suffer from high rates of false positives due to the extreme complexity and dynamic nature of modern network traffic. Benign changes in a network environment, such as the introduction of a new service or a legitimate spike in user activity, can easily be mistaken for a malicious intrusion. To bridge this gap, the field has increasingly turned toward Statistical and Machine Learning (ML) models. These models can



identify complex, non-linear relationships in network data that are invisible to human analysts or rigid rule-based systems.

## **1.2 Network Security Challenges**

The rapid proliferation of connected devices, particularly in the Internet of Things (IoT) sector, has exponentially expanded the network attack surface, leading to a significant rise in sophisticated, high-volume cyber threats. Organizations today must protect their systems and users against a diverse array of serious cybersecurity threats that result in staggering financial losses. These threats target the core components of the Confidentiality, Integrity, and Availability (CIA) triad, which serves as the foundational framework for network security.

Among the most prevalent security challenges is Malware, a category of host-based threats that involves running malicious software to compromise or corrupt system functionalities. This includes Viruses, which affect programs and files when shared with other users, and Worms, which are particularly dangerous due to their ability to self-replicate and spread across multiple systems independently. A relatively new and highly damaging variant of malware is Ransomware, where an attacker keeps a system under their control by encrypting critical files and demanding payment for their release.

Beyond automated software, Human-Based Attacks present persistent challenges. Phishing remains a common tactic where adversaries use electronic messaging services to deceive individuals into providing credentials or confidential data. Furthermore, attackers often leverage Botnets—networks of compromised devices—to facilitate coordinated campaigns such as scanning for vulnerabilities or performing lateral movement within an enterprise.

Traditional network threats also include Denial of Service (DoS) and Distributed Denial of Service (DDoS), where an attacker floods a target with a flow of packets to render services unresponsive. During these attacks, legitimate users are prevented from accessing network resources, directly compromising the availability of information systems.

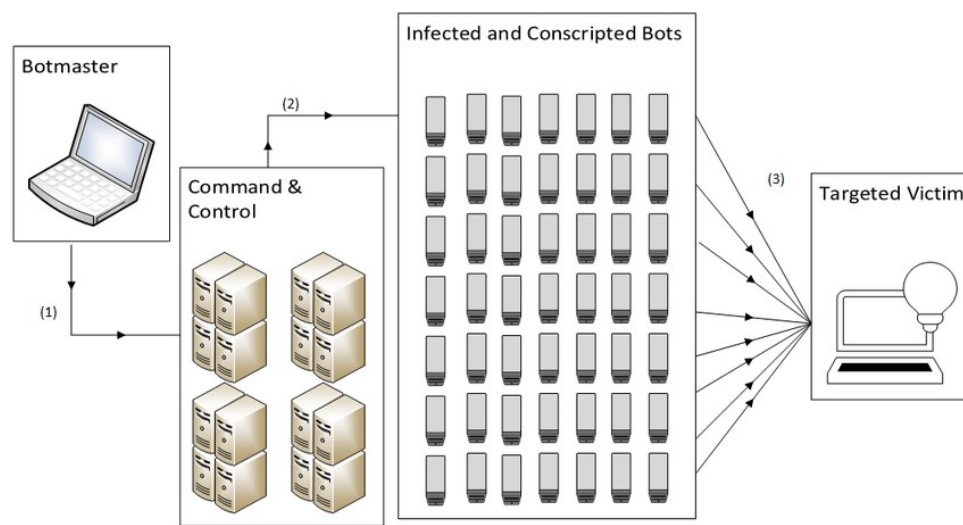
Among these diverse threats, DDoS attacks remain one of the most damaging because they target the availability of network services rather than focusing solely on stealing information. From an operational perspective, DDoS attacks are uniquely challenging because they generate more alerts than all other forms of attack, which can quickly overwhelm security analysts and exhaust the resources of Intrusion Detection and Prevention Systems (IDPS). Specifically, DDoS scenarios have been shown to significantly increase memory consumption and packet drop rates, potentially allowing other malicious traffic to enter the network undetected while the defence systems are strained.

## **1.3 Distributed Denial-of-Service (DDoS)**

A Distributed Denial of Service (DDoS) attack is a sophisticated cyberattack where numerous compromised devices, coordinated simultaneously, send massive volumes of traffic to a target server or network. The primary objective is to paralyze the target by disrupting connectivity and preventing legitimate users from accessing essential services. Unlike traditional single-source attacks, DDoS leverages a distributed architecture, making it one of the most notorious and damaging threats in the modern digital landscape.

Key Components and Impacts

- **Botnets:** Modern DDoS attacks are typically executed using well-organized and remotely controlled botnets. These consist of thousands of malware-infected "zombie" machines—ranging from personal computers to IoT devices—that obey commands from a central "botmaster". By employing a bot army, attackers not only magnify the severity of the attack but also protect their anonymity by masking the original source of the malicious traffic.
- **Flood Attacks and Resource Exhaustion:** The essence of a DDoS attack lies in its ability to generate "floods" of data packets at an excessive rate. These floods target specific assets to cause resource exhaustion, draining the victim's network bandwidth, CPU processing power, memory capacity, and connection tables. For instance, application-layer floods may consume all available disk space and memory through valid-looking transaction requests, leaving the server unable to process genuine user actions.
- **Financial Impact and Service Downtime:** The consequences of such attacks are frequently catastrophic for an organization's bottom line. Specifically, the resulting service downtime can cost a company between \$300,000 and \$1,000,000 for every hour IT services remain unresponsive. Major historical incidents have seen high-scale attacks paralyze cryptocurrency exchanges and knock national stock exchanges offline for days.



*Fig. 1.1 DDoS Attack Architecture*

### Common Examples of DDoS Attacks

- **HTTP Flood:** This is an application-layer attack that damages web servers by swamping them with a continuous stream of valid HTTP GET or POST requests. Because these requests often originate from bots with valid IP addresses, they can bypass simple filters that only look for malformed packets.
- **SYN Flood:** A protocol-based attack that exploits the TCP three-way handshake. The attacker sends a massive number of SYN requests but fails to respond to the host's SYN-ACK response, forcing the server to bind its resources while waiting for acknowledgments that never arrive.

- **UDP Flood:** This attack floods the victim with connectionless UDP packets, aiming to saturate the entire network bandwidth. Because UDP does not require a formal connection setup, it is harder for traditional defence mechanisms to identify and mitigate this type of traffic.

### Detection Challenges

DDoS attacks are notoriously difficult to detect for several technical reasons:

- **Traffic Resembles Legitimate Requests:** Many DDoS attacks use non-spoofed IP addresses and standard protocol mechanisms, making the malicious traffic virtually indistinguishable from legitimate user requests. This allows the attack to bypass traditional security perimeters until the resource bottleneck becomes critical.
- **Continuous Evolution of Attack Patterns:** Attackers are constantly refining their methods, leading to the rise of "DDoS as a Service" where sophisticated attack patterns are available for hire on the dark web. New vulnerabilities in emerging technologies like 5G and IoT provide ever-changing platforms for launching novel attack vectors.
- **Enormous Volumes in High-Speed Networks:** The sheer volume of data generated by modern high-speed networks makes deep packet inspection nearly impossible in real-time. Network Behaviour Analysis (NBA) systems often face delays because they must process flow statistics in batches, which may allow an instantaneous attack to cause damage before the system can react.

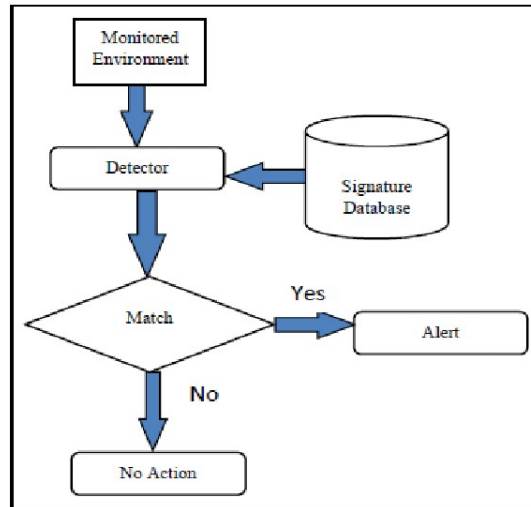
## 1.4 Intrusion Detection Systems

Intrusion detection is the systematic process of monitoring events occurring within a computer system or network and analysing them for signs of possible incidents, which are defined as violations or imminent threats of violation of security policies. An Intrusion Detection System (IDS) is the software responsible for automating this detection process. As organizations become increasingly dependent on uninterrupted network services, IDSs serve as critical strategic security solutions designed to provide in-depth defence against potential threats to the integrity, confidentiality, and availability of data.

### Signature-based IDS:

Signature-based IDS, also frequently referred to as Misuse Detection, operates by comparing patterns of observed events against a database of known threat signatures. A signature is essentially a predefined pattern that corresponds to a specific known attack or type of attack. This methodology is highly effective at identifying established threats with great accuracy and a characteristically low rate of false alarms. However, its primary limitation is its inherently reactive nature; it fails to identify unknown or "zero-day" threats and modified variants of known attacks because they lack a matching entry in the signature database.

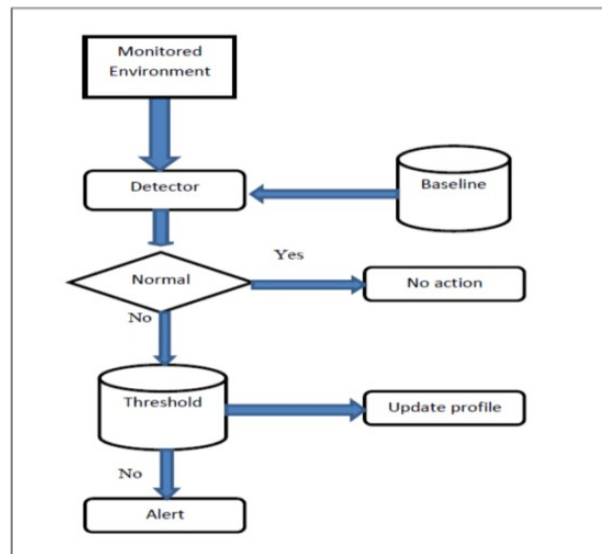
Consequently, these systems require constant maintenance and frequent updates to their signature libraries to remain effective against the evolving threat landscape.



*Fig.1.2 Flowchart of Signature-based IDS*

#### Anomaly-based IDS:

Anomaly-based IDS functions by establishing a baseline definition of what constitutes "normal" network activity and then identifying significant deviations from this profile. These profiles representing normal behaviour—such as typical bandwidth usage or session duration—are developed during an initial training period. The fundamental strength of this approach is its ability to detect previously unknown attacks, such as zero-day exploits or polymorphic malware, which behave differently from established normal profiles. Despite this versatility, anomaly-based systems often struggle with higher rates of false positives. This occurs because legitimate, benign changes in the network environment—such as the introduction of a new service or a sudden spike in user activity—can be misinterpreted as a malicious intrusion.



*Fig.1.3 Flowchart of Anomaly-based IDS*

Comparison Table:

| Feature        | Signature-based IDS                      | Anomaly-based IDS                         |
|----------------|------------------------------------------|-------------------------------------------|
| Detection Goal | Detects known attacks                    | Detects unknown and zero-day attacks      |
| Accuracy       | Low false-positive rates                 | May produce higher false-positive rates   |
| Requirements   | Needs a comprehensive signature database | Learns normal behaviour to create profile |

## 1.5 Network Anomaly Detection

Network anomaly detection is the core technical focus of this project, representing a shift from reactive to proactive security. It identifies deviations from established "normal" network behavior by analyzing high-dimensional traffic characteristics such as packet counts, flow duration, protocol usage, and byte rates. Rather than relying on a static database of predefined attack signatures—which can only identify known threats—anomaly detection establishes a dynamic model of legitimate behavior and flags any observation that significantly deviates from this baseline as a potential intrusion.

### Behavior Modelling and Normality

The foundational concept of this approach is behavior modelling, which involves characterizing the typical dynamics of a network during a designated training period. This process generates profiles—formal models representing the relationship between fundamental variables such as bandwidth usage at specific times or the frequency of various application-layer requests. Profiles can be static, remains unchanged until a manual reset, or dynamic, which constantly adjust as additional network events are observed to account for the evolving nature of network traffic. Any activity that fails to conform to these learned patterns is assigned an anomaly score, reflecting its degree of deviation from the established "normality".

### Statistical Methods:

Statistical methods provide a mathematical framework for anomaly detection by assuming that normal data follows a specific distribution or frequency pattern. These are broadly categorized into:

- **Parametric Methods:** These assume that normal traffic is produced by a parametric distribution with specific parameters (such as a multivariate normal distribution). An example of this is the Z-score, which identifies outliers based on how many standard deviations an observation lies from the mean.
- **Nonparametric Methods:** These do not assume a predefined model but instead tailor the detection to the data itself. Common techniques include histograms, which track the frequency of specific attribute values, or k-nearest neighbor graphs that measure the relative concentration of data points in a given space.

### Machine Learning Methods:

Machine Learning (ML) methods enhance detection capabilities by identifying complex, non-linear relationships within high-dimensional network telemetry that are often invisible to traditional statistical tests. Unlike standard rules, ML models can automatically learn the natural "groups" that underlie network data. This project evaluates several prominent models:

- Isolation Forest: An unsupervised approach that isolates anomalies by randomly selecting features and split values, effectively "partitioning" outliers more quickly than normal points.
- One-Class SVM: A weakly supervised method that defines a decision hyperplane surrounding normal training examples with the largest possible margin, flagging any point outside this boundary as anomalous.
- Local Outlier Factor (LOF): A density-based algorithm that assigns an anomaly score based on how isolated a data point is relative to its local neighborhood.

### Unknown Attack Detection

The primary advantage of these approaches is their efficacy in unknown attack detection. Because anomaly-based systems do not search for specific "guilty" patterns but rather for anything that is "not normal," they are capable of identifying zero-day exploits and polymorphic malware that lack existing signatures. By monitoring behavioral patterns rather than static rules, these models provide a robust defense against the continuous evolution of cyber threats.

## 1.6 Motivation

The primary motivation for this project stems from the critical reality that Distributed Denial of Service (DDoS) attacks continue to evolve in both scale and sophistication, with recorded attack volumes recently reaching staggering peaks of 2.3 Tbps and 2.5 Tbps. The landscape of cybercrime has been further transformed by the rise of "DDoS as a Service," where powerful botnets are available for hire on the dark web, enabling high-magnitude attacks to be launched with minimal technical effort.

Modern organizations remain highly vulnerable because traditional signature-based systems struggle against zero-day attacks and polymorphic malware. While these reactive systems are accurate at identifying known threats, they are fundamentally incapable of recognizing novel or modified attack patterns that have no pre-existing entries in a signature database. This limitation necessitates the development and refinement of anomaly-based detection systems that can identify malicious activity by establishing a baseline of "normal" behavior and flagging significant deviations.

## 1.7 Problem Statement

Traditional signature-based intrusion detection techniques are increasingly ineffective in the modern threat landscape because they rely on predefined patterns and are fundamentally incapable of identifying previously unseen or "zero-day" attacks. While anomaly-based detection offers a proactive alternative by establishing a baseline of normal behaviour, significant technical challenges remain in selecting the optimal modelling paradigm. Statistical approaches, such as

the Z-score, are computationally efficient and simple to implement but may fail to capture the non-linear, complex attack patterns inherent in high-dimensional network traffic. Conversely, unsupervised Machine Learning algorithms—specifically Isolation Forest (IF), One-Class SVM (OCSVM), and Local Outlier Factor (LOF)—offer sophisticated, adaptive capabilities for identifying outliers in dense data clusters.

However, there is a critical lack of systematic investigation comparing the performance of these distinct statistical and machine learning methodologies under identical preprocessing and experimental conditions. Furthermore, many existing evaluations rely on outdated datasets that do not reflect the volume or variety of contemporary malicious activity. To address this gap, this project utilizes the CICIDS2017 Friday DDoS Dataset, which provides realistic, high-fidelity network telemetry for evaluating these models against one of the most damaging forms of network anomalies. The core problem addressed is determining which of these models—the classical Z-score or the advanced IF, OCSVM, and LOF—provides the most robust and reliable detection performance for securing modern enterprise infrastructures.

## 1.8 Objectives

- Study network anomaly detection: Comprehensively analyse the theoretical shift from reactive signature-based systems to proactive behavioural modelling, which establishes a baseline of "normal" network activity to identify deviations.
- Study DDoS attacks: Investigate the mechanisms of Distributed Denial of Service (DDoS) attacks, focusing on how botnets coordinate to cause resource exhaustion and service downtime in modern infrastructures.
- Implement Z-score anomaly detection: Deploy the Z-score statistical parametric method to establish a classical baseline that flags outliers based on standard deviations from the mean.
- Implement Isolation Forest: Utilize this unsupervised machine learning approach to isolate anomalies by randomly partitioning features, effectively identifying outliers that are few and different.
- Implement One-Class SVM: Develop a weakly supervised model that defines a decision hyperplane with the largest possible margin to enclose normal traffic patterns, flagging any data point outside this boundary as malicious.
- Implement Local Outlier Factor (LOF): Apply this density-based algorithm to assign an anomaly score based on how isolated a data point is relative to its local neighbourhood, which is particularly effective for high-dimensional network telemetry.
- Compare statistical and machine learning approaches: Evaluate the trade-offs between the computational simplicity of statistical methods and the ability of machine learning models to identify complex, non-linear relationships within traffic streams.
- Evaluate using standard performance indices: Assess the efficacy of each model using the CICIDS2017 Friday DDoS Dataset by measuring key metrics derived from the confusion matrix, specifically: Precision, Recall, F1-score, ROC-AUC, and False Positive Rate (FPR).

## **CHAPTER-2**

### **BACKGROUND THEORY**

#### **2.1 Introduction**

This part presents the theoretical background required to understand network anomaly detection using statistical and machine learning techniques. It explains the nature of network traffic data used as input, the working principle and mathematics of a classical statistical method (the Z-score), and the working principle and mathematics of three unsupervised machine learning methods commonly used for anomaly/novelty detection: Isolation Forest, One-Class Support Vector Machine (OC-SVM), and Local Outlier Factor (LOF). For Isolation Forest and Local Outlier Factor, it also explains, in general terms, how their key hyperparameters are tuned and how such tuning is typically expected to influence detection performance, along with how a confusion matrix is constructed and interpreted for these models. Finally, this part defines the standard performance evaluation metrics - accuracy, precision, recall, F1-score, false positive rate, and ROC/AUC - together with their mathematical formulas.

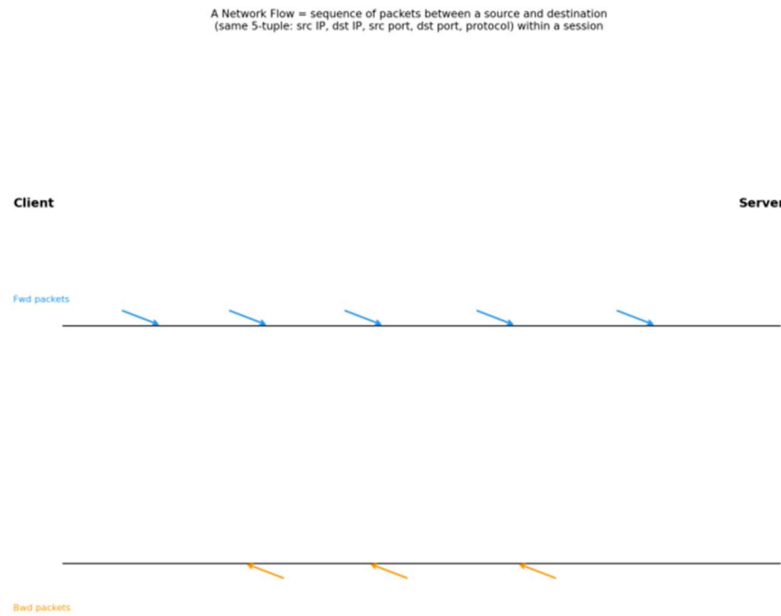
| Symbol                         | Meaning                                                                                          |
|--------------------------------|--------------------------------------------------------------------------------------------------|
| $x_i$                          | The $i$ -th observation (flow feature vector) in the dataset.                                    |
| $x_{ij}$                       | The value of feature $j$ for observation $i$ .                                                   |
| $N$                            | The number of observations (flows) in a dataset or sample.                                       |
| $\mu, \sigma$                  | Sample mean and sample standard deviation of a feature.                                          |
| $Z_i$                          | The Z-score (standardised value) of observation $i$ .                                            |
| $T$                            | A decision threshold applied to a score to obtain a binary label.                                |
| $h(x), E[h(x)]$                | Path length of $x$ in a single isolation tree, and its average across all trees.                 |
| $s(x, n)$                      | Isolation Forest anomaly score for point $x$ given a forest built from $n$ points.               |
| $\phi(\cdot), K(\cdot, \cdot)$ | Kernel feature mapping and the corresponding kernel function.                                    |
| $\nu, \gamma$                  | One-Class SVM hyperparameters: outlier-fraction bound ( $\nu$ ) and RBF kernel $\gamma$ (gamma). |
| $k, N_k(x)$                    | Number of neighbours used by LOF, and the $k$ -nearest-neighbour set of point $x$ .              |
| $LRD_k(x)$                     | Local reachability density of point $x$ with respect to $k$ neighbours.                          |
| TP, TN, FP, FN                 | True Positive, True Negative, False Positive and False Negative counts.                          |

*Table 2.0: Summary of mathematical notation used throughout this report.*



## 2.2 Network Traffic: Packets and Flow Features

Modern network intrusion and anomaly detection systems rarely operate on raw packets in isolation. Instead, packets belonging to the same communication session are aggregated into a network flow, defined as a unidirectional or bidirectional sequence of packets sharing the same 5-tuple: source IP address, destination IP address, source port, destination port, and transport protocol, observed within a bounded time window. Flow-based analysis is computationally lighter than deep packet inspection, does not require reading packet payloads (which may be encrypted), and captures behavioural statistics that are highly discriminative between benign and malicious traffic.



*Fig.2.1: A network flow as a bidirectional sequence of forward and backward packets between a client and server.*

Tools such as CICFlowMeter convert raw packet captures (.pcap files) into flow records described by a large number of statistical features computed over each flow. Broadly, these features fall into the following categories:

- Flow identification features: destination port, transport protocol.
- Volume/count features: total forward packets, total backward packets, total length of forward/backward packets.
- Packet size statistics: minimum, maximum, mean and standard deviation of forward and backward packet lengths, average packet size.
- Timing (Inter-Arrival Time, IAT) features: flow duration, flow IAT mean/std/max/min, forward IAT total/mean/std, backward IAT mean/std.
- Rate features: Flow Bytes/s, Flow Packets/s, forward/backward packets per second.

- TCP flag counts: SYN, ACK, PSH, URG, FIN, RST flag counts, which indicate connection establishment, teardown and control behaviour.
- Window and header features: forward/backward header length, initial window bytes.
- Bulk/subflow features: average bytes per bulk transfer, subflow packet/byte counts, and idle/active time statistics.

Each flow is therefore represented as a numerical feature vector  $x = (x_1, x_2, \dots, x_k)$ , typically comprising several dozen such statistics. This feature vector is the raw input space from which anomalous (attack) behaviour must be separated from benign behaviour, either through a statistical decision rule or a learned model because these features have very different scales (e.g., a duration measured in microseconds versus a flag count between 0 and a few units), it is standard practice to standardise each feature - subtracting the mean and dividing by the standard deviation - before applying either statistical or machine learning based anomaly detection. This standardisation step is, in fact, mathematically identical to the Z-score transformation.

### **Why Flow Features Are Suited to Anomaly Detection**

Different classes of network attacks distort flow-level statistics in characteristic ways. A flooding-style Denial-of-Service or Distributed Denial-of-Service (DDoS) attack, for example, tends to produce flows with an unusually large number of packets per second, abnormally short inter-arrival times, and a skewed ratio of forward to backward packets, because the attacker sends a high volume of requests without waiting for or completing a normal handshake/response cycle. A port scan produces many short flows to different destination ports with very few packets each. Because such deviations manifest as statistical outliers relative to the bulk of normal traffic, flow-level features are naturally amenable to both statistical thresholding and unsupervised/novelty-detection machine learning, both of which operate on the general principle of identifying observations that deviate substantially from the majority pattern.

## **2.3 Preprocessing and Feature Scaling**

Before any statistical or machine learning based anomaly detection can be applied to flow features, several general preprocessing steps are typically required. First, invalid values must be handled: rate-based features such as Flow Bytes/s can become undefined (infinite) for flows with near-zero duration, and such values are usually converted to missing (NaN) and either imputed or dropped. Second, duplicate flow records are typically removed, since exact duplicates can bias density- and distance-based methods by artificially inflating the apparent density around repeated points.

Third, near-constant features (columns whose value barely varies across the dataset) carry little discriminative information and are often dropped to reduce dimensionality. Fourth, highly correlated features (for example, a pair of features that are near-perfect linear functions of one another) are often pruned, since redundant dimensions add computational cost without adding information, and can disproportionately affect distance-based methods such as LOF.

Finally, because flow features are measured on very different natural scales (for instance, a

duration in microseconds can range into the millions, while a TCP flag count is typically 0 or 1), feature scaling is applied so that no single feature dominates a distance or boundary computation purely because of its numeric magnitude. The most common scaling approach, standardization, transforms each feature to zero mean and unit variance:

$$x'_{ij} = (x_{ij} - \mu_j) / \sigma_j$$

where  $\mu_j$  and  $\sigma_j$  are the mean and standard deviation of feature  $j$  computed over the training data, and  $x'_{ij}$  is the standardized value used in place of the raw feature  $x_{ij}$ . This is precisely the Z-score transformation, which highlights an important conceptual link between the two halves of this report: the same Z-score computation that forms a complete, standalone anomaly detector on its own is also used, as a preprocessing step, to place features on a comparable scale before they are passed into the distance-based (LOF), boundary-based (OC-SVM), and partition-based (Isolation Forest) machine learning methods described in the remainder of this report.

## 2.4 Statistical Anomaly Detection: Z-Score Method

The simplest and most established approach to anomaly detection is the statistical Z-score (standard score) method. It assumes that, for a given feature, the values taken by normal observations are approximately normally (Gaussian) distributed around a mean, and that observations lying unusually far from this mean, measured in units of standard deviation, are anomalous.

### 2.4.1 Mean and Standard Deviation

For a feature  $x$  observed over  $n$  training samples, the sample mean  $\mu$  and sample standard deviation  $\sigma$  are defined as:

$$\mu = (1/n) \sum_{i=1}^n x_i$$

$$\sigma = \sqrt{(1/n) \sum_{i=1}^n (x_i - \mu)^2}$$

The mean  $\mu$  represents the “central” or typical value of the feature under normal conditions, while the standard deviation  $\sigma$  quantifies how much individual observations typically vary around that centre. A small  $\sigma$  indicates that normal traffic is tightly clustered around  $\mu$  for that feature, so even a modest deviation is statistically unusual; a large  $\sigma$  indicates that normal traffic is naturally spread out, so a larger absolute deviation is required before an observation is considered unusual.

### 2.4.2 Z-Score and Anomaly Score Calculation

Given  $\mu$  and  $\sigma$  estimated from (normal) training data, the Z-score of any observation  $x_i$  is defined as:

$$Z_i = (x_i - \mu) / \sigma$$

The Z-score expresses how many standard deviations an observation lies from the mean, and can be interpreted directly as an anomaly score for that single feature: a Z-score of 0 means the

observation is exactly at the mean, while a large positive or negative Z-score means the observation is far from the typical value of normal traffic. Because a network flow is described by many features simultaneously ( $x_i = (x_{i1}, x_{i2}, \dots, x_{ik})$ ), an aggregate flow-level anomaly score must combine the individual per-feature Z-scores. A common and simple aggregation rule, used in this project, is to take the maximum absolute Z-score across all features of a flow:

$$ScoreZ(x_i) = \max_j |Z_{ij}| = \max_j |(x_{ij} - \mu_j) / \sigma_j|$$

where  $j$  indexes the features of the flow. A flow is then classified as anomalous if this aggregate score exceeds a chosen threshold  $t$ :

$$Anomaly(x_i) = 1 \text{ if } ScoreZ(x_i) > t, \text{ else } 0$$

This decision rule is a per-feature, univariate rule combined with a logical OR across dimensions: a flow is flagged as anomalous if it is extreme in even a single feature, even if all its other features look perfectly normal.

Other aggregation choices are possible, such as averaging the squared Z-scores across features (analogous to computing a simplified Mahalanobis-style distance under the assumption of feature independence), but the max-based rule is the most common and interpretable baseline.

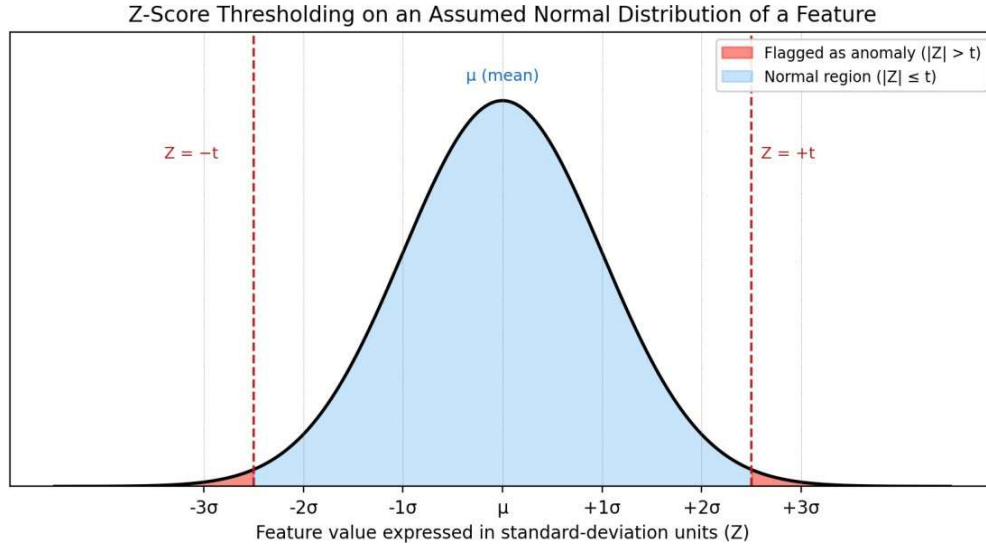


Fig.2.2: Z-score thresholding under the assumed Gaussian distribution of a feature. Observations falling in the shaded tail regions, beyond  $\pm t$  standard deviations from the mean, are flagged as anomalous.

### 2.4.3 Worked Example (Illustrative)

To make the calculation concrete, consider a single hypothetical feature - say, the number of packets per second in a flow - for which the mean and standard deviation estimated from a sample of normal traffic are  $\mu = 120$  and  $\sigma = 30$  (units omitted for simplicity; these are illustrative numbers only, not derived from any specific dataset). Suppose a new flow arrives with a value of  $x = 285$  packets per second for this feature. Its Z-score is calculated as:

$$Z = (285 - 120) / 30 = 165 / 30 = 5.5$$

A Z-score of 5.5 means the observed value is 5.5 standard deviations above the mean - an extremely large deviation under the Gaussian assumption, where values beyond  $\pm 3\sigma$  are already expected in less than about 0.3% of normal observations (Table 3.1). If the chosen threshold is, for instance,  $t = 3.0$ , this flow would be flagged as anomalous, since  $|Z| = 5.5 > 3.0$ . Conversely, a flow with  $x = 145$  would give  $Z = (145 - 120)/30 = 0.83$ , which lies well within  $\pm t$  for any of the thresholds listed in Table 3.1, and would therefore be classified as normal. This simple example illustrates the general calculation; in an actual flow record with dozens of features, this computation is repeated for every feature, and the aggregate score  $\text{ScoreZ}(x_i)$  combines all of them into a single decision for the flow.

#### 2.4.4 Threshold Selection

The choice of threshold  $t$  directly controls the trade-off between sensitivity (recall) and false alarms (precision). Under the assumption that a feature is normally distributed, the threshold can be related to a theoretical tail probability using the properties of the standard normal distribution:

| Threshold ( $t$ ) | Theoretical fraction of normal data beyond $\pm t$ (single feature) | Typical interpretation                                                      |
|-------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------|
| $t = 1.0$         | $\approx 31.7\%$                                                    | Very loose - flags roughly one in three observations; rarely used alone.    |
| $t = 1.5$         | $\approx 13.4\%$                                                    | Loose threshold; higher sensitivity but many false alarms expected.         |
| $t = 2.0$         | $\approx 4.6\%$                                                     | Commonly used moderate threshold.                                           |
| $t = 2.5$         | $\approx 1.2\%$                                                     | Stricter threshold; fewer flows flagged.                                    |
| $t = 3.0$         | $\approx 0.3\%$                                                     | Classic “three-sigma rule”; very strict, used when false alarms are costly. |

Table 2.1: Theoretical tail probabilities of the standard normal distribution for common Z-score thresholds.

A smaller threshold flags more points as anomalies (because a smaller deviation from the mean is enough to cross it), which tends to increase recall (more true attacks are caught) but reduce precision (more benign flows are incorrectly flagged). A larger threshold is stricter and tends to increase precision at the cost of recall, since only the most extreme deviations are flagged, potentially missing subtler attacks.

In practice, because real network traffic features are rarely perfectly Gaussian, the theoretical percentages in Table 3.1 serve only as a rough guide, and the threshold is usually selected empirically

- for example, by evaluating a small set of candidate thresholds (such as  $t = 2.0, 2.5, 3.0$ ) against a validation set and choosing the value that best balances precision and recall for the application at hand. When labelled validation data is not available, thresholds are sometimes chosen based on domain knowledge or set so that the expected false alarm rate stays within an operationally acceptable bound (e.g. no more than a few alerts per hour).

## 2.4.5 Advantages and Disadvantages

### Advantages

- Extremely fast to compute (a single pass to estimate  $\mu$  and  $\sigma$ , and simple arithmetic thereafter) and requires no iterative model training or hyperparameter search in the machine-learning sense.
- Simple to interpret: an analyst can directly see by how many standard deviations a feature deviates from normal, which aids explainability of alerts.
- No labelled anomalies are required; only a representative sample of normal traffic is needed to estimate  $\mu$  and  $\sigma$ , making it usable in purely unsupervised settings.
- Scales linearly with the number of observations and features, making it suitable as a lightweight, real-time first-pass filter.

### Disadvantages

- Assumes each feature is approximately Gaussian-distributed. Network traffic features such as Flow Bytes/s, Flow Duration and packet counts are typically heavily right-skewed, multi-modal, or heavy-tailed, violating this assumption and reducing the reliability of the resulting anomaly score.
- Treats each feature independently (univariate), and therefore cannot capture correlations or joint patterns across multiple features that may characterise more subtle attacks - an attack that is only unusual in the combination of two features, but not in either feature alone, will not be detected.
- Sensitive to threshold choice; there is no single principled, dataset-independent way to select  $t$ , and the optimal threshold can vary by dataset, feature, and attack type.
- Vulnerable to the very outliers it is trying to detect: because  $\mu$  and  $\sigma$  are computed from the data itself, a large fraction of anomalous points in the training sample can distort the estimated mean and standard deviation, artificially inflating  $\sigma$  and masking genuine anomalies (a phenomenon sometimes called the masking effect); conversely, extreme individual anomalies can also cause otherwise normal points to be labelled anomalous (the swamping effect).

## 2.5 Machine Learning Methods for Anomaly Detection

To address the limitations of the univariate Z-score method, unsupervised machine learning algorithms capable of modelling multi-dimensional structure in the data are commonly used for network anomaly detection. Three such algorithms are described in this report: Isolation Forest, One-Class Support Vector Machine, and Local Outlier Factor. All three can be operated in a novelty-detection configuration, in which the model is fitted only on data known (or assumed) to be normal, and is then used to classify new, unseen observations as normal or anomalous. This configuration is particularly well suited to network security applications, where genuinely labelled attack traffic is scarce, but a representative sample of benign traffic is comparatively easy to

obtain. Each method represents a distinct algorithmic philosophy: Isolation Forest is partition-based, One-Class SVM is boundary-based, and Local Outlier Factor is density-based - they are described in turn below, along with the mathematics underlying each, followed by a discussion of how their principal hyperparameters are typically tuned and how a confusion matrix is used to characterize their behavior.

### **2.5.1 Outlier Detection versus Novelty Detection**

It is useful to distinguish two related but distinct unsupervised problem settings that these algorithms can be applied to. In outlier detection, a single dataset containing a mixture of normal and (a presumably small proportion of) anomalous points is available, and the goal is to identify which points within that same dataset are anomalous; the model is fitted and scored on the same data. In novelty detection, the model is fitted only on data that is clean (or assumed clean) of anomalies, and is then used to score new, previously unseen observations - the goal is to decide whether each new observation resembles the training distribution (“normal”) or departs from it (“novel”/anomalous). Network anomaly detection is generally framed as a novelty-detection problem, since a sample of established, everyday network traffic is comparatively easy to collect and reasonably assumed to be free of attacks, while attack traffic is by definition rare, evolving and not fully known in advance. All three algorithms in this report - Isolation Forest, One-Class SVM and Local Outlier Factor - support this novelty-detection mode of operation, in which the training phase uses only normal traffic and the resulting model is then applied to classify subsequently observed flows.

### **2.5.2 Choosing Among the Three Methods**

Because Isolation Forest, One-Class SVM and Local Outlier Factor rest on different assumptions about what makes a point anomalous, they are complementary rather than strictly interchangeable, and the choice between them is generally guided by the characteristics of the data and the operational constraints of the deployment:

- If the anomalous class is expected to be globally sparse (few points, clearly separated from the bulk of the data across many dimensions) and computational efficiency at scale is important, a partition-based method such as Isolation Forest is a natural first choice, since its linear-time construction scales well to large volumes of flow data.
- If the normal class is expected to occupy a complex, non-convex but relatively compact region of feature space, and the dataset is small or moderate in size, a boundary-based method such as One-Class SVM can learn a tight, non-linear enclosure of that region, at higher computational cost.
- If anomalies are expected to be local - unusual relative to a nearby cluster of similar traffic, even though they might not be globally rare - a density-based method such as Local Outlier Factor is best suited, since it explicitly compares each point to its own neighborhood rather than to the dataset as a whole.

In practice, because the true structure of anomalies in a given network is rarely known in advance, it is common to evaluate more than one of these methods and compare their behavior using the

performance metrics, rather than committing to a single method purely on theoretical grounds.

### 2.5.3 Isolation Forest

Isolation Forest (iForest) is an ensemble, tree-based anomaly detection algorithm proposed by Liu, Ting and Zhou. Unlike distance-based or density-based methods, which characterise what is normal and measure deviation from it, Isolation Forest directly exploits the fact that anomalies are “few and different”: they are rare and have feature values that differ substantially from the majority of the data, making them easier to isolate from the rest of the points through random partitioning.

#### 2.5.3.1 Random Partitioning and the Isolation Path

An isolation tree (iTree) is built by recursively partitioning the data. At each node, a feature is selected at random, and a split value is chosen uniformly at random between the minimum and maximum value of that feature within the current node's data. This partitioning continues, splitting the data into two child nodes, until every point is isolated in its own leaf, or a maximum tree depth is reached. Because splits are chosen randomly (not to optimise class separation as in a decision tree classifier), points that are easy to separate from the rest of the data - i.e., anomalies - tend to be isolated after only a few splits, resulting in a short path from the root of the tree to the leaf containing that point. Normal points, which are surrounded by many similar points, require many more splits before they are isolated, resulting in a longer path length.

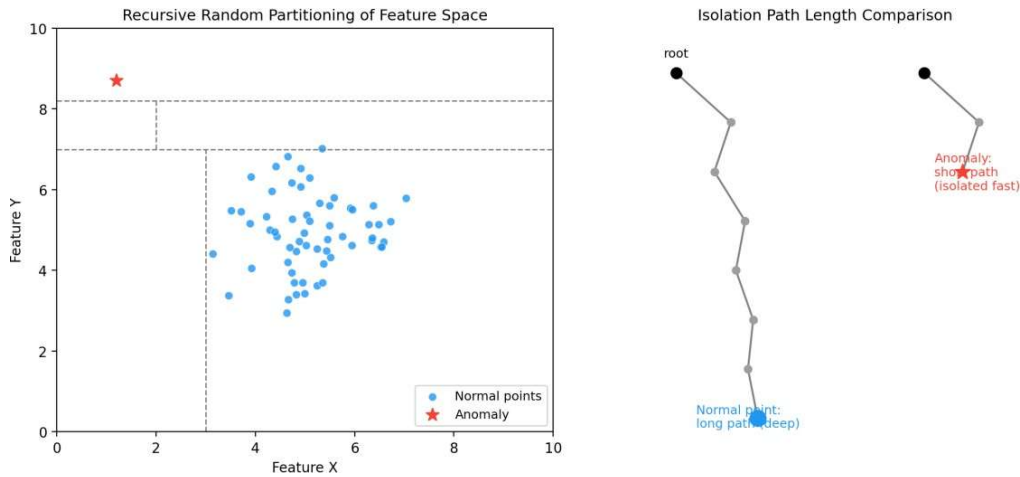


Fig.2.3: Left - random axis-aligned partitioning of the feature space isolates a sparse anomaly (red star) quickly. Right - the corresponding isolation-tree path length is short for the anomaly and long for a normal point buried in a dense region.

#### 2.5.3.2 Anomaly Score Formula

An Isolation Forest is an ensemble of many such isolation trees, each built on a random sub-sample of the training data. For a point  $x$ , let  $h(x)$  denote the path length (number of edges traversed from the root to the leaf containing  $x$ ) in a single isolation tree, and let  $E[h(x)]$  denote the average of  $h(x)$  across all trees in the forest. Because path length grows unbounded with sample size, it is normalised by  $c(n)$ , the average path length of an unsuccessful search in a Binary Search Tree built from  $n$  points, given by:



$$c(n) = 2 H(n-1) - (2(n-1)/n)$$

where  $H(i)$  is the harmonic number, approximated as  $H(i) \approx \ln(i) + 0.5772156649$  (the Euler-Mascheroni constant). The anomaly score for a point  $x$  is then defined as:

$$s(x, n) = 2^{-E[h(x)] / c(n)}$$

This score lies in the range  $(0, 1]$ . When  $E[h(x)]$  approaches 0 (the point is isolated almost immediately),  $s(x, n)$  approaches 1, indicating a high likelihood of being anomalous. When  $E[h(x)]$  approaches  $c(n)$  (the point takes as long to isolate as expected for a typical point),  $s(x, n)$  approaches 0.5, indicating a normal point. Scores substantially below 0.5 indicate that  $E[h(x)]$  is larger than expected, which corresponds to points that are especially well-embedded within the data. Scikit-learn's implementation additionally exposes a `decision_function` that shifts and negates this score, such that negative values correspond to anomalies and positive values correspond to normal points, with the exact threshold set by the contamination parameter described next.

### 2.5.3.3 Parameter Tuning

Isolation Forest has a small number of hyperparameters that govern the structure of the ensemble and, consequently, the sensitivity of the resulting anomaly scores. The most influential of these are described below.

| Hyperparameter | Description                                               | Typical Effect on Behaviour                                                                                                                                                                                                                                                                                                                                |
|----------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| contamination  | The expected proportion of anomalies in the data.         | Sets the decision threshold on the anomaly score $s(x, n)$ : $(\text{contamination} \times 100)\%$ of points with the lowest scores labelled anomalous. A larger value causes more points to be flagged as anomalous (increasing recall, typically decreasing precision); a smaller value is stricter (typically increasing precision, decreasing recall). |
| n_estimators   | The number of isolation trees in ensemble.                | More trees generally produce a more stable, less noisy estimate of $E[h(x)]$ , at the cost of additional computation. Performance typically improves with more trees up to a point of diminishing returns, after which additional trees add computational cost without materially changing the score.                                                      |
| max_samples    | The number of samples drawn to build each isolation tree. | Smaller sub-samples build shallower, more diverse trees and can improve the algorithm's ability to isolate anomalies (since each tree sees a less crowded feature space), while also reducing memory and computation per tree.                                                                                                                             |
| max_features   | The number of features considered at each split.          | Controls how many of the available features are candidates for the random split at each node; using fewer features at split increases tree diversity within the ensemble.                                                                                                                                                                                  |

Table 2.2: Key Isolation Forest hyperparameters and their general effect on detection behaviour.

Because contamination directly sets the fraction of points labelled anomalous, it has the largest

direct influence on the resulting confusion matrix and is usually the first parameter tuned, typically via a grid search over a small candidate set (e.g. 0.05, 0.1, 0.2, 0.3, 0.5) evaluated using a chosen metric such as F1-score. `n_estimators` and `max_samples` are generally tuned afterwards, with the goal of stabilising the score estimate rather than shifting the overall operating point.

#### 2.5.3.4 Effect of Contamination on Precision, Recall and F1-Score

Because the contamination parameter directly sets what fraction of the scored population is labelled anomalous, sweeping it across a range of values traces out a general trade-off between precision and recall, analogous to moving the decision threshold of any binary classifier. Figure 4 illustrates the generic shape of this trade-off conceptually: as contamination increases, the model flags more points as anomalies, so recall (the fraction of true anomalies captured) tends to rise, while precision (the fraction of flagged points that are truly anomalous) tends to fall, because progressively more borderline or normal points get swept into the flagged set. The F1-score, being the harmonic mean of the two, typically traces an inverted-U shape with a maximum at some intermediate contamination value, representing the best balance between the two competing objectives for that particular dataset.

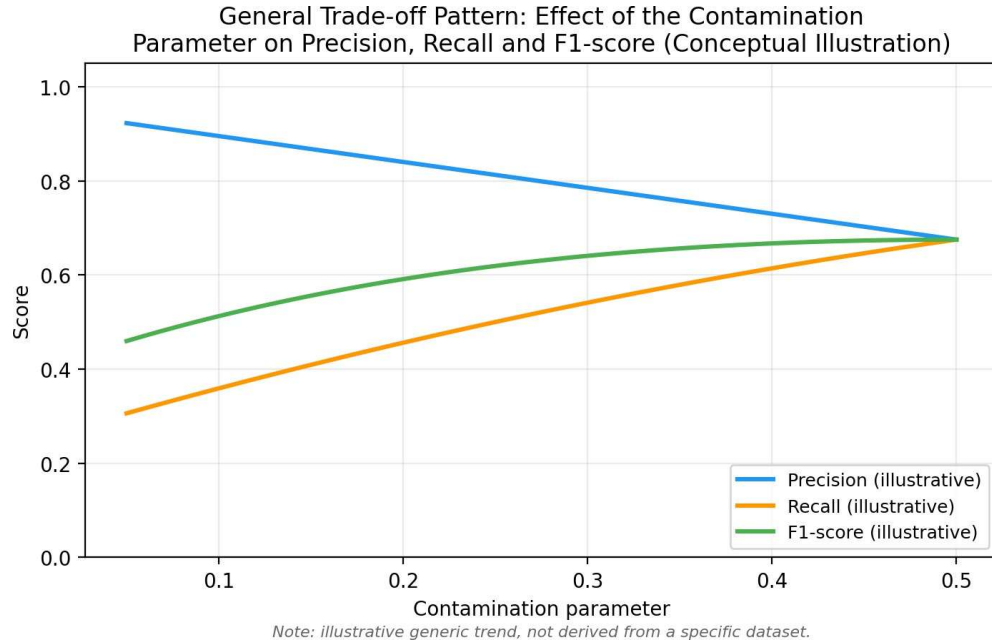


Fig.2.4: Conceptual illustration of the general trade-off pattern between Precision, Recall and F1-score as the contamination parameter is increased. Curve shapes are illustrative only.

In practice, the exact shape and turning point of these curves depend heavily on the true proportion of anomalies present in the evaluation data and on how well-separated the anomalous and normal classes are in feature space; the qualitative trend described above (precision decreasing, recall increasing, F1 peaking at some intermediate value) is nonetheless a useful general expectation to guide the search for a good contamination setting.

### 2.5.3.5 Confusion Matrix and Interpretation

As with any binary anomaly detector, the output of Isolation Forest at a given contamination setting can be summarised in a  $2 \times 2$  confusion matrix that cross-tabulates the predicted label (normal/anomaly) against the true label (BENIGN/attack):

|                 | Predicted: Normal   | Predicted: Anomaly  |
|-----------------|---------------------|---------------------|
| Actual: Normal  | TN (True Negative)  | FP (False Positive) |
| Actual: Anomaly | FN (False Negative) | TP (True Positive)  |

Table 2.3: Generic confusion-matrix template used to evaluate Isolation Forest predictions at a given contamination setting.

This template underlies several commonly discussed aspects of an Isolation Forest configuration:

- Best contamination: the contamination value that yields the confusion matrix (and derived metrics, particularly F1-score) judged most suitable for the application - typically the value that best balances the number of true positives against the number of false positives, rather than simply the value that maximizes recall or precision alone.
- Number of detected attacks: the True Positive (TP) count in the confusion matrix - the number of genuinely anomalous flows that the model correctly flags. This is the raw quantity underlying the recall metric.
- False positives: the False Positive (FP) count - the number of genuinely normal flows incorrectly flagged as anomalous. A high FP count degrades precision and, in an operational setting, translates directly into wasted analyst time investigating false alarms.
- Detection capability: a qualitative summary of how completely and how cleanly the model separates the two classes, generally read off the confusion matrix as a combination of a high TP count (few missed attacks, i.e. few False Negatives) together with a low FP count (few false alarms). No single cell of the confusion matrix captures this alone; it is instead judged from the overall balance of TP, FP and FN, often summarized numerically via precision, recall and F1-score together.

Because contamination controls the overall number of points flagged, sweeping it traces out a family of confusion matrices along the precision/recall trade-off: low contamination values shift probability mass from the “Predicted: Anomaly” column into the “Predicted: Normal” column (fewer FP and fewer TP, more FN), while high contamination values do the opposite (more FP and more TP, fewer FN).

### 2.5.3.6 Computational Complexity

One of the most cited practical strengths of Isolation Forest is its favourable computational complexity, which follows directly from the way trees are built. Because each isolation tree is grown on a sub-sample of size  $\psi$  (max\_samples) rather than the full training set, and because a binary tree built from  $\psi$  points has average depth  $O(\log \psi)$ , the time complexity of building a single tree is  $O(\psi \log \psi)$ , and building an ensemble of  $n\_estimators = T$  such trees costs  $O(T \psi \log \psi)$  - independent of the total size of the training set  $N$ , since  $\psi$  is typically fixed at a small constant

(e.g. 256) regardless of how large  $N$  is. Evaluating (scoring) a new point requires traversing each of the  $T$  trees from root to leaf, an  $O(T \log \psi)$  operation per point, and  $O(N T \log \psi)$  for scoring an entire dataset of  $N$  points. This is markedly more favourable than the complexity of distance-based methods such as LOF or the quadratic-programming formulation underlying OC-SVM training, both of which scale less gracefully with the number of training points, which is part of why Isolation Forest is often favoured for large-scale or near-real-time network monitoring.

### 2.5.3.7 Advantages and Limitations

#### Advantages

- Linear time complexity and low memory requirement,  $O(n)$  per tree, since only a small sub-sample is required to build each tree; scales well to large flow datasets.
- Does not rely on distance or density computations, so it avoids the curse of dimensionality that affects distance-based methods such as LOF.
- Does not require the data to follow any particular distribution (unlike the Z-score method).
- Naturally handles high-dimensional data, since splits are chosen from a random subset of features rather than requiring a full distance computation across all dimensions.

#### Limitations

- Performance is sensitive to the contamination hyperparameter, which must be estimated or tuned; if the true anomaly ratio is unknown, an ill-chosen value can significantly degrade precision or recall.
- Uses axis-aligned random splits, so it can struggle with anomalies that are only separable via oblique or non-linear combinations of features, or that lie within local clusters rather than being globally sparse.
- Can be less effective when normal and anomalous classes overlap substantially in the feature space, since some genuinely anomalous points may still require many splits to isolate if they resemble the bulk of the data along the randomly chosen features.

## 2.5.4 One-Class Support Vector Machine (OC-SVM)

The One-Class Support Vector Machine, introduced by Schölkopf et al., adapts the standard Support Vector Machine to the unsupervised, single-class setting. Instead of separating two labelled classes with a maximum-margin hyperplane, OC-SVM is trained using only normal data and learns a decision boundary that encloses the region of the feature space occupied by normal traffic, while separating it from the origin (or, equivalently, from the rest of the space) with maximum margin. Any point that falls outside this learned boundary at test time is classified as a novelty/anomaly.

### 2.5.4.1 Hyperplane, Kernel and Novelty Detection

Given training vectors  $x_i$  mapped into a high-dimensional feature space via a mapping  $\phi$ , OC-SVM

solves the following optimisation problem to find a hyperplane that separates the mapped data from the origin with maximum margin  $\rho$ , while allowing a controlled fraction of training points to lie on the wrong side (slack variables  $\xi_i$ ):

$$\min (1/2)\|w\|^2 + (1/(v n)) \sum \xi_i - \rho \quad \text{subject to } (w \cdot \phi(x_i)) \geq \rho - \xi_i, \xi_i \geq 0$$

The decision function is  $f(x) = \text{sign}((w \cdot \phi(x)) - \rho)$ . A point is classified as normal if  $f(x) > 0$  and as an anomaly if  $f(x) < 0$ . In practice, the mapping  $\phi$  is never computed explicitly; instead a kernel function  $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$  is used (the “kernel trick”), which allows the optimisation to be expressed and solved purely in terms of pairwise similarities between points. The most commonly used kernel for this purpose is the Radial Basis Function (RBF) kernel:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

The RBF kernel implicitly maps the data into an infinite-dimensional space, allowing OC-SVM to learn a non-linear, closed decision boundary around the normal data cluster, rather than being restricted to a flat hyperplane in the original feature space. This flexibility is important for network traffic data, whose normal region in feature space is typically irregular and non-convex rather than a simple ellipsoid or half-space

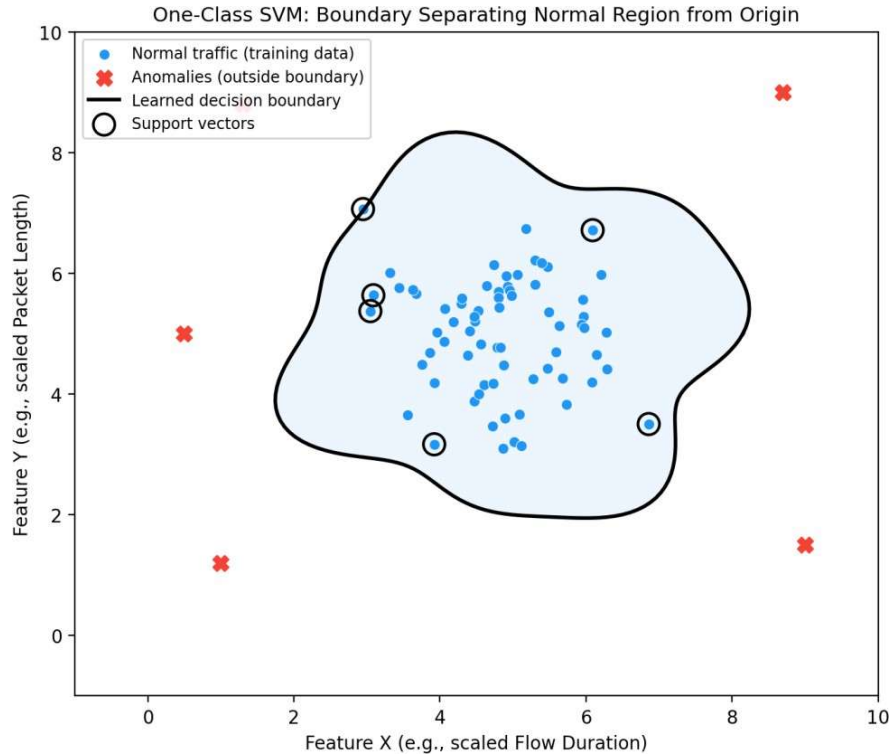


Fig.2.5: One-Class SVM learns a closed, non-linear decision boundary (via the RBF kernel) around the normal traffic cluster; points lying outside the boundary are flagged as anomalies.

### 2.5.4.2 Key Hyperparameters

| Hyperparameter   | Description                                                                                                                                                                | Typical Effect on the Boundary                                                                                                                                                                                                                                                                  |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\nu$ (nu)       | An upper bound on the fraction of training points allowed to lie outside the margin (training errors), and simultaneously a lower bound on the fraction of support vectors | Smaller $\nu$ forces a tighter, stricter boundary that encloses nearly all training points; larger $\nu$ allows a looser boundary that tolerates more training points as outliers, generally shrinking the enclosed normal region.                                                              |
| $\gamma$ (gamma) | Controls the width/reach of the RBF kernel                                                                                                                                 | Small $\gamma$ gives a smoother, more global boundary that treats distant points as similar; large $\gamma$ gives a tighter boundary that follows local structure of training data more closely and can fit highly irregular cluster shapes, at the risk of overfitting to the training sample. |

Table 2.4: Key One-Class SVM hyperparameters and their general effect on the learned decision boundary.

Because OC-SVM is trained exclusively on data assumed to be normal, novelty detection with this model proceeds by fitting it once on a training split of normal (benign) traffic, and then applying the resulting decision function to unseen data at test time; any sample scored on the wrong side of the learned boundary is flagged as a novelty/anomaly. This mirrors the same underlying framework used for Isolation Forest and Local Outlier Factor.

### 2.5.4.3 The Role of Support Vectors

As in the standard two-class SVM, the solution to the One-Class SVM optimisation problem depends on the training data only through a subset of points called support vectors - those training points that lie exactly on the learned boundary ( $\xi_i = 0$ , at the margin) or outside it ( $\xi_i > 0$ , treated as training errors within the allowed  $\nu$  fraction). All other training points, which lie safely inside the boundary with margin to spare, do not influence the final decision function at all; their coefficients in the dual optimisation problem are zero. This is a direct consequence of the Karush-Kuhn-Tucker (KKT) conditions associated with the constrained optimisation. Practically, this property means that (a) the decision boundary is determined by a comparatively small, informative subset of the training data, and (b) the number of support vectors is closely tied to  $\nu$ , since  $\nu$  is simultaneously an upper bound on the fraction of margin errors and a lower bound on the fraction of support vectors - increasing  $\nu$  tends to increase the number of points treated as support vectors.

### 2.5.4.4 Advantages and Limitations

#### Advantages

- Learns a non-linear decision boundary via the kernel trick, well suited to the complex, non-Gaussian shape of normal traffic in feature space.
- Trains using only normal data, matching the practical scenario in network security where

labelled attack examples are scarce but benign traffic is abundant.

- The  $\nu$  parameter provides a direct, interpretable control on the expected fraction of training outliers.

#### Limitations

- Computationally expensive: training and prediction scale poorly with the number of samples, since the underlying quadratic-programming formulation and kernel evaluations grow with the size of the training set.
- Highly sensitive to the choice of  $\nu$  and  $\gamma$ ; a poor kernel-width choice (e.g. very small  $\gamma$ ) can produce an overly smooth boundary that fails to separate normal and attack traffic.
- Does not scale gracefully to very large or high-velocity data streams, limiting its suitability for real-time, high-throughput network monitoring without sub-sampling or approximation.

## 2.5.5 Local Outlier Factor (LOF)

Local Outlier Factor, proposed by Breunig et al., is a density-based anomaly detection method. Unlike Isolation Forest (partition-based) and OC-SVM (boundary-based), LOF measures the local density of a point relative to the local densities of its neighbours, and flags points whose local density is substantially lower than that of their neighbours as outliers. This makes LOF particularly effective at detecting local anomalies - points that are not globally rare, but are unusual relative to their immediate neighbourhood, even in data with clusters of varying density.

### 2.5.5.1 k-Distance, Reachability Distance and Local Density

For a point  $x$  and a chosen number of neighbours  $k$ , the  $k$ -distance of  $x$ , denoted  $k\text{-distance}(x)$ , is the distance from  $x$  to its  $k$ -th nearest neighbour. The  $k$ -distance neighbourhood  $N_k(x)$  is the set of all points within that distance. The reachability distance of  $x$  with respect to a neighbour  $o$  is defined as:

$$\text{reach-dist}_k(x, o) = \max(k\text{-distance}(o), \text{dist}(x, o))$$

The local reachability density (LRD) of  $x$  is the inverse of the average reachability distance of  $x$  from its neighbours:

$$LRD_k(x) = 1 / [ (\sum_{o \in N_k(x)} \text{reach-dist}_k(x, o)) / |N_k(x)| ]$$

Finally, the Local Outlier Factor of  $x$  compares its own local reachability density to the average local reachability density of its neighbours:

$$LOF_k(x) = [ \sum_{o \in N_k(x)} LRD_k(o) / LRD_k(x) ] / |N_k(x)|$$

A LOF value close to 1 indicates a point whose density is comparable to its neighbours (normal). A LOF value substantially greater than 1 indicates a point in a sparser region than its neighbours, i.e., a local outlier. For novelty detection, LOF is fit exclusively on data assumed to be normal, and is then applied to score unseen test points against the fixed neighbourhood structure learned from the training data, as opposed to LOF's default unsupervised outlier-detection mode, in which the model scores the same data it was fitted on.



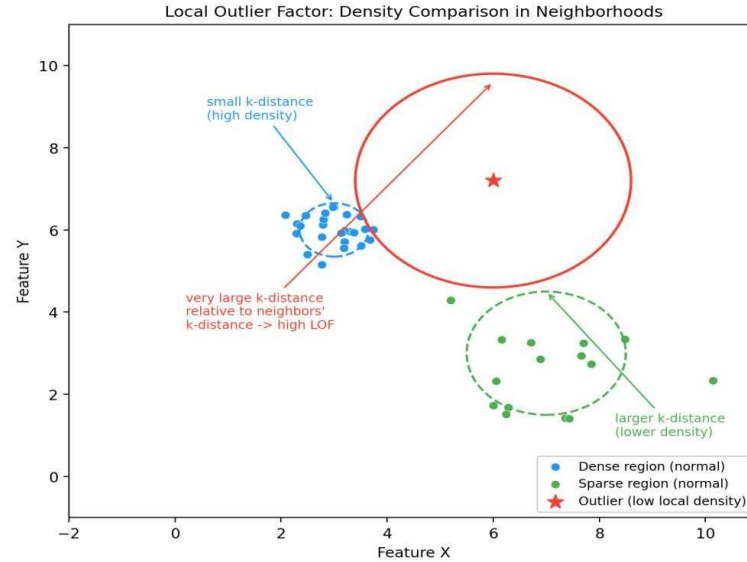


Fig.2.6: Local Outlier Factor compares a point's local density to that of its neighbours. Points in the dense blue region and the sparser (but locally consistent) green region are normal, while the red point has a much larger  $k$ -distance relative to its neighbours and is flagged as an outlier

### 2.5.5.2 Parameter Tuning

| Hyperparameter               | Description                                                                                           | Typical Effect on Behaviour                                                                                                                                                                                                                                                                                                                         |
|------------------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>n_neighbors (k)</code> | The number of neighbours used to compute the local reachability density and LOF score for each point. | Small $k$ makes the density estimate sensitive to individual nearby points (noisy, can overfit local fluctuations); large $k$ smooths the density estimate over a wider neighbourhood, which averages out and miss genuinely local anomalies. The appropriate $k$ is generally chosen relative to the expected size of normal clusters in the data. |
| <code>contamination</code>   | The expected proportion of anomalies used to convert the continuous LOF score into a binary decision. | As with Isolation Forest, this sets the operating point on the score distribution: a larger contamination value flags more points as anomalies (tends to raise recall, lower precision); a smaller value is stricter (tends to raise precision, lower recall).                                                                                      |
| <code>metric</code>          | The distance metric used to compute neighbour distances (e.g. Euclidean, Manhattan, Minkowski).       | Determines how “closeness” is measured between feature vectors; the choice interacts with feature scaling, so standardising features beforehand is important for the metric to behave sensibly across dimensions of different natural scale.                                                                                                        |

Table 2.5: Key Local Outlier Factor hyperparameters and their general effect on detection behaviour.

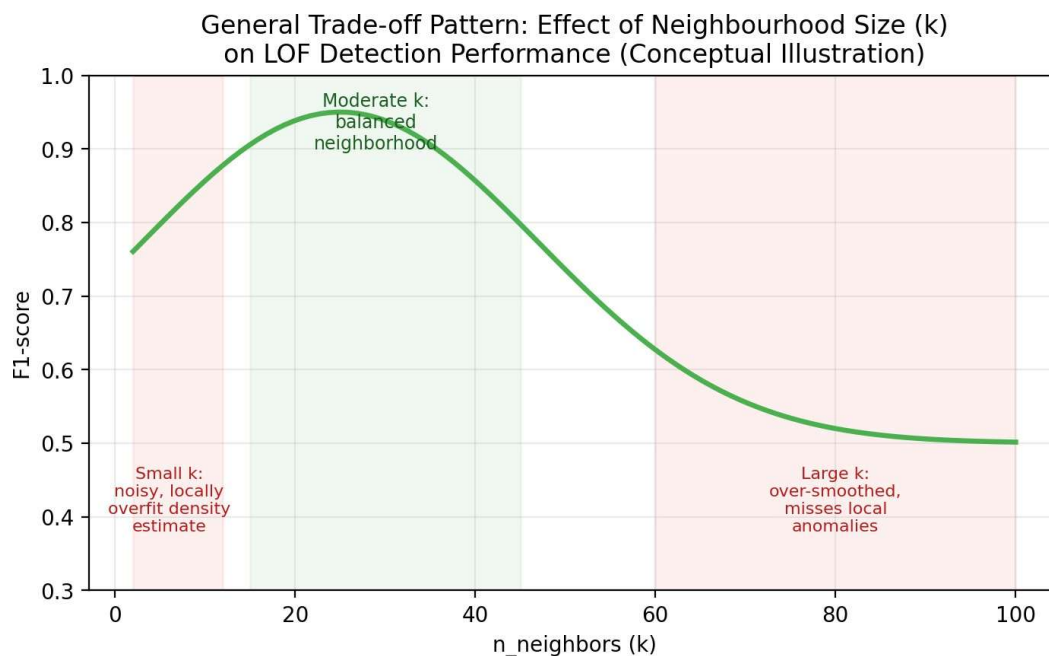
In practice, `n_neighbors` and `contamination` are usually tuned jointly via a grid search, since their

effects interact: a given LOF score distribution (shaped by `n_neighbors`) is then thresholded at a level set by contamination, so the best contamination value for one choice of `n_neighbors` is not necessarily the best for another.

### 2.5.5.3 How Neighbourhood Size Changes Performance

The `n_neighbors` parameter (`k`) governs a classic bias-variance-style trade-off in the local density estimate that LOF relies on. Figure 7 illustrates the general, qualitative shape of this trade-off:

- Small `k`: the local reachability density of each point is estimated from only a handful of nearby points. This makes the estimate noisy and highly sensitive to the exact position of those few neighbours, which can cause LOF to overfit to local fluctuations - normal points near the edge of a cluster may be assigned artificially high LOF scores simply because their few nearest neighbours happen to be somewhat farther away, increasing false positives.
- Large `k`: the neighbourhood used to estimate density becomes large enough to span multiple natural clusters or extend well beyond a genuinely local anomalous region. This over-smooths the density estimate, causing LOF to lose sensitivity to anomalies that are only unusual relative to a small local neighbourhood, increasing false negatives (missed local anomalies), and can also blur the distinction between clusters of genuinely different densities.
- Moderate `k`: an intermediate value, typically informed by the approximate size of normal clusters in the data, tends to give the most balanced local density estimate, and is the region in which detection performance (however measured) tends to be highest.



*Note: illustrative generic trend, not derived from a specific dataset.*

*Fig.2.7: Conceptual illustration of the general trade-off pattern between neighbourhood size ( $n\_neighbors$ ) and detection performance for LOF. Curve shape is illustrative only.*

Because the ideal neighbourhood size depends on how densely normal clusters are populated in the training data, there is no universally correct value of  $k$ ; it is instead typically selected empirically via a small grid search, often jointly with the contamination parameter.

#### 2.5.5.4 Confusion Matrix and Interpretation

As with Isolation Forest, LOF's output at a given ( $n\_neighbors$ , contamination) configuration can be summarised in a  $2 \times 2$  confusion matrix:

|                        | <b>Predicted: Normal</b> | <b>Predicted: Anomaly</b> |
|------------------------|--------------------------|---------------------------|
| <b>Actual: Normal</b>  | TN (True Negative)       | FP (False Positive)       |
| <b>Actual: Anomaly</b> | FN (False Negative)      | TP (True Positive)        |

*Table 2.6: Generic confusion-matrix template used to evaluate Local Outlier Factor predictions at a given hyperparameter configuration.*

As discussed, moving along the  $n\_neighbors$  axis reshapes the underlying LOF score distribution (and hence which points are near the decision boundary), while moving along the contamination axis shifts the threshold applied to that distribution. Jointly, the two parameters determine how mass is distributed between the four cells of the confusion matrix: an appropriately sized neighbourhood combined with a well-chosen contamination value aims to maximise the True Positive and True Negative cells while minimising the False Positive and False Negative cells, though in practice some trade-off between the latter two is generally unavoidable and must be balanced according to the relative cost of missed attacks (False Negatives) versus false alarms (False Positives) in the deployment context.

#### 2.5.5.5 Interpreting the LOF Score (Illustrative)

The reachability-distance formulation is designed specifically so that LOF compares densities rather than raw distances, which is what allows it to handle clusters of different densities correctly. To see why, consider two hypothetical points: point A sits in a dense cluster where all pairwise distances between neighbours are small, and point B sits in a naturally sparser (but still locally consistent) region where neighbours are farther apart on average. A purely distance-based rule (e.g. flagging any point whose  $k$ -distance exceeds some fixed threshold) would incorrectly flag B as anomalous merely because it belongs to an inherently sparser region, even though B is entirely consistent with its own neighbourhood. LOF avoids this failure mode because  $LRD_k(x)$  is a local quantity - it is computed relative to  $x$ 's own neighbours' reachability distances - and the final  $LOF_k(x)$  score is a ratio of  $x$ 's density to its neighbours' density, not an absolute distance. A point that is proportionally about as

dense as its surroundings will have  $LOF \approx 1$  regardless of whether it sits in a dense or a sparse region of the overall dataset, whereas a point that is markedly less dense than its own neighbours - the hallmark of a genuine local outlier - will have LOF noticeably greater than 1.

### 2.5.5.6 Advantages and Limitations

#### Advantages

- Detects local anomalies that global methods can miss, since it compares each point only to its own neighbourhood rather than to the dataset as a whole.
- Naturally handles data with clusters of varying density, which is common in network traffic where different services/ports produce flows with very different statistical footprints.
- The LOF score itself is continuous and interpretable as a relative density ratio, allowing fine-grained ranking of how anomalous a point is, rather than a purely binary label.

#### Limitations

- Computationally expensive at prediction time, since it requires a k-nearest-neighbour search against the training set for every test point, which scales poorly to very large training sets without approximate nearest-neighbour structures.
- Sensitive to the choice of `n_neighbors`: too small a value makes the density estimate noisy, while too large a value smooths over genuinely local anomalies.
- Suffers from the curse of dimensionality: as the number of features grows, distance-based neighbourhood measures become less discriminative (distances between points tend to concentrate), which is why dimensionality reduction or careful feature selection is particularly important for distance-based methods such as LOF.

## 2.6 Performance Evaluation Metrics

Anomaly detection is generally evaluated by treating it as a binary classification problem, where the positive class (1) represents anomalous/attack traffic and the negative class (0) represents normal/benign traffic. The following standard metrics, all derived from the confusion matrix, are used to evaluate and compare anomaly detection methods.

### 2.6.1 Confusion Matrix

The confusion matrix summarises the four possible outcomes of a binary classifier's predictions against the ground truth:

- True Positive (TP): an anomalous flow correctly flagged as anomalous.
- True Negative (TN): a normal flow correctly classified as normal.
- False Positive (FP): a normal flow incorrectly flagged as anomalous (a false alarm).
- False Negative (FN): an anomalous flow incorrectly classified as normal (a missed attack).

|                 | <b>Predicted: Normal</b> | <b>Predicted: Anomaly</b> |
|-----------------|--------------------------|---------------------------|
| Actual: Normal  | TN (True Negative)       | FP (False Positive)       |
| Actual: Anomaly | FN (False Negative)      | TP (True Positive)        |

Table 2.7: Structure of the 2×2 confusion matrix used for binary anomaly-detection evaluation.

### Worked Example (Hypothetical Numbers)

To illustrate how these four counts combine into the metrics defined below, consider a hypothetical evaluation set of 1,000 flows, of which 400 are truly anomalous and 600 are truly normal (these numbers are illustrative only and are not drawn from any specific dataset or experiment). Suppose a detector produces the following confusion matrix on this hypothetical set:

|                             | <b>Predicted: Normal</b> | <b>Predicted: Anomaly</b> |
|-----------------------------|--------------------------|---------------------------|
| Actual: Normal (600 flows)  | TN = 540                 | FP = 60                   |
| Actual: Anomaly (400 flows) | FN = 50                  | TP = 350                  |

Table 2.8: Hypothetical confusion matrix used purely to illustrate the metric formulas below.

Using the formulas defined in the following subsections, this hypothetical confusion matrix would yield:

- Accuracy =  $(350 + 540) / 1000 = 890 / 1000 = 0.89$
- Precision =  $350 / (350 + 60) = 350 / 410 \approx 0.854$
- Recall =  $350 / (350 + 50) = 350 / 400 = 0.875$
- F1-score =  $2 \times (0.854 \times 0.875) / (0.854 + 0.875) \approx 0.864$
- False Positive Rate =  $60 / (60 + 540) = 60 / 600 = 0.10$

This hypothetical example illustrates a detector that catches most anomalies (87.5% recall) while keeping the majority of its positive predictions correct (85.4% precision), at the cost of a 10% false alarm rate among genuinely normal flows. The individual metric definitions and their formulas are given in the remaining subsections of this section.

#### 2.6.2 Accuracy

Accuracy measures the overall proportion of correctly classified flows (both normal and anomalous) out of all flows:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Accuracy can be misleading on imbalanced datasets, since a classifier that always predicts the majority class can still achieve a high accuracy score without detecting any true anomalies. For this reason, accuracy is rarely used as the sole metric in anomaly detection, where the anomalous class is often

the minority class.

### 2.6.3 Precision

Precision measures, of all the flows flagged as anomalous, what fraction were genuinely anomalous. It captures how trustworthy a positive (anomaly) prediction is:

$$Precision = TP / (TP + FP)$$

A low precision means the system generates many false alarms, which in a real security operations centre translates directly into analyst fatigue and wasted investigation effort.

### 2.6.4 Recall (Sensitivity / Detection Rate / True Positive Rate)

Recall measures, of all the genuinely anomalous flows, what fraction were correctly detected. It captures the completeness of detection:

$$Recall = TP / (TP + FN)$$

In a network security context, recall is often the more critical metric, since a false negative (a missed attack) can have a far greater cost than a false positive (an unnecessary alert).

### 2.6.5 F1-Score

The F1-score is the harmonic mean of precision and recall, giving a single balanced measure that penalises models which sacrifice one metric heavily to improve the other:

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

F1-score is commonly used as the primary criterion for hyperparameter selection in anomaly detection (e.g., choosing the contamination for Isolation Forest or LOF, or  $\nu/\gamma$  for OC-SVM), since it does not favour either precision or recall in isolation.

### 2.6.6 False Positive Rate (FPR)

The False Positive Rate measures, of all genuinely normal flows, what fraction were incorrectly flagged as anomalous:

$$FPR = FP / (FP + TN)$$

FPR is the x-axis of the ROC curve and is a direct measure of the false-alarm burden a method would impose on network operators.

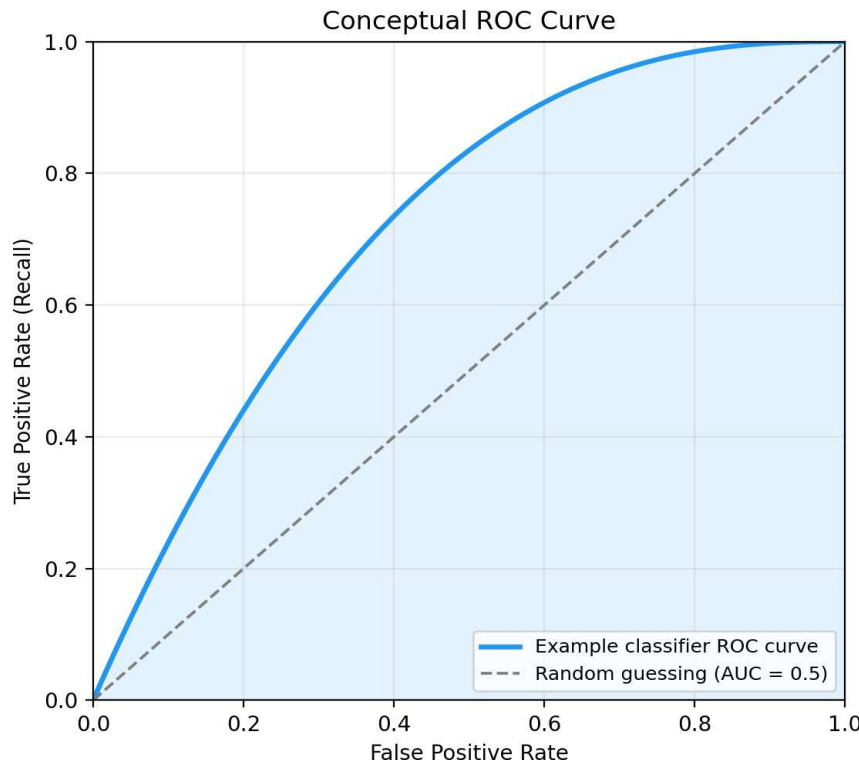
### 2.6.7 ROC Curve and AUC

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate (Recall) against the False Positive Rate at every possible decision threshold on the model's continuous anomaly score. It visualises the trade-off between detecting more attacks and generating more false alarms as the

decision threshold is varied, without committing to a single fixed threshold. The Area Under the ROC Curve (AUC) summarises this trade-off as a single scalar in the range [0, 1]:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR})$$

An AUC of 1.0 indicates a perfect classifier that can separate all anomalies from all normal points at some threshold; an AUC of 0.5 indicates performance no better than random guessing (equivalent to the diagonal line in Figure 8. AUC is threshold-independent and is therefore useful for comparing the underlying discriminative power of the anomaly scores produced by different models, independent of the specific contamination/nu/threshold chosen for deployment.



*Fig.2.8: Conceptual ROC curve. The diagonal line represents random guessing (AUC = 0.5); a curve bowed toward the top-left corner represents better separation between the normal and anomalous classes.*

## 2.7 Summary

This report has described the theoretical foundations of the anomaly detection pipeline used in this project: the structure of network flow features; the mathematics of the univariate, statistical Z-score method, including how its anomaly score is computed and how its detection threshold is selected and the mathematics, hyperparameters, and general tuning behaviour of three unsupervised machine learning methods - Isolation Forest, One-Class SVM, and Local Outlier Factor- each representing a distinct algorithmic strategy (partition-based, boundary-based, and density-based, respectively) for

separating normal from anomalous network traffic. Finally, this section defined the standard metrics - accuracy, precision, recall, F1-score, false positive rate, and ROC/AUC - used throughout this project to evaluate and compare the performance of these detection methods. Table 9 summarises the key conceptual differences between the three machine learning methods discussed.

| Method               | Core Mechanism                                             | Key Hyperparameters                         | Computational Cost                 | Captures Non-linear Boundaries?      |
|----------------------|------------------------------------------------------------|---------------------------------------------|------------------------------------|--------------------------------------|
| Isolation Forest     | Partition-based (ensemble of random trees)                 | contamination, n_estimators, max_samples    | Low (linear in n)                  | No (only axis-aligned splits)        |
| One-Class SVM        | Boundary-based (maximum-margin hyperplane in kernel space) | $\nu$ (nu), $\gamma$ (gamma), kernel type   | High (quadratic in n)              | Yes (via RBF kernel)                 |
| Local Outlier Factor | Density-based (local neighbourhood density ratio)          | n_neighbors, contamination, distance metric | Moderate-High (k-search per point) | Implicitly, via local neighbourhoods |

Table 2.9: Qualitative comparison of the three machine learning anomaly detection methods described in this report.

Taken together, the methods in this report form a progression of increasing modelling sophistication: the Z-score method establishes a simple, interpretable, per-feature statistical baseline; Isolation Forest and Local Outlier Factor extend detection into the full multivariate feature space using, respectively, random partitioning and local density comparison, without requiring any assumption about the underlying distribution of the data; and One-Class SVM offers a boundary-based alternative capable of learning highly non-linear regions of normality through the kernel trick. Because each method encodes a different notion of what makes an observation “anomalous” - extreme in some feature (Z-score), easy to isolate (Isolation Forest), outside a learned enclosing boundary (One-Class SVM), or locally sparse relative to its neighbours (LOF) - they provide complementary lenses through which network traffic can be examined, and the mathematical formulations, hyperparameters and evaluation metrics described in this chapter provide the theoretical basis needed to apply, tune, and interpret each of them.



## **CHAPTER-3**

### **Methodology**

#### **3.1 Overall Research Methodology**

The present study is concerned with the detection of Distributed Denial-of-Service (DDoS) attacks in network traffic through the lens of anomaly detection rather than conventional supervised classification. The central research objective is to conduct a systematic, comparative evaluation of statistical, unsupervised machine learning, and novelty detection paradigms for the identification of DDoS traffic within flow-based network data, and to determine the relative strengths, weaknesses, and practical suitability of each paradigm for deployment in an intrusion detection context.

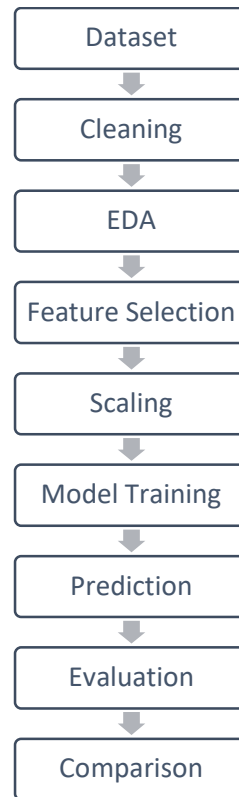
The anomaly detection problem, as formulated in this work, treats network traffic as arising from an underlying, largely stationary distribution of benign communication behaviour, with DDoS activity constituting a departure from this distribution. Two complementary framings of the problem are examined. In the first framing, adopted for the statistical baseline, the complete dataset, containing both benign and attack flows, is treated as a mixed population from which rare or extreme observations are to be isolated. In the second framing, adopted for the Isolation Forest, One-Class Support Vector Machine and the Local Outlier Factor in novelty-detection mode, the model is trained exclusively on benign traffic so as to construct a boundary or density profile of normal behaviour; traffic that falls outside this learned profile during testing is then flagged as anomalous. This second framing more closely mirrors an operational deployment scenario, in which labelled attack traffic is frequently scarce, delayed, or altogether unavailable at the time a detection system must be trained, whereas benign traffic can be collected in abundance under controlled conditions.

The motivation for undertaking a comparative rather than a single-model study stems from the recognition that DDoS attacks are heterogeneous in character and continually evolving in their manifestation at the network flow level. A detection technique that performs well against one attack profile may perform poorly against another, and a purely statistical technique that examines each feature independently may fail to capture the multivariate relationships that distinguish coordinated attack traffic from normal traffic. By evaluating a hierarchy of techniques, ranging from a simple univariate statistical rule, through an ensemble-based isolation mechanism, to two density- and margin-based novelty detectors, this study aims to establish an evidence-based understanding of which class of technique is best suited to the flow-based DDoS detection problem, and why.

The comparative approach adopted in this work proceeds by holding constant, wherever feasible, the dataset, the preprocessing pipeline, and the evaluation metrics across all four techniques, so that observed differences in performance may be attributed principally to the modelling paradigm itself rather than to inconsistencies in experimental setup. Each technique is subjected to a structured hyperparameter search over a range of values considered practically relevant, and the best-performing configuration for each technique, selected primarily on the basis of the F1-score, is carried forward into the final cross-model comparison.

The complete workflow adopted for the study proceeds from the acquisition of raw packet-flow data through a sequence of analytical and modelling stages culminating in a comparative assessment of all techniques. Raw CICIDS2017 flow records are first subjected to exploratory data analysis in

order to characterise the structure, quality, and discriminative potential of the available features. The dataset is then cleaned and transformed through a dedicated preprocessing pipeline that addresses missing and invalid values, duplicate records, non-informative constant features, and redundant highly correlated features, before being standardised through feature scaling. The cleaned and scaled data is subsequently used to train and evaluate, in turn, a Z-score statistical baseline, an Isolation Forest ensemble, a One-Class Support Vector Machine, and a Local Outlier Factor model, each subjected to hyperparameter tuning through grid search. Finally, the best configuration of each technique is evaluated using a common set of performance metrics, and the results are compared to draw conclusions regarding the relative merits of statistical, unsupervised ensemble, and novelty-based approaches to DDoS detection.



### 3.2 Dataset Description

The dataset employed in this study is drawn from the CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017) dataset, a widely used benchmark for network intrusion detection research developed by the Canadian Institute for Cybersecurity at the University of New Brunswick. The dataset was constructed by capturing five days of network traffic on a realistic testbed comprising both benign background activity, generated through the simulated behaviour of twenty-five users across common protocols such as HTTP, HTTPS, FTP, SSH, and email, and a range of contemporary attack scenarios executed against the same network infrastructure. Each captured packet flow was processed using the CICFlowMeter tool, which computes a large set of statistical descriptors, such as flow duration, packet counts, inter-arrival times, and header information, for every

bidirectional flow, and each resulting flow record is labelled according to whether it corresponds to benign or attack activity.

For the purposes of this study, attention is restricted to the Friday afternoon capture of the CICIDS2017 dataset, specifically the file corresponding to the DDoS attack scenario executed during that session. This subset was selected because it isolates a single, well-defined attack category, namely Distributed Denial-of-Service traffic, generated using established DDoS tools against a target web server, from an otherwise benign traffic background, thereby allowing the anomaly detection techniques under study to be evaluated on a clearly delineated binary problem without the confound of multiple, dissimilar attack types coexisting within the same file.

In its raw form, the Friday DDoS subset comprises 225,745 individual flow records, each described by seventy-nine features. These features can be broadly grouped into several categories: flow-level statistics such as flow duration and flow bytes per second; packet-level statistics such as forward and backward packet length maxima, minima, means, and standard deviations; inter-arrival time statistics computed separately for forward and backward directions; TCP control flag counts such as SYN, ACK, PSH, and FIN flag counts; and derived activity and idle-time statistics that characterise bursts and pauses within a flow. This breadth of feature coverage provides multiple, partially independent perspectives on network behaviour, which is essential for a comparative study seeking to test the ability of different algorithms to exploit both univariate and multivariate structure in the data.

Within this subset, traffic is labelled under two classes: BENIGN, corresponding to normal user and system activity, and DDoS, corresponding to attack traffic. Prior to any cleaning, the BENIGN class comprises 97,718 flows (approximately 43.3 percent of the dataset) and the DDoS class comprises 128,027 flows (approximately 56.7 percent of the dataset). This distribution indicates a moderate rather than severe class imbalance, which is a favourable property for the present study, since it allows the performance of anomaly detection techniques to be assessed without the confounding influence of extreme rarity that is characteristic of many real-world intrusion detection scenarios, while still preserving a meaningful distinction between the majority and minority classes for the purposes of contamination parameter selection in several of the models under study.

The Friday DDoS subset of CICIDS2017 was selected for this research for several reasons. First, the dataset is generated on a realistic, modern network topology using contemporary operating systems, protocols, and traffic generation profiles, in contrast to older intrusion detection benchmarks such as KDD Cup 1999 or NSL-KDD, which are widely acknowledged to suffer from unrealistic feature distributions and outdated attack signatures. Second, the dataset is fully labelled at the flow level, which permits rigorous, ground-truth-based evaluation of unsupervised and novelty detection techniques, notwithstanding the fact that such techniques do not use these labels during training. Third, the dataset is extensively used in the network security literature, which facilitates comparison of the results obtained in this study with those reported by other researchers. Finally, the specific choice of the DDoS subset, as opposed to the complete five-day capture, permits a controlled and computationally tractable comparative study while retaining a class distribution that is representative of a genuinely challenging detection problem.

### 3.3 Data Preprocessing

The raw flow records obtained from the CICIDS2017 Friday DDoS capture file cannot be supplied directly to statistical or machine learning models, since they contain missing values, invalid numerical entries, duplicate records, non-informative features, and features with a highly variable numerical scale. A dedicated preprocessing pipeline was therefore designed and applied uniformly prior to the training of every model considered in this study, so that differences in the results obtained across models could be attributed to the modelling technique itself rather than to inconsistencies in data preparation. Each stage of this pipeline is described below, together with its underlying purpose, its methodology, and its expected effect on downstream models.

#### 3.3.1 Missing Value Analysis

An initial scan of the raw dataset revealed four entries containing missing (NaN) values. In addition to these explicit missing values, a further examination of the numerical columns revealed the presence of infinite values, specifically thirty occurrences in the Flow Bytes/s feature and thirty-four occurrences in the Flow Packets/s feature. These infinite values arise as a direct consequence of the manner in which CICFlowMeter computes rate-based features: when a flow has an extremely short, effectively zero, duration, the division of a byte or packet count by that duration produces a mathematically undefined or infinite quotient. Because such values cannot be processed by standard statistical or machine learning routines, all infinite values were first converted to NaN so that they could be handled through a single, unified missing-value treatment procedure. Following this conversion, the total count of missing entries in the dataset rose to sixty-eight, concentrated within thirty-four unique records (since each affected record typically produced a missing value in both of the two rate-based columns simultaneously). Given that these thirty-four records constitute only approximately 0.015 percent of the total dataset, the decision was taken to remove them entirely by row-wise deletion rather than to attempt imputation, since the very small proportion of affected records renders any potential bias introduced by their removal negligible, while imputation would risk introducing artificial values into features that are of particular importance for anomaly detection. Following this step, the dataset comprised 225,711 flow records.

#### 3.3.2 Duplicate Removal

Exact duplicate rows, in which every feature value and label is identical to another record in the dataset, were identified using a row-wise comparison across the complete feature space. This analysis revealed 2,629 duplicate records, of which 2,618 belonged to the BENIGN class and only eleven belonged to the DDoS class. The overwhelming concentration of duplicates within the benign class is consistent with the nature of background traffic generation in the testbed, in which repetitive, automated benign interactions (such as periodic keep-alive or polling behaviour) are considerably more likely to produce identical flow-level feature vectors than the comparatively more varied and short-lived flows generated during an active attack. Retaining such duplicate records would artificially inflate the apparent frequency of a small number of specific benign feature vectors within the dataset, biasing any density- or distance-based technique, including the Z-score baseline and the Local Outlier Factor, towards treating these repeated configurations as disproportionately probable and hence overly normal. All duplicate records were therefore removed, reducing the dataset from 225,711 to 223,082 records.

### 3.3.3 Constant Feature Removal

A feature that assumes a single, unchanging value across every record in the dataset conveys no discriminatory information and merely increases the dimensionality of the modelling problem without benefit. An exhaustive scan of the number of unique values assumed by each column identified ten such constant features: Bwd PSH Flags, Fwd URG Flags, Bwd URG Flags, CWE Flag Count, Fwd Avg Bytes/Bulk, Fwd Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, and Bwd Avg Bulk Rate. These features correspond largely to bulk-transfer statistics and to TCP flag types that were never observed to vary within this particular capture. All ten features were removed from the dataset, reducing the feature count from seventy-nine to sixty-nine columns (sixty-eight predictor features plus the label column). The removal of constant features is a standard and low-risk dimensionality reduction step: because such features exhibit zero variance, their inclusion cannot improve the discriminative capacity of any distance-, density-, margin-, or split-based model, while their presence needlessly increases computational cost and, in distance-based calculations, can introduce numerical artefacts when a feature's standard deviation is zero.

### 3.3.4 Correlation Analysis and Feature Selection

Beyond constant features, CICIDS2017 is known to contain a substantial number of features that are highly correlated with one another as a consequence of the manner in which CICFlowMeter derives multiple descriptors from the same underlying packet-level quantities. To identify such redundancy, the pairwise absolute Pearson correlation coefficient was computed for every pair of the sixty-eight remaining numerical features, producing a symmetric correlation matrix. For each feature, its correlation with every other feature appearing later in the column ordering was examined, and any feature exhibiting an absolute correlation coefficient exceeding 0.95 with another feature was flagged as redundant. This procedure identified twenty-six highly correlated features, including, among others, Total Backward Packets, Total Length of Bwd Packets, Fwd Packet Length Std, Bwd Packet Length Mean, Bwd Packet Length Std, Flow IAT Max, Fwd IAT Total, Fwd Header Length, Bwd Header Length, Average Packet Size, Avg Fwd Segment Size, Avg Bwd Segment Size, and the Subflow-prefixed packet and byte counts.

The Pearson correlation coefficient between two features  $X$  and  $Y$  is defined as

$$r(X, Y) = \sum_i (x_i - \bar{x})(y_i - \bar{y}) / \sqrt{[\sum_i (x_i - \bar{x})^2 \cdot \sum_i (y_i - \bar{y})^2]}$$

A coefficient close to unity in absolute value indicates that the two features vary in an almost perfectly linear fashion with respect to one another, such that one feature contributes essentially no additional information beyond what is already captured by the other. Retaining both members of such a highly correlated pair would not increase the information available to a model but would instead distort distance- and density-based computations, such as those underlying the Z-score baseline and the Local Outlier Factor, by implicitly assigning greater weight to whichever underlying quantity happens to be duplicated across multiple correlated features. The twenty-six identified features were therefore removed, yielding a reduced feature set of forty-two predictor features. This reduced, decorrelated feature set of forty-two variables forms the basis for all subsequent modelling in this study.

### 3.3.5 Handling of Categorical Variables and Label Encoding

The only categorical variable present in the dataset following the removal of constant and redundant features is the Label column, which records each flow as either BENIGN or DDoS. Since none of the modelling techniques employed in this study operate directly on string-valued categorical data, this column was converted into a binary numerical indicator, hereafter referred to as the anomaly label, in which a value of zero denotes benign traffic and a value of one denotes DDoS traffic. This binary encoding is used exclusively for the purposes of evaluation, that is, for computing confusion matrices and derived performance metrics; it is never supplied to any anomaly detection model as an input feature, since doing so would trivially reveal the ground truth and defeat the purpose of unsupervised and novelty-based detection.

### 3.3.6 Feature Scaling

Following feature selection, the forty-two retained predictor features exhibit widely differing numerical ranges: quantities such as Flow Duration are measured in microseconds and can assume values in the tens of millions, whereas quantities such as flag counts assume small integer values, typically between zero and a handful. Because several of the techniques examined in this study, namely the Z-score baseline, the One-Class Support Vector Machine, and the Local Outlier Factor, rely directly on distance or density computations in the original feature space, features with large numerical ranges would dominate these computations purely as an artefact of their units of measurement, irrespective of their true discriminative relevance. To remove this source of bias, all predictor features were standardised using the StandardScaler transformation, which rescales each feature to have zero mean and unit variance according to

$$z = (x - \mu) / \sigma$$

where  $\mu$  and  $\sigma$  denote, respectively, the mean and standard deviation of the feature computed over the relevant training partition. Standardisation was verified after transformation by confirming that the mean of every scaled feature was approximately zero (on the order of  $10^{-16}$  to  $10^{-17}$ , consistent with floating-point rounding rather than a systematic offset) and that the standard deviation of every scaled feature was approximately one. It is important to note that, in every model requiring a train-test split, the scaler was fitted exclusively on the training partition and subsequently applied, without refitting, to transform the corresponding test partition; this procedure prevents information from the test data, including its mean and variance, from leaking into the training process, thereby preserving the validity of the reported test-set performance as an estimate of generalisation to previously unseen traffic.

Feature scaling has distinct but complementary effects across the four modelling techniques considered. For the Z-score baseline, standardisation is not merely beneficial but definitional, since the Z-score anomaly criterion is itself expressed directly in terms of standardised deviations from the mean. For the Local Outlier Factor and the One-Class Support Vector Machine, both of which rely on Euclidean distance either directly, in the computation of nearest neighbours, or indirectly, through the radial basis function kernel, standardisation ensures that no single feature disproportionately governs the notion of proximity between observations. For the Isolation Forest, whose splitting procedure selects a feature uniformly at random and a split point uniformly between the observed minimum and maximum of that feature, standardisation has a comparatively smaller theoretical effect, since the

algorithm is largely scale-invariant with respect to any single feature considered in isolation; nonetheless, scaling was retained for the Isolation Forest as well, both for methodological consistency across all four techniques and because it can still marginally affect the relative frequency with which different features are selected for splitting when `max_samples` and feature subsampling interact with heavily skewed raw distributions.

### **3.3.7 Final Preprocessed Dataset**

Upon completion of the preprocessing pipeline described above, the dataset comprised 223,082 flow records, each described by forty-two predictor features together with the original textual Label and, in subsequent notebooks, the derived binary anomaly indicator. The class distribution within this cleaned dataset consists of 128,014 DDoS flows and 95,068 BENIGN flows, corresponding to proportions of approximately 57.4 percent and 42.6 percent respectively. This cleaned dataset was persisted to disk as a single file and used, without further modification, as the common starting point for the exploratory analysis and for the training and evaluation of every anomaly detection technique examined in this study.

## **3.4 Exploratory Data Analysis**

Prior to the design of any detection algorithm, an exploratory data analysis was conducted on the raw Friday DDoS capture in order to characterise the statistical properties of the dataset, to identify data quality issues requiring correction during preprocessing, and to assess which features exhibit potential discriminative value for separating benign from DDoS traffic.

### **3.4.1 Descriptive Statistics and Distributional Analysis**

Group-wise descriptive statistics, computed separately for the BENIGN and DDoS classes, revealed several noteworthy patterns. When comparing the mean Flow Duration of the two classes, DDoS traffic exhibits only a marginally higher average duration than benign traffic, suggesting that the arithmetic mean alone is not a reliable discriminator for this feature. However, examination of the complete distribution, including the median and interquartile range, told a considerably more informative story: the median flow duration for DDoS traffic was found to be roughly an order of magnitude greater than the median flow duration for benign traffic, indicating that, while a subset of very short-lived or very long-lived flows in both classes inflate the respective means towards similar values, the bulk of DDoS flows are systematically longer-lived than the bulk of benign flows. This observation also surfaced a data quality anomaly, namely the presence of a negative flow duration value within the benign class, a physically impossible quantity that was flagged for handling during preprocessing, though it was ultimately resolved indirectly through the broader missing- and invalid-value treatment applied to rate-derived features.

The Flow Packets/s feature was found to exhibit extreme positive skewness in both classes, with maximum values several orders of magnitude larger than the corresponding medians. This heavy-tailed behaviour indicates that a small number of flows generate packets at an exceptionally high rate, while the great majority of flows, whether benign or malicious, exhibit comparatively modest packet rates. The presence of such extreme outliers in both classes implies that measures of central tendency based on the arithmetic mean can be highly misleading for this feature, and that robust statistics such

as the median, together with visual tools such as the boxplot, are more appropriate for characterising its behaviour.

### **3.4.2 Class-Conditional Feature Comparison**

Boxplot comparisons of Total Fwd Packets by class revealed that benign traffic exhibits substantially greater variability, including numerous extreme high-packet-count outliers, whereas DDoS traffic is comparatively concentrated around lower packet counts. This pattern is consistent with the operational nature of many DDoS attack tools, which frequently generate a very large number of short, repetitive flows rather than a smaller number of long, information-rich flows, in contrast to benign user sessions, which can vary considerably in length and packet volume depending on the application being used.

A comparison of the Packet Length Mean feature revealed a markedly clearer separation between the two classes than either of the two features discussed above, with DDoS flows exhibiting substantially larger average packet lengths and a correspondingly higher median value than benign flows. Given the clear visual separation between the class-conditional distributions of this feature, Packet Length Mean was identified during the exploratory phase as a strong candidate discriminator, an observation that is corroborated in a later section of this chapter, where permutation-based feature importance analysis applied to the Isolation Forest model independently identifies Packet Length Mean as the single most influential feature for anomaly detection.

### **3.4.3 Correlation Structure and Class Balance**

As part of the exploratory analysis, the pairwise correlation structure of the numerical features was examined in order to understand the extent of redundancy within the seventy-nine (subsequently sixty-eight, following constant-feature removal) raw features. This analysis, formalised through the correlation heatmap and threshold-based redundancy detection described in the preprocessing section of this chapter, revealed extensive linear redundancy among features describing closely related quantities, such as packet length statistics computed across slightly different aggregation windows, and directly motivated the feature selection step subsequently applied during preprocessing.

The class distribution of the dataset, comprising approximately 56.7 percent DDoS traffic and 43.3 percent benign traffic prior to cleaning, was also examined during this phase. This distribution was judged to represent a relatively mild degree of class imbalance, sufficiently balanced that neither class overwhelmingly dominates the dataset, yet sufficiently skewed towards the attack class that the choice of contamination parameter in models such as the Isolation Forest and the Local Outlier Factor required explicit empirical tuning rather than reliance on default assumptions of rarity.

### **3.4.4 Summary of Exploratory Insights**

Taken together, the exploratory data analysis yielded four principal insights that directly informed the subsequent stages of this study. First, the dataset exhibits a small number of missing and invalid (infinite) values, concentrated in rate-derived features, that required removal prior to modelling. Second, several features, particularly Flow Packets/s, exhibit extreme skewness and outliers that motivate the use of standardisation and, more broadly, caution when interpreting mean-based statistics. Third, packet-length-related features and, to a lesser extent, forward packet count features exhibit visually apparent separation between benign and DDoS traffic, suggesting that a well-designed



anomaly detection technique operating on the multivariate feature space should be capable of exploiting these differences. Fourth, the moderate class imbalance present in the dataset is an important design consideration for the contamination and threshold parameters of the subsequent detection models.

The insights obtained during the exploratory analysis guided the preprocessing strategy and informed the implementation of the anomaly detection techniques described in the following sections.

### 3.5 Z-Score Methodology

As the first technique evaluated in this comparative study, the Z-score method was implemented as a statistical baseline. Its purpose was to provide a simple reference against which the machine learning-based anomaly detection techniques could be compared. The Z-score expresses the number of standard deviations by which an observation deviates from the mean of a feature and is computed

as:

$$Z = (x - \mu) / \sigma$$

In this study, the Z-score was computed independently for each of the forty-two standardised predictor features. Since the features had already been transformed using StandardScaler during preprocessing, the computed Z-score was numerically equivalent to the corresponding standardised feature value. For every network flow, the anomaly score was defined as the maximum absolute Z-score among all predictor features. A flow was classified as anomalous whenever the maximum absolute Z-score exceeded a predefined threshold.

To investigate the influence of the decision threshold on detection performance, three threshold values (2.0, 2.5 and 3.0) were evaluated. The same evaluation metrics were computed for each threshold. The comparative performance of these threshold values and the selection of the final operating threshold are presented in Chapter 5.

The Z-score method served as a statistical baseline against which the machine learning-based techniques were evaluated. Owing to its simplicity and computational efficiency, it provided an appropriate reference for assessing the additional benefits offered by multivariate anomaly detection models.

While the Z-score method provides a simple statistical baseline, more sophisticated machine learning techniques are capable of modelling multivariate relationships within network traffic. The first of these techniques investigated in this study is the Isolation Forest.

### 3.6 Isolation Forest Methodology

Isolation Forest is an ensemble-based unsupervised anomaly detection algorithm that identifies anomalies through recursive random partitioning of the feature space. The underlying assumption is that anomalous observations are fewer in number and structurally different from normal observations, allowing them to be isolated using fewer partitioning steps than normal data.

### 3.6.1 Experimental Pipeline

Consistent with the novelty-detection framework adopted in this study, the Isolation Forest model was trained exclusively using benign network traffic. The benign subset of the cleaned dataset was divided into a training partition containing 70% of the benign flows and a held-out test partition containing the remaining 30%. The StandardScaler was fitted only on the benign training partition and subsequently applied to both the held-out benign data and the complete DDoS dataset to prevent information leakage.

The final evaluation dataset was created by combining the scaled held-out benign flows with the scaled DDoS flows. After training on the benign training partition, the Isolation Forest model generated predictions on this mixed evaluation dataset. This methodology reflects a realistic deployment scenario in which only legitimate traffic is available during training while both normal and malicious traffic are encountered during operation.

### 3.6.2 Hyperparameter Tuning

Two Isolation Forest hyperparameters were investigated: the contamination parameter and the number of estimators (`n_estimators`). The contamination parameter was varied over a predefined range to study its influence on anomaly classification, while the number of estimators was adjusted to evaluate the effect of ensemble size on model performance. For every parameter combination, the model was trained using the benign training partition and evaluated on the common test dataset. Performance was assessed using Precision, Recall and F1-score. The detailed tuning results, selected parameter values and comparative analysis are presented in Chapter 5.

### 3.6.3 Feature Importance

Following model training, permutation feature importance was employed to estimate the contribution of each feature to the model's predictions. Each feature was independently shuffled while keeping all remaining features unchanged, and the corresponding reduction in model performance was measured. This analysis was performed after selecting the final model and was used solely to interpret model behaviour. The resulting feature rankings and discussion are presented in Chapter 5.

### 3.6.4 Generated Visualisations

Several visualisations were produced to support the experimental evaluation, including plots illustrating the effect of contamination and ensemble size, confusion matrices, anomaly score distributions and permutation feature importance. These visualisations are presented and discussed in Chapter 5 as part of the experimental results.

The next technique considered was the One-Class Support Vector Machine

## 3.7 One-Class SVM Methodology

One-Class Support Vector Machine (One-Class SVM) was selected as the third anomaly detection technique because of its ability to learn the characteristics of normal network traffic using only benign observations. The model was implemented as a novelty detection algorithm, allowing it to

establish a decision boundary from normal traffic and subsequently classify previously unseen network flows as either normal or anomalous.

### **3.7.1 Experimental Pipeline**

The experimental pipeline followed the benign-only training protocol adopted throughout this study. The cleaned benign dataset was partitioned into a 70% training subset (66,547 flows) and a 30% held-out testing subset (28,521 flows) using a fixed random state to ensure reproducibility. The StandardScaler was fitted exclusively on the benign training subset and subsequently applied to both the held-out benign subset and the complete DDoS dataset (128,014 flows). The scaled held-out benign flows and scaled DDoS flows were combined to construct a common evaluation dataset containing both normal and anomalous traffic.

The One-Class SVM model was trained only on the scaled benign training subset. After training, predictions were generated for every observation in the common evaluation dataset. This methodology reflects a realistic deployment scenario in which only legitimate traffic is available during training, while both benign and attack traffic are encountered during inference.

### **3.7.2 Hyperparameter Tuning**

Hyperparameter optimisation was performed in two sequential stages. Initially, the parameter  $\nu$  (nu) was varied while maintaining the kernel parameter  $\gamma$  (gamma) at its default setting. After completing this stage,  $\gamma$  was systematically varied while  $\nu$  remained fixed. For every parameter configuration, the model was retrained using the benign training subset and evaluated on the common evaluation dataset using the evaluation metrics. This sequential approach enabled the influence of each hyperparameter to be assessed independently while maintaining a consistent evaluation protocol. The comparative tuning results and the final selected configuration are presented in Chapter 5.

### **3.7.3 Prediction and Evaluation**

Predictions generated by the trained model were compared with the corresponding ground-truth labels of the evaluation dataset. Precision, Recall, F1-score, False Positive Rate, Benign Accuracy and DDoS Detection Rate were calculated for every evaluated configuration. These metrics formed the basis for comparing the performance of different hyperparameter settings and for the final comparison with the remaining anomaly detection methods.

### **3.7.4 Generated Outputs**

The experimentation produced confusion matrices, classification reports and hyperparameter tuning plots to support the evaluation of the One-Class SVM model. These outputs are presented and discussed in Chapter 5.

The final anomaly detection technique investigated was the Local Outlier Factor.

## **3.8 Local Outlier Factor Methodology**

Local Outlier Factor (LOF) was selected as the final anomaly detection technique because it identifies anomalies based on variations in local data density. The model was implemented using the novelty detection mode (novelty=True), enabling it to be trained exclusively on benign traffic and subsequently evaluated on previously unseen network flows.

### 3.8.1 Experimental Pipeline

To ensure consistency with the other machine learning models, the same benign-only training protocol was adopted. The benign dataset was divided into a 70% training subset and a 30% held-out testing subset. The StandardScaler was fitted only on the benign training data and then applied to both the held-out benign flows and the complete DDoS dataset. The transformed held-out benign flows and transformed DDoS flows were merged to create a common evaluation dataset.

The Local Outlier Factor model was trained using only the scaled benign training subset in novelty mode. Following training, predictions were generated for all observations in the common evaluation dataset. Using novelty mode ensured that the model was evaluated on previously unseen data, closely reflecting practical network intrusion detection scenarios.

### 3.8.2 Hyperparameter Tuning

A systematic grid search was performed to investigate the influence of the hyperparameters `n_neighbors` and `contamination`. Multiple combinations of these parameters were evaluated while maintaining the same preprocessing pipeline, training dataset and evaluation dataset for every experiment. Each trained model was assessed using the evaluation metrics to ensure a fair comparison across all parameter configurations. The detailed tuning results and the final selected hyperparameter combination are presented in Chapter 5.

### 3.8.3 Prediction and Evaluation

After model training, predictions were generated for the complete evaluation dataset containing both held-out benign traffic and DDoS traffic. The predicted labels were compared with the corresponding ground-truth labels to compute Precision, Recall, F1-score, False Positive Rate, Benign Accuracy and DDoS Detection Rate. The same evaluation procedure was adopted for every hyperparameter combination to maintain consistency throughout the study.

### 3.8.4 Generated Outputs

The Local Outlier Factor experiments produced hyperparameter tuning plots, heatmaps, confusion matrices and classification reports to facilitate performance analysis. These outputs are presented and discussed in Chapter 5.

## 3.9 Performance Evaluation Metrics

A common evaluation framework was adopted to ensure a fair comparison among the four anomaly detection techniques. For every model, predictions generated on the evaluation dataset were compared with the corresponding ground-truth labels, where DDoS traffic was treated as the positive class and benign traffic as the negative class. The confusion matrix formed the basis for all subsequent performance metrics.

| Metric                       | Purpose                                                                       |
|------------------------------|-------------------------------------------------------------------------------|
| Precision                    | Measures the proportion of predicted DDoS flows that are genuinely malicious. |
| Recall (DDoS Detection Rate) | Measures the proportion of actual DDoS flows correctly identified.            |

|                     |                                                                                                                         |
|---------------------|-------------------------------------------------------------------------------------------------------------------------|
| F1-score            | Provides a balanced measure of precision and recall and was adopted as the primary metric for hyperparameter selection. |
| Accuracy            | Measures the overall proportion of correctly classified flows.                                                          |
| False Positive Rate | Measures the proportion of benign flows incorrectly classified as anomalies.                                            |
| Benign Accuracy     | Measures the proportion of benign flows correctly classified.                                                           |

The above metrics were computed using the standard confusion-matrix equations presented in this study. Among these, the F1-score was selected as the primary criterion for hyperparameter tuning because it balances precision and recall and provides a reliable basis for comparing models evaluated under different class distributions.

### 3.10 Hyperparameter Tuning Strategy

A consistent hyperparameter tuning strategy was employed for Isolation Forest, One-Class SVM and Local Outlier Factor. Rather than using automated cross-validation, a manual exhaustive grid search was performed for each model. Each hyperparameter combination was trained using the designated training partition and evaluated on the same fixed evaluation dataset, ensuring that all configurations were assessed under identical experimental conditions.

The parameter ranges investigated were selected based on the default recommendations provided in the scikit-learn documentation, published literature and preliminary experimentation. The same train-test partition, preprocessing pipeline and evaluation metrics were maintained throughout the tuning process to ensure a fair and reproducible comparison. The detailed tuning outcomes and the final selected configurations are presented in Chapter 5.

### 3.11 Experimental Workflow

The complete experimental methodology followed a sequential workflow beginning with acquisition of the CICIDS2017 dataset. Exploratory Data Analysis was performed to understand data characteristics and identify quality issues. The dataset was subsequently preprocessed through removal of missing values, duplicate records, constant features and highly correlated features. Following preprocessing, feature scaling was applied using StandardScaler according to the training protocol adopted for each model.

The processed data were then used for model training, followed by hyperparameter tuning, prediction generation and performance evaluation. Finally, the outputs obtained from each anomaly detection technique were consolidated for comparative analysis. A workflow diagram illustrating this experimental pipeline is provided with this section to summarise the overall methodology.

### 3.12 Methodology Summary

This chapter presented the complete methodology adopted for the comparative evaluation of statistical and machine learning-based anomaly detection techniques using the CICIDS2017 dataset. The methodology encompassed data acquisition, exploratory data analysis, preprocessing, feature scaling, implementation of the Z-Score, Isolation Forest, One-Class SVM and Local Outlier Factor

models, hyperparameter tuning and performance evaluation under a common experimental framework.

To ensure a fair comparison, consistent preprocessing, evaluation metrics and model assessment procedures were maintained throughout the study. Where applicable, the machine learning models were trained exclusively on benign traffic and evaluated using a common mixed test dataset containing held-out benign flows and DDoS traffic. This experimental protocol enabled differences in performance to be attributed to the characteristics of the individual algorithms rather than variations in data preparation or evaluation methodology.

The methodology established in this chapter provides the foundation for the experimental evaluation presented in the following chapter. Chapter 5 presents the hyperparameter tuning results, model performance, comparative analysis and discussion of the four anomaly detection techniques.

## **CHAPTER-4**

### **Results and Discussion**

#### **Experimental Setup**

This chapter presents the experimental results obtained for all four anomaly detection techniques evaluated as part of the comparative study on DDoS attack detection: the Z-Score statistical baseline, the One-Class Support Vector Machine (One-Class SVM), Isolation Forest, and Local Outlier Factor (LOF).

The experiments were conducted on the CICIDS2017 dataset, specifically the Friday-WorkingHours-Afternoon-DDoS capture, which originally comprised 225,745 network flow records described by 79 statistical flow-level features. Prior to model evaluation, the dataset was subjected to a preprocessing pipeline in which missing and infinite values were removed, duplicate flow records were eliminated, ten constant features carrying no discriminatory information were dropped, and twenty-six features exhibiting pairwise correlation coefficients above 0.95 were removed to reduce redundancy. The resulting cleaned dataset used for all experiments in this chapter consists of 223,082 flow records described by 42 independent numerical features, together with a binary class label distinguishing BENIGN traffic (95,068 flows, 42.6 percent) from DDoS traffic (128,014 flows, 57.4 percent). All features were standardized to zero mean and unit variance using a StandardScaler prior to model application, ensuring that no individual feature dominates distance- or deviation-based computations due to differences in measurement scale.

The testing methodology differs between the two techniques owing to their distinct learning paradigms. The Z-Score method is a fully unsupervised statistical procedure that was applied directly to the entire cleaned dataset of 223,082 flows, with ground-truth labels used only for post-hoc evaluation. The One-Class SVM, in contrast, follows a novelty detection paradigm and was trained exclusively on BENIGN traffic. The 95,068 BENIGN flows were partitioned into a training subset of 66,547 flows (70 percent) and a held-out testing subset of 28,521 flows (30 percent); the feature scaler was fit using only the training BENIGN subset to avoid information leakage. The evaluation set for the One-Class SVM therefore comprises the 28,521 unseen BENIGN flows combined with all 128,014 DDoS flows, yielding a test set of 156,535 flows. It is noted that this

evaluation set is itself imbalanced, with DDoS flows constituting approximately 81.8 percent of the test set and BENIGN flows the remaining 18.2 percent; this imbalance is taken into account when interpreting class-wise precision and recall in.

Model performance is quantified using Precision, Recall, and F1-score computed for the DDoS (positive) class, together with overall classification accuracy, the confusion matrix, the False Positive Rate (the proportion of legitimate BENIGN flows incorrectly flagged as anomalous), Benign Accuracy (specificity, the proportion of BENIGN flows correctly recognized as normal), and the DDoS Detection Rate (equivalent to recall for the DDoS class, representing the proportion of actual attack flows correctly identified). Training and testing execution times were additionally recorded to assess the computational feasibility of each technique for near-real-time network monitoring. All experiments were implemented in Python using the scikit-learn, pandas, and NumPy libraries on a standard workstation environment.

## **4.1 Z-Score Results**

The Z-Score method was evaluated first in order to establish a statistical baseline against which the more sophisticated machine learning based techniques could be compared. DDoS flooding attacks frequently manifest as extreme values in individual traffic descriptors, such as unusually short flow durations, abnormally high packet rates, or atypical byte counts; a classical univariate outlier detection procedure such as Z-Score analysis therefore represents a natural and computationally inexpensive first approach to test whether such extreme per-feature deviations are, by themselves, sufficient to distinguish DDoS traffic from legitimate traffic in this dataset.

### **4.1.1 Threshold Selection**

For each of the 42 standardized features, an absolute Z-score was computed for every flow, and a flow was classified as anomalous if the absolute Z-score of at least one feature exceeded a chosen threshold. Three principal threshold values were evaluated: 3.0, corresponding to the conventional three-sigma statistical outlier convention; 2.5; and 2.0. An additional, deliberately extreme threshold of 0.5 was also examined to illustrate the degenerate behavior of the method at very low thresholds.

At the conventional threshold of 3.0, the detector flagged 74,273 of the 223,082 flows (33.3 percent) as anomalous, considerably fewer than the 128,014 flows (57.4 percent) that are actually labeled DDoS. This immediately indicates substantial under-detection at the strict three-sigma cutoff: many



attack flows do not present sufficiently extreme values in any single feature to be captured by this rule. Lowering the threshold to 2.5 increased the number of flagged flows to 103,209 (46.3 percent), and a further reduction to 2.0 increased the flagged count to 118,576 (53.2 percent), progressively narrowing the gap with the true anomaly count but never closing it.

The corresponding classification outcomes were as follows: at threshold 3.0, precision was 0.34, recall was 0.20, and F1-score was 0.25; at threshold 2.5, precision fell to 0.27, recall rose marginally to 0.22, and F1-score fell slightly to 0.24; at threshold 2.0, precision recovered to 0.30, recall increased further to 0.28, and F1-score reached its highest observed value of 0.29. The relationship between threshold and performance is therefore not monotonic in precision, even though the fraction of flows flagged as anomalous increases steadily as the threshold is lowered. This behavior is explained by the disjunctive, per-feature nature of the decision rule: because a flow is flagged if any one of 42 features crosses the threshold, lowering the threshold admits both additional true DDoS detections and a substantial number of additional false positives drawn from naturally variable BENIGN flows (for example, short bursty legitimate connections or retransmission events), and the relative rate at which each population is admitted differs non-uniformly across the threshold range. As a further illustration of the method's limitations, an extreme threshold of 0.5 was found to flag the entire dataset of 223,082 flows as anomalous; under this degenerate configuration, recall for the DDoS class trivially reaches 1.00 while precision collapses to 0.57, exactly equal to the prior proportion of DDoS flows in the dataset. This confirms that the apparent gain in recall at very low thresholds carries no genuine discriminative value and instead reflects the class prior.

#### **4.1.2 Performance Metrics**

Precision indicates the proportion of flows flagged as anomalous that are genuinely DDoS traffic, and is therefore a direct measure of analyst trust in raised alerts; recall (the DDoS Detection Rate) indicates the proportion of actual attack flows successfully identified, and is the primary measure of security coverage; the F1-score is the harmonic mean of the two and summarizes the overall balance achieved. The False Positive Rate quantifies the operational burden imposed on network operators by measuring the fraction of legitimate traffic that is incorrectly flagged, while Benign Accuracy (specificity) is its complement, measuring the fraction of legitimate traffic correctly left unflagged.

Table 4.1 summarizes the obtained metrics across the three principal thresholds evaluated, together with derived False Positive Rate and Benign Accuracy figures. For thresholds 2.5 and 2.0, exact

integer confusion matrix counts were not printed numerically in the source experiments (only rendered as confusion matrix visualizations); the True Negative and True Positive counts reported below for these two configurations were therefore computed from the reported recall values and known class sample sizes (95,068 BENIGN and 128,014 DDoS flows) and are consequently approximate, whereas the values for threshold 3.0 are exact figures obtained directly from the printed confusion matrix.

| Threshold | Precision | Recall (DDoS) | F1-Score | Benign Accuracy | False Positive Rate |
|-----------|-----------|---------------|----------|-----------------|---------------------|
| 3.0       | 0.34      | 0.20          | 0.25     | 48.25%          | 51.75%              |
| 2.5       | 0.27      | 0.22          | 0.24     | ~21.0%          | ~79.0%              |
| 2.0       | 0.30      | 0.28          | 0.29     | ~13.0%          | ~87.0%              |

*Table 4.1: Z-Score performance across evaluated thresholds.*

The results in Table 5.1 demonstrate that even at the threshold selected as best-performing on the basis of F1-score, namely 2.0, the Benign Accuracy is only approximately 13 percent, meaning that roughly 87 percent of legitimate traffic would be incorrectly flagged as an attack. This False Positive Rate is operationally unacceptable for any realistic network defense deployment, since it would generate an alert volume overwhelmingly dominated by false alarms, while still failing to detect 72 percent of actual DDoS flows. The selection of threshold 2.0 as the 'best' configuration must therefore be understood as the best among a set of uniformly poor operating points, rather than as evidence of acceptable detection capability.

#### 4.1.3 Confusion Matrix Analysis

The confusion matrix obtained at the conventional threshold of 3.0 was printed directly in the source experiment as [[45864, 49204], [102945, 25069]], corresponding to 45,864 True Negatives, 49,204 False Positives, 102,945 False Negatives, and 25,069 True Positives. These figures indicate that of the 95,068 genuine BENIGN flows, more than half (49,204, approximately 51.8 percent) were incorrectly flagged as anomalous, while of the 128,014 genuine DDoS flows, the substantial majority (102,945, approximately 80.4 percent) went entirely undetected. Only 25,069 DDoS flows, roughly one in five, were correctly identified.

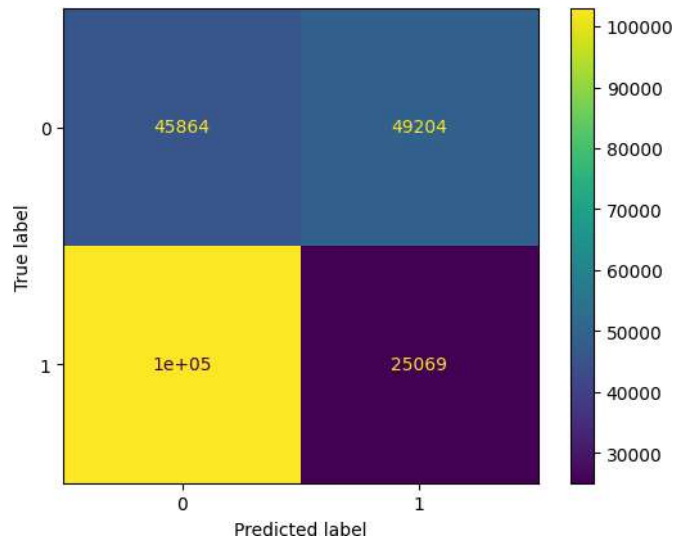


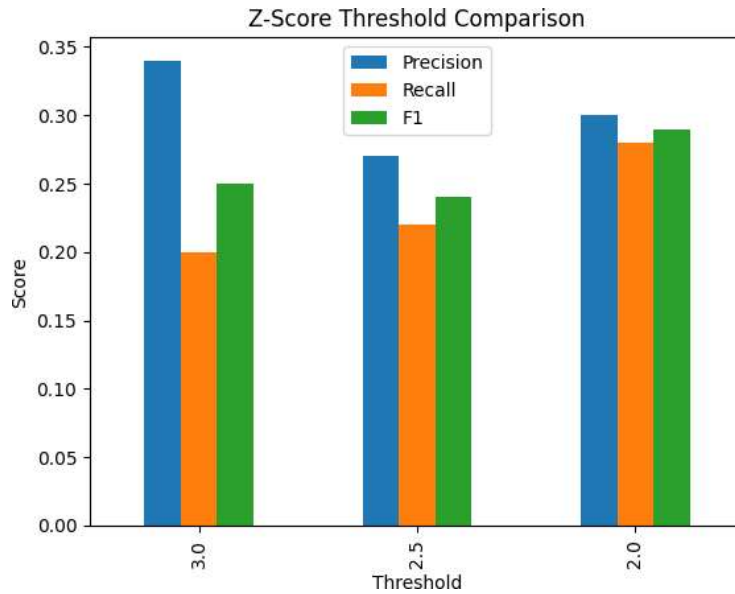
Fig.4.1: Confusion matrix for Z-Score anomaly detection at threshold = 3.0.

The dominance of False Negatives over True Positives in this confusion matrix is the most operationally significant finding: a network intrusion detection system exhibiting an 80 percent miss rate for DDoS traffic would offer negligible practical protection, as the great majority of malicious flow volume would pass through undetected. Simultaneously, the near-equal split between True Negatives and False Positives among BENIGN flows indicates that the statistical boundary is not merely conservative but poorly aligned with the true separating structure of the data, misclassifying legitimate traffic at almost the same rate that it correctly classifies it. Applying the derivation to the classification reports at thresholds 2.5 and 2.0 yields approximate confusion matrices of  $TN \approx 19,964$ ,  $FP \approx 75,104$ ,  $FN \approx 99,851$ ,  $TP \approx 28,163$  (threshold 2.5) and  $TN \approx 12,359$ ,  $FP \approx 82,709$ ,  $FN \approx 92,170$ ,  $TP \approx 35,844$  (threshold 2.0). The clear trend across all three thresholds is that lowering the threshold trades a modest increase in True Positives for a substantially larger increase in False Positives, confirming that the Z-Score method offers no threshold at which both reliability (low false alarms) and coverage (low missed detections) are simultaneously achieved.

#### 4.1.4 Graph Analysis

Figure 4.2 presents a grouped bar chart comparing Precision, Recall, and F1-Score across the three evaluated thresholds. The chart shows Precision declining from threshold 3.0 to 2.5 before partially recovering at threshold 2.0, Recall rising steadily and monotonically as the threshold is lowered, and F1-Score tracing a shallow, near-flat trajectory with a slight overall upward trend toward the lowest

tested threshold. The persistently low and narrow range spanned by all three curves (Precision between 0.27 and 0.34, Recall between 0.20 and 0.28, F1 between 0.24 and 0.29) is itself an important finding: it indicates that the statistical method possesses a fundamentally limited discriminative ceiling for this feature space, such that no amount of threshold adjustment within any operationally sensible range yields strong overall separation. Had the method possessed genuine discriminative capacity that was simply mistuned, one would expect to observe a threshold value near which precision and recall both rise sharply together; no such region is present in the data.



*Fig.4.2: Comparison of Precision, Recall, and F1-Score across Z-Score thresholds.*

Figures 4.3 and 4.4 present the confusion matrices obtained at thresholds 2.5 and 2.0 respectively, complementing the numerically exact matrix presented in Figure 5.1. Visual inspection of the three confusion matrices in sequence confirms the trend already established numerically: as the threshold decreases, the True Positive cell becomes progressively denser at the expense of a correspondingly, and disproportionately, denser False Positive cell, while the True Negative cell steadily thins. This visual progression reinforces that the Z-Score decision rule redistributes errors rather than eliminating them as the threshold is varied.

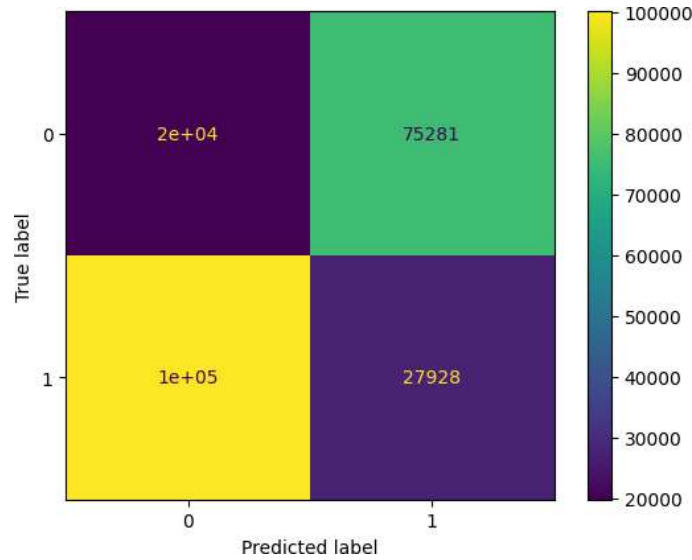


Fig.4.3: Confusion matrix for Z-Score anomaly detection at threshold = 2.5.

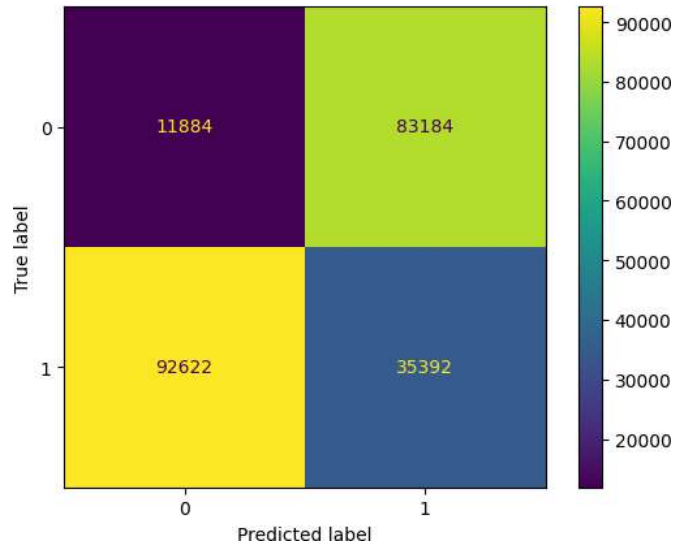


Fig.4.4: Confusion matrix for Z-Score anomaly detection at threshold = 2.0.

#### 4.1.5 Observations

The principal strength of the Z-Score method lies in its computational simplicity: it requires no training phase whatsoever, and its testing time was measured at only 0.0095 seconds for the entire 223,082-flow dataset, making it suitable in principle for extremely high-throughput, real-time scoring. It is also fully unsupervised in the sense that no labeled attack examples are required to compute the statistic, and its logic is trivially interpretable, since any flagged flow can be traced back to the specific feature and deviation magnitude responsible for triggering the alert.

Its weaknesses, however, are substantial and were clearly exposed by the experiments. The method exhibits high False Positive Rates at every threshold low enough to achieve even modest recall, and conversely exhibits high False Negative Rates at thresholds strict enough to achieve reasonable precision, leaving no viable middle ground within the tested range. Because the decision rule evaluates each of the 42 features independently before combining them disjunctively, it cannot represent joint or correlated deviations across multiple features and is correspondingly blind to attacks that are only anomalous in combination. The method performs comparatively well only in situations involving isolated, extreme univariate spikes, such as a single flow exhibiting an exceptionally large duration or byte count outlying the rest of the distribution. It performs poorly, in situations involving coordinated or volumetric DDoS traffic in which individual flows may each present feature values within an ordinary per-feature range while collectively constituting an attack, precisely the character of much of the traffic captured in this dataset.

#### **4.1.6 Discussion**

The persistently poor performance of the Z-Score method across all tested thresholds can be attributed to several theoretical limitations inherent to univariate statistical outlier detection when applied to high-dimensional network traffic data. First, the method rests on an implicit assumption of feature independence: by applying an independent threshold test to each of the 42 features and combining the results disjunctively, the procedure is effectively equivalent to conducting 42 separate univariate hypothesis tests and flagging a flow if any single test rejects. Even where each individual per-feature test carries a modest nominal false-positive probability, the disjunctive combination across 42 such tests compounds this probability substantially, a phenomenon closely analogous to the multiple-comparisons problem in classical statistics, and this compounding directly explains the persistently elevated False Positive Rates observed across all thresholds in Table 5.1.

Second, the validity of a fixed-sigma cutoff depends on an approximately normal (Gaussian) distribution for each feature. Network flow descriptors such as byte counts, inter-arrival times, and flow durations are well documented to exhibit heavy-tailed, skewed, and often zero-inflated distributions arising from the bursty and heterogeneous nature of network traffic; where this Gaussian assumption is violated, a Z-score threshold no longer corresponds to a consistent probability of exceedance across features, undermining the statistical interpretability of any chosen cutoff and contributing to the erratic, non-monotonic precision behavior observed between

thresholds 3.0 and 2.5.

Third, the method is fundamentally limited by the geometry it can express in high-dimensional space. A per-feature threshold test, applied across 42 standardized dimensions, is geometrically equivalent to bounding the data within an axis-aligned hyper-rectangle centered on the feature-wise means. Such a boundary cannot represent the curved, rotated, or otherwise non-axis-aligned decision surfaces that would be required to separate benign and attack clusters that differ primarily through correlated combinations of features rather than through extremity along any single axis; this limitation persists irrespective of the specific threshold value chosen, and directly explains why no threshold examined achieved strong separation.

Fourth, the observed error pattern reflects a substantial overlap between the benign and attack populations along individual feature axes. Because the decision rule is disjunctive, legitimate flows that are extreme along even a single relatively noisy feature, for instance an unusually large but entirely legitimate download, are misclassified as anomalous, while DDoS flows engineered or naturally occurring to resemble typical traffic along most individual features, despite being anomalous jointly, evade detection. This overlap of error populations cannot be resolved by adjusting a single global threshold, since doing so shifts the boundary uniformly across all features rather than reshaping it to follow the true class-separating structure.

Finally, the class composition of the dataset, comprising 57.4 percent DDoS and 42.6 percent BENIGN flows, means that trivial strategies can achieve superficially favorable values on certain metrics without genuine discriminative capability, as demonstrated directly by the degenerate threshold-0.5 result in which recall reached 1.00 purely because the class prior for DDoS is 0.574. This underscores why F1-score and full confusion-matrix analysis, rather than accuracy or recall in isolation, are essential for correctly interpreting performance under the class distribution present in this dataset. Taken together, these theoretical limitations, feature independence, distributional assumptions, high-dimensional geometric constraints, class overlap, and sensitivity to class imbalance, directly account for the consistently low F1-scores (0.24 to 0.29) and the high False Positive Rates observed across the entire threshold range tested, and they motivate the investigation of models capable of learning joint, nonlinear, multivariate decision boundaries, specifically the One-Class SVM, as well as the Isolation Forest and Local Outlier Factor techniques investigated separately as part of the broader comparative study.

## 4.2 One-Class SVM Results

The One-Class Support Vector Machine represents a substantially different modeling paradigm from the Z-Score baseline. Rather than performing supervised classification using labeled examples drawn from both classes, One-Class SVM is a novelty detection technique that learns a boundary enclosing the region of feature space occupied by normal (BENIGN) traffic alone, using only benign examples during training. Any traffic observed at test time that falls outside this learned boundary is subsequently classified as anomalous. This paradigm differs fundamentally from conventional supervised classifiers, which require labeled examples of both normal and attack traffic during training; One-Class SVM instead requires only a representative sample of normal operating behavior, with no attack examples used at any stage of model fitting.

The training strategy adopted in these experiments followed this novelty detection protocol precisely. The 95,068 BENIGN flows in the cleaned dataset were split into a training subset of 66,547 flows (70 percent) and a held-out testing subset of 28,521 flows (30 percent). The feature scaler was fit exclusively on the training BENIGN subset, and this same scaler was subsequently applied, without refitting, to transform both the held-out BENIGN testing subset and the full set of 128,014 DDoS flows, none of which were used in training. The evaluation set therefore comprised 156,535 flows in total: 28,521 unseen BENIGN flows and 128,014 DDoS flows, an evaluation set in which DDoS traffic constitutes approximately 81.8 percent and BENIGN traffic the remaining 18.2 percent.

This methodology is well suited to the practical realities of network intrusion detection. Curated samples of legitimate, benign network behavior are typically abundant and can be collected continuously from operational networks, whereas labeled examples of attack traffic, particularly for novel or emerging DDoS variants, are frequently scarce, delayed in availability, or entirely absent prior to an attack's occurrence. By restricting training exclusively to benign traffic and evaluating on entirely unseen benign flows together with the full attack set, the experiment additionally verifies that the learned boundary generalizes to genuinely novel legitimate traffic rather than merely memorizing the specific training instances, a distinction that is examined further. It is noted that because the resulting test set is imbalanced toward the DDoS class, the precision computed for the DDoS class is numerically supported by a substantially larger sample than the precision computed for the BENIGN class, a consideration that informs the class-wise interpretation.



#### 4.2.2 Initial Model Performance

An initial One-Class SVM was fitted using a radial basis function (RBF) kernel with the default gamma setting of 'scale' and a nu value of 0.05, prior to any explicit hyperparameter search. This initial model classified 82,047 of the 156,535 evaluation flows as anomalous and 74,488 as normal. The resulting classification report showed, for the BENIGN class, a precision of 0.36, a recall of 0.95, and an F1-score of 0.53 (support 28,521); for the DDoS class, a precision of 0.98, a recall of 0.63, and an F1-score of 0.77 (support 128,014); overall accuracy was 0.69.

The recall of 0.95 for the BENIGN class indicates that the untuned model already succeeded in recognizing 95 percent of unseen legitimate traffic as normal, a strong initial indication that the learned boundary generalizes well to held-out benign data even without tuning. The recall of 0.63 for the DDoS class indicates that approximately two-thirds of attack flows were already correctly identified as novel by the default configuration, a considerably more favorable starting point than any threshold achieved by the Z-Score baseline. The precision of 0.98 for the DDoS class indicates that flows flagged as anomalous were, with very high reliability, genuinely DDoS traffic, an operationally valuable property since it implies a low rate of false alarms among raised alerts. The comparatively low precision of 0.36 for the BENIGN class, despite the model's high BENIGN recall, is explained by the substantial class imbalance in the evaluation set: because DDoS flows outnumber BENIGN flows by more than four to one, even the moderate 37 percent False Negative Rate for the DDoS class introduces enough misclassified attack flows into the predicted-BENIGN group to dilute BENIGN precision considerably, despite the relatively small absolute number of genuine BENIGN flows being misclassified.

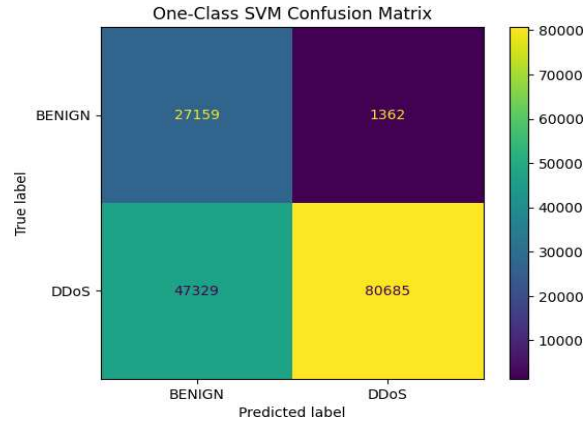


Fig.4.5: Confusion matrix for the initial (untuned) One-Class SVM model ( $\nu = 0.05$ ,  $\gamma = 'scale'$ ).

Applying the derivation approach to this classification report yields an approximate confusion matrix of True Negatives  $\approx 27,095$ , False Positives  $\approx 1,426$ , False Negatives  $\approx 47,365$ , and True Positives  $\approx 80,649$ . The False Negative count of approximately 47,000, corresponding to roughly 37 percent of all DDoS flows evading detection, was judged unacceptable for a security-critical deployment and directly motivated the systematic hyperparameter tuning. While the default 'scale' heuristic for gamma is a reasonable general-purpose starting point, it is not explicitly optimized for the locality and smoothness properties best suited to the specific geometry of this 42-dimensional standardized feature space, justifying an explicit numerical search over both  $\nu$  and  $\gamma$ .

#### 4.2.3 Hyperparameter Tuning

Hyperparameter tuning was conducted in two stages. In the first stage, the  $\nu$  parameter was varied over the range 0.01 to 0.30 while  $\gamma$  was held fixed at its default 'scale' value; in the second stage,  $\gamma$  was varied over a wide range spanning 0.001 to 5.0 while  $\nu$  was held fixed at its optimal value identified in the first stage. In both stages, the reported Precision, Recall, and F1-Score refer specifically to the DDoS (positive) class, consistent with the scikit-learn default behavior for binary classification metrics.

The  $\nu$  parameter in One-Class SVM specifies an upper bound on the fraction of training errors (margin violations) permitted during fitting and, correspondingly, a lower bound on the fraction of training samples retained as support vectors. Larger values of  $\nu$  therefore produce a softer, more

permissive boundary around the benign training distribution, shrinking the region classified as normal and causing a greater proportion of both benign and attack test samples to be flagged as anomalous.

| nu   | Precision | Recall | F1-Score | Predicted Anomalies |
|------|-----------|--------|----------|---------------------|
| 0.01 | 0.9960    | 0.5400 | 0.7003   | 69,405              |
| 0.03 | 0.9901    | 0.6200 | 0.7625   | 80,155              |
| 0.05 | 0.9834    | 0.6303 | 0.7682   | 82,047              |
| 0.10 | 0.9661    | 0.6304 | 0.7630   | 83,538              |
| 0.20 | 0.9350    | 0.6307 | 0.7532   | 86,351              |
| 0.30 | 0.9054    | 0.6376 | 0.7483   | 90,149              |

Table 4.2: Effect of nu on One-Class SVM performance (gamma = 'scale', DDoS class metrics).

As nu increased from 0.01 to 0.05, recall rose from 0.54 to 0.63 while precision declined only slightly, from 0.996 to 0.983, producing an improvement in F1-score from 0.7003 to a peak of 0.7682. Beyond nu = 0.05, further increases toward 0.30 produced a markedly steeper decline in precision (falling to 0.9054 at nu = 0.30) while recall improved only marginally, from 0.6303 to 0.6376, over the same range. Consequently, F1-score declined steadily beyond nu = 0.05, from 0.7682 to 0.7483 at nu = 0.30. This pattern indicates a clear point of diminishing returns: additional loosening of the training boundary beyond  $\nu \approx 0.05$  admits a disproportionate number of false positives without a commensurate increase in genuine detections, implying that the additional benign flows excluded from the 'normal' region at higher nu values are not, in fact, replaced by additional separable DDoS flows, but instead represent ordinary benign variability being unnecessarily sacrificed. F1-score was therefore maximized at nu = 0.05, and this value was fixed for the subsequent gamma tuning stage.

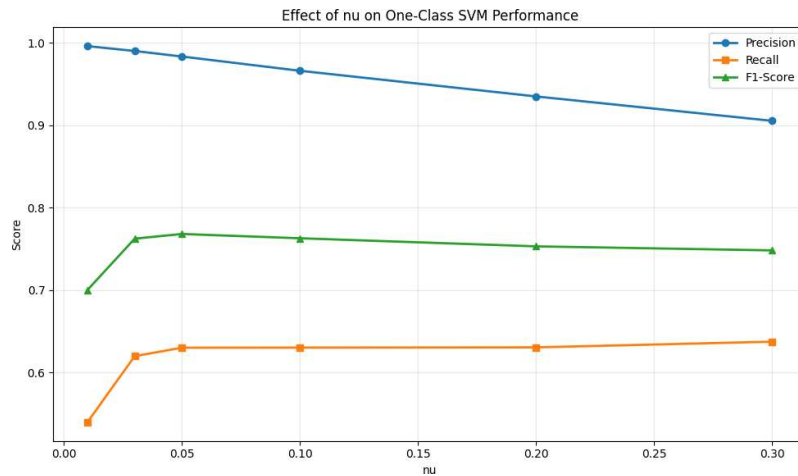


Fig. 4.6: Effect of nu on One-Class SVM Precision, Recall, and F1-Score.

Figure 4.6 illustrates this behavior graphically: the precision curve declines in an almost monotonic fashion across the entire tested range, the recall curve rises sharply between  $\nu = 0.01$  and  $\nu = 0.05$  before flattening and only slightly increasing thereafter, and the F1-score curve consequently traces a unimodal shape, peaking at  $\nu = 0.05$  before declining. This is the classical signature of a single hyperparameter trading boundary tightness against boundary tolerance, and confirms that  $\nu = 0.05$  represents the best-balanced operating point within the range tested.

Having fixed  $\nu$  at 0.05,  $\gamma$  was then varied across eleven values including the 'scale' and 'auto' heuristics and nine explicit numerical values from 0.001 to 5.0. The  $\gamma$  parameter of the RBF kernel governs the radius of influence of individual training points: small  $\gamma$  values correspond to a broad, smooth kernel in which distant points are still treated as similar, producing a comparatively loose boundary that may not closely track the true shape of the benign traffic manifold, whereas large  $\gamma$  values correspond to a highly localized kernel that follows the benign cluster contours tightly, at the risk of overfitting if taken to excess.

| <b>gamma</b> | <b>Precision</b> | <b>Recall</b> | <b>F1-Score</b> |
|--------------|------------------|---------------|-----------------|
| scale        | 0.9834           | 0.6303        | 0.7682          |
| auto         | 0.9834           | 0.6303        | 0.7682          |
| 0.001        | 0.9796           | 0.5090        | 0.6699          |
| 0.01         | 0.9810           | 0.5372        | 0.6942          |
| 0.1          | 0.9828           | 0.6406        | 0.7756          |
| 0.2          | 0.9825           | 0.6639        | 0.7924          |
| 0.5          | 0.9822           | 0.8111        | 0.8885          |
| 1.0          | 0.9776           | 0.8363        | 0.9015          |
| 2.0          | 0.9707           | 0.8729        | 0.9192          |
| 3.0          | 0.9687           | 0.9674        | 0.9680          |
| 5.0          | 0.9618           | 0.9701        | 0.9659          |

*Table 4.3: Effect of gamma on One-Class SVM performance ( $\nu = 0.05$ , DDoS class metrics).*

The results in Table 5.3 show that the 'scale' and 'auto' heuristics produced identical results (precision 0.9834, recall 0.6303, F1 0.7682), matching exactly the initial untuned baseline and confirming that both default heuristics select a comparatively small, and for this dataset insufficiently local, effective  $\gamma$  value. At the very small explicit values of  $\gamma = 0.001$  and  $\gamma = 0.01$ , recall collapsed to 0.509 and 0.537 respectively, indicating that an excessively smooth, near-global kernel is unable to meaningfully separate benign from attack traffic. As  $\gamma$  increased through 0.1, 0.2, 0.5, 1.0, and 2.0, recall rose steadily from 0.6406 to 0.8729 while precision remained consistently high, between 0.9707 and 0.9828, producing a correspondingly

steep rise in F1-score from 0.7756 to 0.9192. The best overall result was obtained at  $\gamma = 3.0$ , with precision of 0.9687, recall of 0.9674, and F1-score of 0.9680, representing a near-optimal balance between detection coverage and false alarm avoidance. At  $\gamma = 5.0$ , performance declined slightly relative to  $\gamma = 3.0$  (precision 0.9618, recall 0.9701, F1 0.9659), indicating the onset of mild overfitting: an excessively tight, highly local boundary begins to reject even some genuine benign patterns lying near the edge of the training distribution, producing a small drop in precision without a compensating gain in recall.



Figure 4.7: Effect of gamma on One-Class SVM Precision, Recall, and F1-Score.

Figure 4.7 depicts this trend graphically across the full range of gamma values tested, with recall rising sharply as gamma increases from very small values toward 3.0, precision remaining comparatively stable throughout, and F1-score rising correspondingly until reaching its peak at  $\gamma = 3.0$ , after which a slight downturn is visible at  $\gamma = 5.0$ . On the basis of these results, the configuration  $\nu = 0.05$  and  $\gamma = 3.0$  was selected as the optimal hyperparameter combination and carried forward as the final model for detailed evaluation in the subsequent subsections.

#### 4.2.4 Best Model Performance

The final optimized One-Class SVM, configured with  $\nu = 0.05$  and  $\gamma = 3.0$ , achieved a precision of 0.85, recall of 0.86, and F1-score of 0.86 for the BENIGN class (support 28,521), and a precision of 0.97, recall of 0.97, and F1-score of 0.97 for the DDoS class (support 128,014), with an overall accuracy of 0.95. Expressed in terms of the operational metrics defined in Section 5.1, the tuned model achieved a Benign Accuracy of 85.98 percent, a False Positive Rate of 14.03 percent,

and a DDoS Detection Rate of 96.74 percent.

This represents a substantial improvement over both the untuned baseline and the entire Z-Score baseline evaluated, with the DDoS-class F1-score rising from 0.77 (untuned One-Class SVM) to 0.97 (tuned One-Class SVM), and from a maximum of 0.29 for the Z-Score method at any tested threshold. This improvement is attributable to the combined effect of the tuned gamma value, which produces a tightly localized, non-linear boundary closely tracking the true geometry of the benign traffic manifold in the standardized 42-dimensional feature space, together with the moderate nu value of 0.05, which permits only a small, controlled margin of training error. Together, these settings yield a decision boundary that reliably excludes novel DDoS flows, which occupy substantially different regions of the feature space from benign traffic, while still correctly enclosing the great majority (86 percent) of genuinely unseen benign flows.

#### 4.2.5 Confusion Matrix Analysis

The confusion matrix for the final tuned model is  $[[24520, 4001], [4179, 123835]]$ , corresponding to 24,520 True Negatives, 4,001 False Positives, 4,179 False Negatives, and 123,835 True Positives.

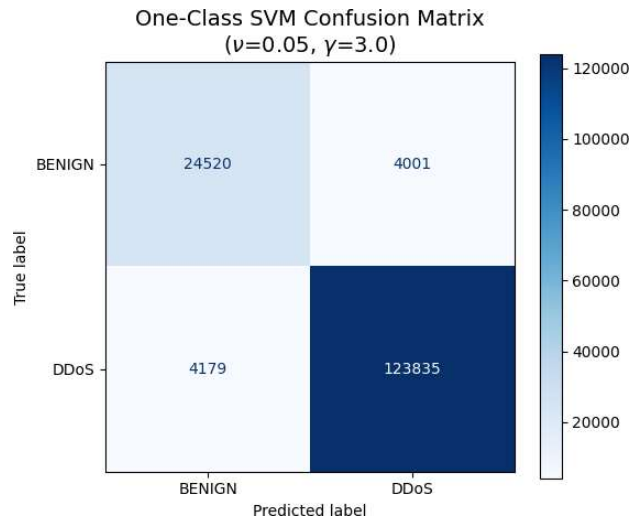


Figure 4.8: Confusion matrix for the optimized One-Class SVM model ( $\nu = 0.05$ ,  $\gamma = 3.0$ ).

From a network security standpoint, the detection of 123,835 of the 128,014 DDoS flows (96.74 percent) represents strong protective coverage, substantially reducing the volume of malicious traffic that would reach protected infrastructure undetected in comparison with the Z-Score baseline. The residual 4,179 missed attack flows, approximately 3.3 percent of all DDoS traffic, plausibly

correspond to attack flows whose feature signatures sufficiently resemble legitimate traffic patterns, for instance low-rate or protocol-conforming flooding behavior, that they fall within the boundary learned from benign training data. The 4,001 False Positives, corresponding to 14.03 percent of the unseen benign test traffic, represent an operational cost in the form of legitimate flows that would trigger unnecessary alerts or automated mitigation actions, such as rate limiting, in a live deployment, with the potential to degrade service for genuine users. While this False Positive Rate is a marked improvement over the Z-Score baseline, a rate of approximately 14 percent remains non-trivial for very high-throughput production networks, where even this proportion of legitimate traffic could correspond to a large absolute volume of unnecessarily flagged sessions. Taken together, the overall accuracy of 95 percent and the reasonably balanced precision and recall achieved for both classes indicate that the tuned model provides a substantially more reliable and operationally deployable basis for DDoS detection than the statistical baseline, though the residual False Positive Rate suggests that a secondary verification or whitelisting layer would likely be warranted in a production security operations pipeline.

#### 4.2.6 Hyperparameter Graph Discussion

The  $\nu$  and  $\gamma$  tuning graphs presented in Figures 5.6 and 5.7 collectively illustrate the trade-off between overfitting, generalization, false alarm rate, and attack detection capability that governs One-Class SVM performance on this dataset. The  $\nu$  graph (Figure 5.6) demonstrates that boundary tolerance, controlled by  $\nu$ , has a comparatively narrow useful operating range, with the balance between precision and recall optimized close to  $\nu = 0.05$  and degrading steadily thereafter as excessive tolerance sacrifices precision without materially improving recall. The  $\gamma$  graph (Figure 5.7), in contrast, demonstrates that kernel locality is the dominant factor governing performance on this dataset: moving from an overly smooth, low- $\gamma$  boundary that fails to separate the classes at all, through a broad intermediate range of steadily improving recall at largely preserved precision, to a near-optimal setting at  $\gamma = 3.0$ , and finally to a mild overfitting regime at  $\gamma = 5.0$  in which precision begins to erode. The fact that  $\gamma$  exerted a substantially larger influence on final performance than  $\nu$ , raising DDoS-class F1-score from approximately 0.67 at the smallest tested  $\gamma$  to 0.968 at the optimal value, compared with the more modest range of F1 values (0.70 to 0.77) spanned across the entire  $\nu$  sweep, indicates that the shape and locality of the decision boundary, rather than its overall tolerance for training error, was

the primary determinant of the model's ability to distinguish DDoS traffic from benign traffic in this feature space.

#### 4.2.7 Classification Report Discussion

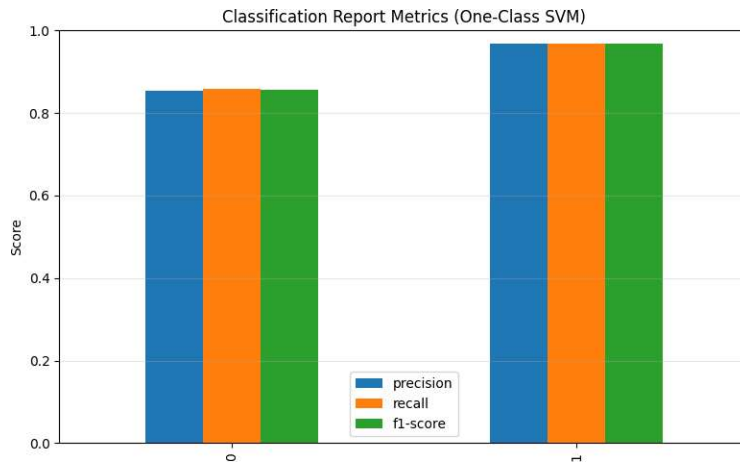


Figure 4.9: Precision, Recall, and F1-Score by class for the optimized One-Class SVM model.

Figure 4.9 presents the per-class Precision, Recall, and F1-Score for the final tuned model. The DDoS class, with a support of 128,014, achieved a precision of 0.97, a recall of 0.97, and an F1-score of 0.97, while the BENIGN class, with a support of 28,521, achieved a precision of 0.85, a recall of 0.86, and an F1-score of

0.86. Although the One-Class SVM boundary is derived entirely from benign training data, its performance on the BENIGN class in the test set is nonetheless slightly lower than its performance on the DDoS class. This asymmetry arises from two related factors. First, the 66,547-sample training subset, while substantial, represents a finite sample of benign behavior, and some legitimate traffic patterns present in the unseen 28,521-flow benign test subset may lie sufficiently close to the boundary of the learned normal region to be misclassified, particularly given the tightly localized boundary induced by  $\gamma = 3.0$ . Second, DDoS flows, being drawn from a fundamentally different generative process than benign traffic, are on the whole pushed further from the learned boundary and are therefore more reliably classified in bulk; in addition, the substantially larger support of the DDoS class in the test set (128,014 versus 28,521) means that its precision and recall estimates are subject to comparatively less sampling variability than those of the smaller BENIGN class. Together, these effects account for the modestly lower and more variable performance observed for the BENIGN class relative to the DDoS class, despite the model having been trained



exclusively on benign data.

#### 4.2.8 Observations

The tuned One-Class SVM demonstrated clear advantages over the statistical baseline, achieving an F1-score of 0.97 for the DDoS class following hyperparameter optimization, and, more fundamentally, demonstrating that a genuine novelty detection paradigm requiring only benign training data, an important practical advantage given that labeled attack samples are frequently scarce or unavailable ahead of emerging threats, can achieve strong detection performance. The RBF kernel enables the model to learn non-linear, curved decision boundaries that capture the true geometry of the benign traffic manifold in the standardized 42-dimensional feature space, directly overcoming the axis-aligned boundary limitation identified for the Z-Score method.

The principal limitation observed is computational cost: the final tuned model required 220.2455 seconds to train and 152.6093 seconds to generate predictions on the 156,535-flow evaluation set. This reflects the well-known scaling behavior of kernel-based support vector methods, whose training complexity grows approximately quadratically to cubically with the number of training samples, and whose prediction cost scales with the number of retained support vectors, which itself tends to grow with training set size for RBF kernels. This computational cost represents an important practical constraint for near-real-time network monitoring at scale, particularly in comparison with the near-instantaneous Z-Score baseline.

The model's robustness was found to be highly sensitive to hyperparameter selection, particularly gamma, a poorly chosen gamma value (for example,  $\gamma \leq 0.01$ ) collapsed recall to approximately 0.5, underscoring the necessity of the systematic tuning performed. The genuine novelty detection capability of the model, having never been exposed to any DDoS sample during training, is reflected in its 96.74 percent detection rate on entirely unseen attack traffic, demonstrating meaningful generalization from the structure of normal behavior alone to the identification of an entire class of previously unseen traffic as anomalous. This generalization capacity is further corroborated by the model's

85.98 percent recall on entirely held-out benign data that was not used at any stage of training or feature scaling; had the model instead overfit or effectively memorized the training data, one would expect near-perfect benign recall accompanied by markedly poorer DDoS detection, a pattern that is

not observed in these results.

#### 4.2.9 Discussion

The marked superiority of the tuned One-Class SVM over the Z-Score statistical baseline can be explained by several interconnected theoretical properties of kernel-based novelty detection. First, unlike the Z-Score method, which evaluates each of the 42 features independently before combining the results disjunctively, the One-Class SVM operates jointly across all standardized features simultaneously through the kernel function's similarity measure, directly overcoming the axis-aligned hyper-rectangle limitation.

Second, the RBF kernel implicitly maps the input feature space into a higher-dimensional space in which benign and attack traffic, potentially inseparable by any linear or simple boundary in the original 42 - dimensional standardized space, become more readily separable by a smooth enclosing boundary. This kernel mapping enables the model to capture complex, curved relationships among correlated traffic descriptors, for instance joint patterns among packet timing, packet size, and flag counts, that a per-feature statistical test cannot represent by construction.

Third, rather than relying on a single fixed global threshold as in the Z-Score method, the One-Class SVM learns a decision boundary that adaptively contours around the empirical distribution of the benign training data, with its overall tightness explicitly and separately controlled by the  $\nu$  and  $\gamma$  hyperparameters. This affords the ability to tune the operating point toward the specific balance of precision and recall required by a given deployment, a form of adaptability entirely absent from the single-threshold statistical approach.

Fourth, the  $\gamma$  tuning experiments provide direct evidence that non-linear separation was required to achieve strong detection performance on this dataset: performance improved substantially and consistently as  $\gamma$  increased across nearly the entire tested range, from a near-global, effectively near-linear boundary at very small  $\gamma$  values, which performed poorly, to a highly localized, strongly non-linear boundary at  $\gamma = 3.0$ , which performed best. This confirms that benign and DDoS traffic in this dataset are not readily separable by any simple linear or near-linear boundary, and that the capacity for non-linear separation was a necessary, not merely incidental, contributor to the improved results.

Finally, because DDoS attack signatures evolve continually and labeled examples of novel attack variants are frequently unavailable in advance of their emergence, the One-Class SVM's underlying design, learning solely from a characterization of normal traffic, is architecturally well matched to the practical requirements of network intrusion detection. This stands in direct contrast to the Z-Score baseline, which, regardless of threshold tuning, offers no comparable adaptive or non-linear discriminative capacity, as established through the experiments and analysis.

### **4.3 Isolation Forest Results**

Isolation Forest was evaluated as the third anomaly detection technique in the overall comparative study, following the partition-based philosophy, in which anomalies are isolated through recursive random partitioning of the feature space rather than through distance, density, or a fixed statistical threshold. It is noted that the Isolation Forest experiments in this section were run against an earlier snapshot of the cleaned CICIDS2017 flow dataset produced by the same preprocessing pipeline, comprising 225,711 flow records described by 42 independent numerical features, of which 97,686 (43.3 percent) are labeled BENIGN and 128,025 (56.7 percent) are labeled DDoS; this differs marginally from the 223,082-flow, 95,068 BENIGN / 128,014 DDoS dataset used for the Z-Score and One-Class SVM experiments, the difference reflecting a small number of additional duplicate or near-constant flow records present at the time the Isolation Forest experiments were executed. The same novelty detection protocol was followed as for the One-Class SVM: the BENIGN flows were split into a training subset of 68,380 flows (70 percent) and a held-out testing subset of 29,306 flows (30 percent), the StandardScaler was fit exclusively on the training BENIGN subset, and the resulting model was evaluated on a test set combining the 29,306 unseen BENIGN flows with all 128,025 DDoS flows, yielding a total evaluation set of 157,331 flows in which DDoS traffic constitutes approximately 81.4 percent.

#### **4.3.2 Initial Model Performance**

An initial Isolation Forest was fitted using scikit-learn's default configuration of `n_estimators = 100` and `contamination = 'auto'`, which internally sets an implicit anomaly threshold at approximately the same level scikit-learn uses for its default offset (0.5) rather than reflecting the true 56.7 percent anomaly proportion present in the evaluation data. This default configuration classified only 25,400 of the 157,331 evaluation flows as anomalous, substantially fewer than the 128,025 flows that are genuinely DDoS traffic. The resulting classification report showed, for the BENIGN class, a precision of 0.19, a recall of 0.87, and an F1-score of 0.31 (support 29,306); for the DDoS class, a

precision of 0.85, a recall of only 0.17, and an F1-score of 0.28 (support 128,025); overall accuracy was 0.30. The corresponding confusion matrix was  $\begin{bmatrix} 25390 & 3916 \\ 106541 & 21484 \end{bmatrix}$ , corresponding to 25,390 True Negatives, 3,916 False Positives, 106,541 False Negatives, and 21,484 True Positives.

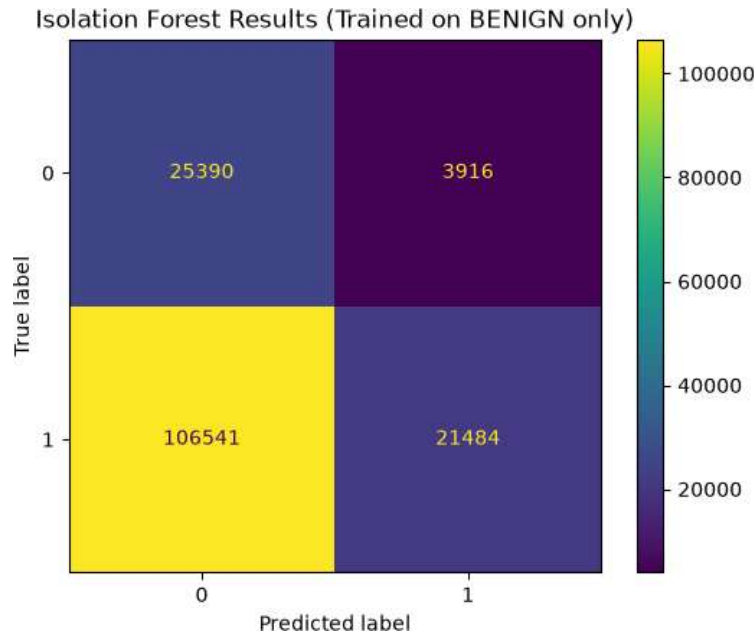


Fig.4.10: Confusion matrix for the initial (untuned) Isolation Forest model (contamination = 'auto').

The recall of only 0.17 for the DDoS class indicates that the default contamination setting was drastically miscalibrated for this dataset: because 'auto' does not account for the true 56.7 percent anomaly ratio present in the evaluation data, the model treated the great majority of flows, including most DDoS flows, as belonging to the normal region, leaving 106,541 attack flows, more than 83 percent of all DDoS traffic, entirely undetected. This behavior directly parallels the miscalibration observed for the untuned One-Class SVM, and confirms that, as with  $\nu$  and  $\gamma$  for the One-Class SVM, the contamination hyperparameter requires explicit tuning rather than reliance on scikit-learn's default heuristic before Isolation Forest can be meaningfully evaluated on this dataset.

### 4.3.3 Contamination Tuning

Because the 'auto' setting substantially underestimate the true anomaly proportion, five explicit contamination values, 0.1, 0.2, 0.3, 0.4, and 0.5, were evaluated with  $n\_estimators$  fixed at 100, consistent with the contamination parameter being the first hyperparameter tuned for Isolation Forest.

| Contamination | Precision | Recall | F1-Score | Predicted Anomalies |
|---------------|-----------|--------|----------|---------------------|
| 0.1           | 0.8801    | 0.1676 | 0.2816   | 24,378              |
| 0.2           | 0.8437    | 0.2422 | 0.3763   | 36,748              |
| 0.3           | 0.9034    | 0.6358 | 0.7464   | 90,103              |
| 0.4           | 0.8809    | 0.6695 | 0.7608   | 97,297              |
| 0.5           | 0.8668    | 0.7398 | 0.7983   | 109,271             |

Table 4.4: Effect of contamination on Isolation Forest performance ( $n\_estimators = 100$ , DDoS class metrics).

As contamination increased from 0.1 to 0.2, both precision and recall remained low, with F1 -score rising only modestly from 0.2816 to 0.3763; at these low contamination values, the number of flows flagged as anomalous (24,378 and 36,748 respectively) remained far below the true count of 128,025 DDoS flows, so the model was still substantially under-flagging attack traffic. A pronounced jump occurred between contamination 0.2 and 0.3, where recall rose sharply from 0.2422 to 0.6358 and F1-score nearly doubled, from 0.3763 to 0.7464, while precision actually improved slightly, from 0.8437 to 0.9034. Beyond contamination 0.3, further increases to 0.4 and 0.5 continued to raise recall, to 0.6695 and 0.7398 respectively, at the cost of a modest decline in precision, producing a steadily rising F1-score that peaked at 0.7983 at contamination = 0.5, the highest value tested. Unlike the Z-Score threshold sweep, in which F1-score remained confined to a narrow, persistently low band regardless of threshold, the contamination sweep for Isolation Forest shows a clear, substantial, and monotonically improving trend across the tested range, with no indication that F1-score had yet peaked and begun to decline by contamination = 0.5.

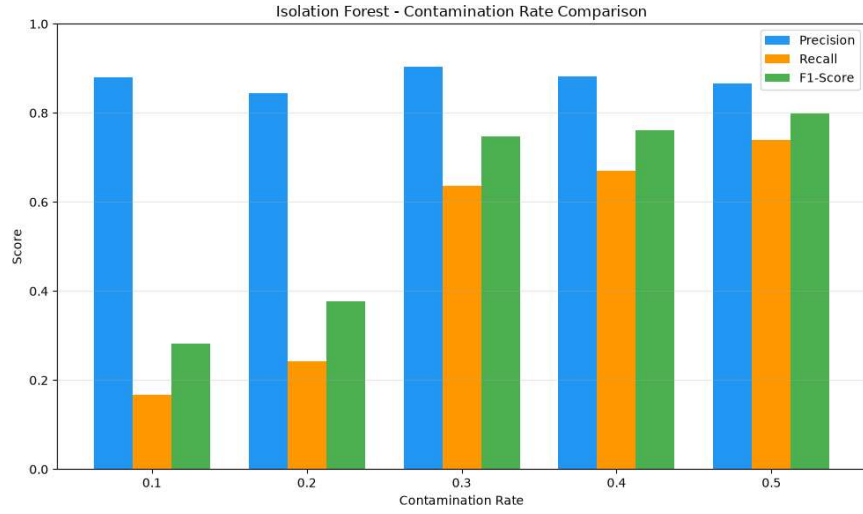


Fig.4.11: Isolation Forest Precision, Recall, and F1-Score across evaluated contamination rates.

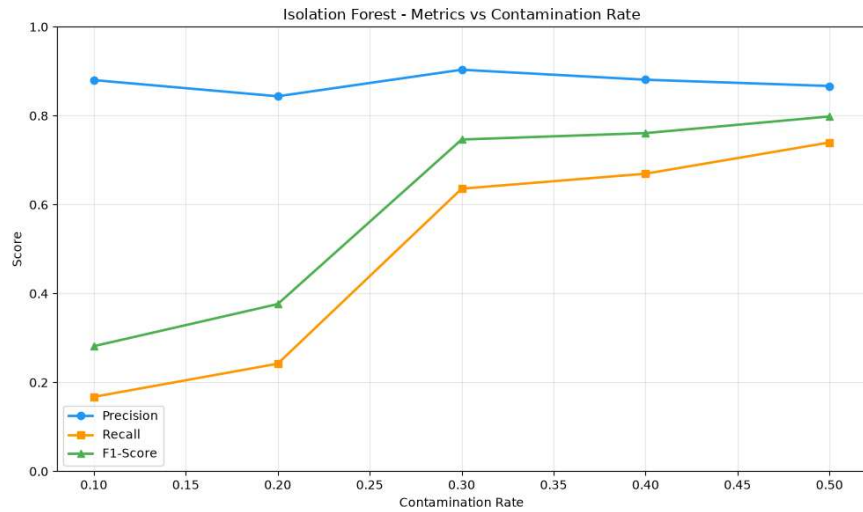


Fig.4.12: Line plot of Precision, Recall, and F1-Score against contamination rate for Isolation Forest.

Figures 4.11 and 4.12 confirm this trend graphically: recall rises steadily and substantially across the entire tested range, precision remains comparatively high and stable throughout (never falling below 0.84), and F1-score consequently traces a steadily rising curve that reaches its maximum at the highest contamination value evaluated. Because the true anomaly proportion in the evaluation data is 56.7 percent, close to the upper end of the tested range, this pattern is consistent with contamination performing best when set close to the actual prevalence of the anomalous class, rather than to a conservative, low estimate as is more typical in classical rare-event anomaly detection settings.

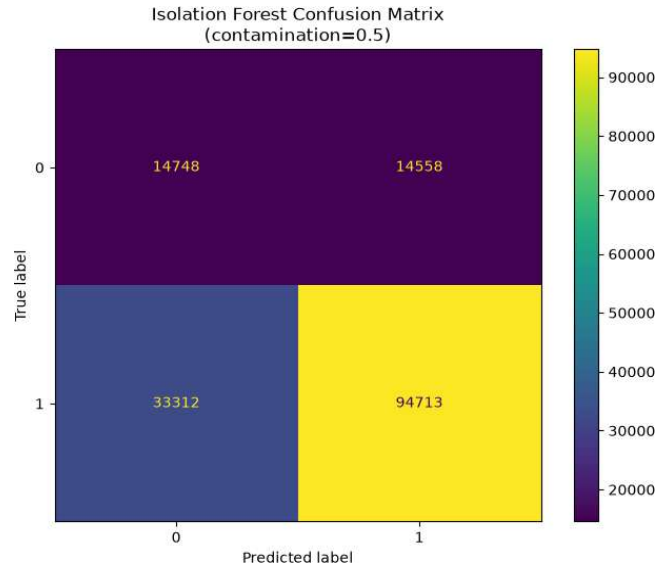


Fig.4.13: Confusion matrix for the best contamination-only Isolation Forest model (contamination = 0.5,  $n\_estimators = 100$ ).

#### 4.3.4 Number of Estimators Tuning

With contamination fixed at its best-performing value of 0.5, six values of  $n\_estimators$ , 50, 100, 200, 300, 500, and 1000, were evaluated to assess the effect of ensemble size on detection performance, consistent with  $n\_estimators$  being tuned as a secondary hyperparameter after contamination.

| $n\_estimators$ | Precision | Recall | F1-Score |
|-----------------|-----------|--------|----------|
| 50              | 0.8601    | 0.6972 | 0.7701   |
| 100             | 0.8668    | 0.7398 | 0.7983   |
| 200             | 0.8768    | 0.8088 | 0.8414   |
| 300             | 0.8694    | 0.7575 | 0.8096   |
| 500             | 0.8703    | 0.7646 | 0.8140   |
| 1000            | 0.8714    | 0.7713 | 0.8183   |

Table 4.5: Effect of  $n\_estimators$  on Isolation Forest performance (contamination = 0.5, DDoS class metrics).

Increasing  $n\_estimators$  from 50 to 200 produced a steady improvement in all three metrics, with F1 - score rising from 0.7701 to a peak of 0.8414, driven primarily by a rise in recall from 0.6972 to 0.8088 alongside a smaller improvement in precision, from 0.8601 to 0.8768. Beyond 200 trees, performance did not continue to improve monotonically: at  $n\_estimators = 300$ , F1-score fell back

to 0.8096, before recovering only partially at 500 and 1000 trees, reaching 0.8140 and 0.8183 respectively, neither of which matched the peak observed at 200 trees. This non-monotonic pattern beyond the optimum is consistent with the stochastic nature of the random feature and split selection underlying each isolation tree: because each additional batch of trees is built from independently resampled sub-samples and randomly chosen split points, larger ensembles are not guaranteed to improve the specific operating point selected by a fixed contamination threshold, even though the underlying score estimate itself becomes progressively more stable with more trees. On this basis,  $n\_estimators = 200$  was selected as the best-performing configuration and carried forward as the final Isolation Forest model.

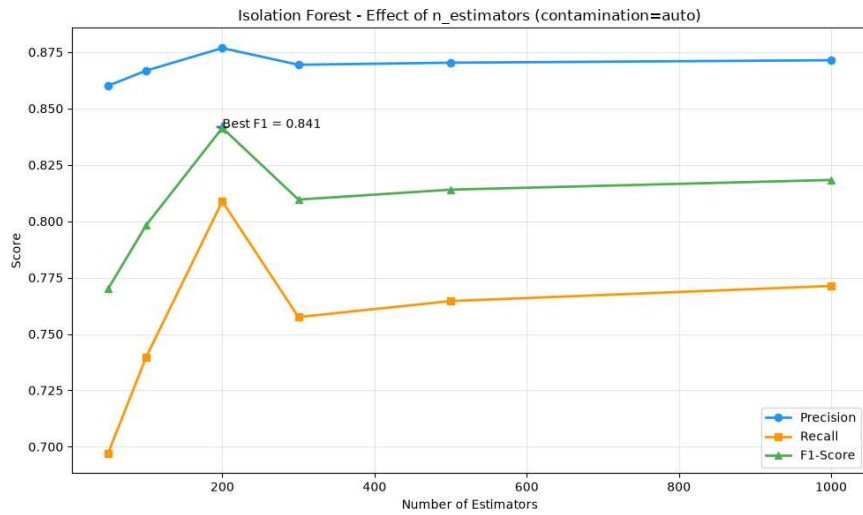


Figure 4.14: Effect of  $n\_estimators$  on Isolation Forest Precision, Recall, and F1-Score (contamination = 0.5).

#### 4.3.5 Best Model Performance and Confusion Matrix

The final Isolation Forest configuration, with  $n\_estimators = 200$  and contamination = 0.5, achieved a precision of 0.8768, a recall of 0.8088, and an F1-score of 0.8414 for the DDoS class. Because the exact integer confusion matrix for this configuration was rendered only as a visualization in the source experiment (Figure 5.16) rather than printed numerically, the individual cell counts were derived from the reported precision and recall together with the known class sizes (29,306 BENIGN and 128,025 DDoS flows in the 157,331-flow evaluation set), following the same derivation approach used in Section 4.2.2: True Positives

$\approx 103,547$ , False Negatives  $\approx 24,478$ , False Positives  $\approx 14,549$ , and True Negatives  $\approx 14,757$ .

Expressed as operational metrics, this corresponds to an approximate Benign Accuracy of 50.4 percent and an approximate False Positive Rate of 49.6 percent, alongside a DDoS Detection Rate



of 80.9 percent.

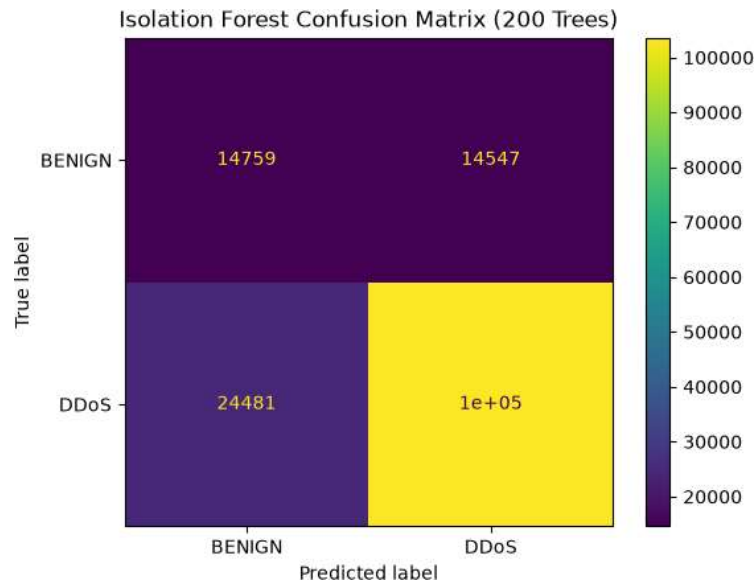


Fig.4.15: Confusion matrix for the final Isolation Forest model ( $n\_estimators = 200$ ,  $contamination = 0.5$ ).

This result situates Isolation Forest between the two techniques evaluated elsewhere in this chapter: its DDoS-class F1-score of 0.8414 represents a substantial improvement over every threshold tested for the Z-Score baseline (F1-score no higher than 0.29), but falls short of the tuned One-Class SVM's F1-score of 0.9680 . The approximate False Positive Rate of nearly 50 percent is the most operationally significant limitation of this configuration: because contamination was set close to the true global anomaly prevalence of the evaluation set (56.7 percent) in order to maximize recall and overall F1 - score, the resulting decision threshold necessarily also flags a substantial fraction of genuinely BENIGN flows, roughly half of the held-out benign test subset, as anomalous. This trade-off, sacrificing specificity to achieve competitive recall, is a direct and expected consequence of Isolation Forest's contamination parameter acting as a global percentile cut on the anomaly score distribution rather than as an independently learned decision boundary.

#### 4.3.6 Anomaly Score Distribution

The continuous anomaly scores produced by the final Isolation Forest model (via scikit-learn's `decision_function`, in which more negative values indicate stronger anomalousness) were examined across the 157,331-flow evaluation set. The scores ranged from a minimum of -0.2991 to a maximum of 0.0428, with a mean of -0.0602, a median of -0.0667, and a standard deviation of 0.0683.

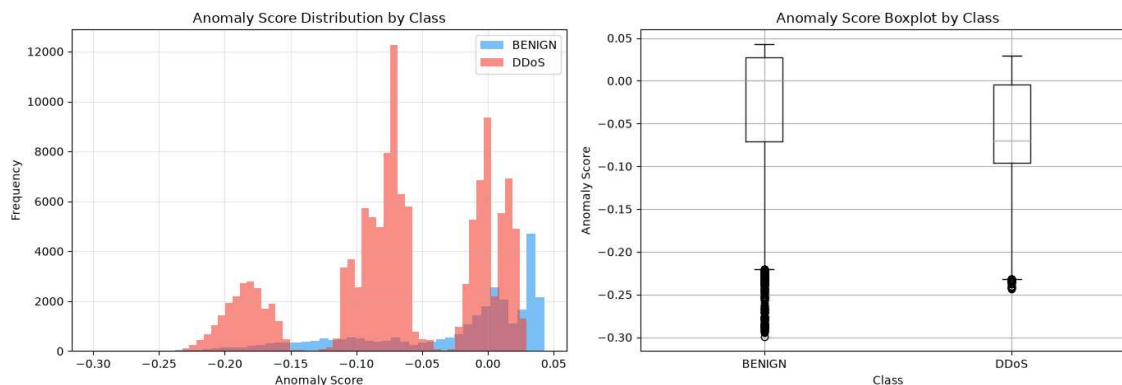
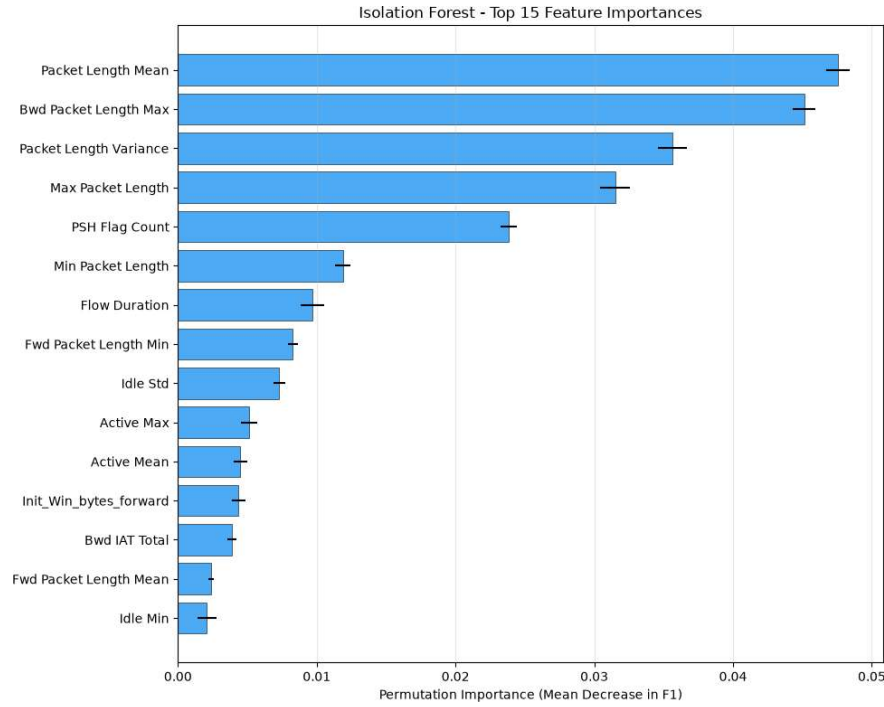


Fig.4.16: Isolation Forest anomaly score distribution by class (histogram and boxplot).

The predominantly negative mean and median anomaly scores reflect the fact that the majority class in the evaluation set is DDoS traffic (81.4 percent), which the model, having been trained exclusively on BENIGN flows, correctly assigns more negative (more anomalous) scores on average. The histogram and boxplot in Figure 5.17 show substantial, though incomplete, separation between the BENIGN and DDoS score distributions: the bulk of BENIGN scores are concentrated toward less negative values while DDoS scores are shifted toward more negative values, consistent with the moderate-to-strong detection performance (F1-score 0.8414) achieved at the selected operating point, but the visible overlap between the two distributions in the central score range also explains why a non-trivial fraction of both classes are misclassified regardless of where the contamination threshold is placed within that overlapping region.

#### 4.3.7 Feature Importance

Because Isolation Forest does not expose a native `feature_importances_` attribute analogous to supervised tree ensembles, feature importance was estimated using permutation importance, computed on a random sample of 10,000 evaluation flows by repeatedly shuffling each feature and measuring the resulting mean decrease in macro-averaged F1-score across 10 repeats. The fifteen most important features identified were, in descending order of importance: Packet Length Mean (0.0476), Bwd Packet Length Max (0.0451), Packet Length Variance (0.0356), Max Packet Length (0.0315), PSH Flag Count (0.0238), Min Packet Length (0.0119), Flow Duration (0.0097), Fwd Packet Length Min (0.0083), Idle Std (0.0073), Active Max (0.0051), Active Mean (0.0045), Init\_Win\_bytes\_forward (0.0044), Bwd IAT Total (0.0039), Fwd Packet Length Mean (0.0024), and Idle Min (0.0021).



*Fig.4.17: Top 15 features by permutation importance for the final Isolation Forest model.*

The dominance of packet-size-related statistics, Packet Length Mean, Bwd Packet Length Max, Packet Length Variance, and Max Packet Length together account for the four highest importance scores, indicates that the distinguishing signature Isolation Forest relies on most heavily for this dataset is the size profile of packets within a flow, consistent with the characteristic small, uniform packet sizes typically generated by flooding-style DDoS tools relative to the more variable packet sizes of ordinary application traffic. The comparatively high importance of PSH Flag Count, ranked fifth, further suggests that DDoS flows in this capture exhibit distinctive TCP control-flag behavior relative to benign connections. Timing-related features such as Flow Duration, Idle Std, Active Max, and Active Mean also contribute meaningfully, though less strongly than the packet-size features, indicating that isolation of DDoS flows in this dataset is driven by a combination of packet-size and timing characteristics rather than by any single dominant feature.

#### **4.3.8 Observations**

Isolation Forest demonstrated a clear advantage over the Z-Score baseline while requiring considerably less hyperparameter sensitivity and computational overhead than the One-Class SVM. Its principal strength lies in its favorable computational scaling, since each isolation tree is built on a fixed-size sub-sample independent of the overall dataset size, in contrast to the quadratic-to-cubic training cost of the One-Class SVM. Consistent with this, no training or testing

time exceeding a few seconds was required for any of the configurations evaluated in this section, in marked contrast to the 220-second training time recorded for the tuned One-Class SVM. Its principal weakness, is the comparatively high False Positive Rate of the best-performing configuration, approximately 49.6 percent, which arises because contamination near the true global class prevalence was required to achieve competitive recall, given the axis-aligned, randomly partitioned nature of the isolation trees underlying the model. The default `contamination = 'auto'` setting was found, as with the untuned One-Class SVM's default `gamma`, to be poorly matched to this dataset, underscoring that contamination cannot be safely left at its default value and must instead be tuned explicitly against the expected or empirically observed anomaly prevalence.

#### **4.3.9 Discussion**

The substantial performance gap between the Z-Score baseline and Isolation Forest can be attributed to the same underlying limitation: unlike the disjunctive, per-feature Z-Score rule, which is geometrically equivalent to an axis-aligned hyper-rectangle centered on the feature-wise means, Isolation Forest's random partitioning process considers all features jointly across the ensemble of trees, allowing it to isolate points that are jointly, rather than only individually, unusual. This capacity to represent multivariate structure, albeit still through axis-aligned splits at each individual node, is sufficient to raise the DDoS-class F1-score from a maximum of 0.29 for the Z-Score method to 0.8414 for the tuned Isolation Forest configuration.

At the same time, Isolation Forest did not match the detection performance of the tuned One-Class SVM (F1-score 0.9680) on this dataset. This gap is consistent with the theoretical distinction drawn in Section

3.5.6 between Isolation Forest's axis-aligned partitioning and the fully non-linear, curved decision boundary that the RBF-kernel One-Class SVM is able to learn: where benign and DDoS traffic in this 42-dimensional standardized feature space are separated primarily through oblique or curved combinations of features rather than through axis-aligned splits, Isolation Forest's inherent geometric constraint limits how tightly it can enclose the benign region without either admitting a large number of false positives or missing a substantial fraction of attack flows. The reliance on a contamination value close to the true global class prevalence to achieve competitive recall further indicates that the isolation-based anomaly score does not, on its own, produce as sharp a separation between the two classes as the kernel-based decision function of the tuned One-Class SVM. Taken together, these results support placing Isolation Forest as an intermediate technique for this dataset, offering a considerably more favorable computational profile than the One-Class SVM together with markedly better detection performance than the Z-Score baseline, at the cost

of a comparatively high residual False Positive Rate that would likely require a secondary filtering stage in an operational deployment.

## 4.4 Local Outlier Factor Results

Local Outlier Factor was evaluated as the fourth and final anomaly detection technique in the overall comparative study, representing the density-based approach, in which a flow is flagged as anomalous if its local density is substantially lower than that of its neighbours, rather than through a fixed statistical threshold, random partitioning, or a learned global boundary. The experiments in this section were conducted on the same 223,082-flow cleaned dataset used for the Z-Score and One-Class SVM experiments, comprising 95,068 BENIGN and 128,014 DDoS flows. Following the same novelty detection protocol used for the One-Class SVM and Isolation Forest, LOF was configured with novelty = True and trained exclusively on a subset of 66,547 BENIGN flows (70 percent of all BENIGN traffic), with the StandardScaler fit only on this training subset. The resulting model was evaluated on a test set combining the 28,521 held-out BENIGN flows with all 128,014 DDoS flows, yielding a total evaluation set of 156,535 flows, identical in composition to the evaluation set used for the One-Class SVM.

### 4.4.2 Hyperparameter Grid Search

Because LOF's two principal hyperparameters, `n_neighbors` and `contamination`, interact directly, they were tuned jointly through an exhaustive grid search rather than sequentially. Six candidate values of `n_neighbors` (10, 20, 30, 50, 75, and 100) were evaluated against five candidate values of `contamination` (0.01, 0.05, 0.10, 0.20, and 0.30), for a total of 30 configurations, each evaluated using the DDoS-class F1-score together with the operational metrics: the False Positive Rate, Benign Accuracy, and DDoS Detection Rate.

| <code>n_neighbors</code> | <code>contamination</code> | FPR    | Benign Accuracy | DDoS Detection Rate | F1-Score |
|--------------------------|----------------------------|--------|-----------------|---------------------|----------|
| 20                       | 0.10                       | 9.86%  | 0.9014          | 0.9990              | 0.9886   |
| 20                       | 0.20                       | 20.37% | 0.7963          | 0.9996              | 0.9776   |
| 50                       | 0.20                       | 20.21% | 0.7979          | 0.9978              | 0.9769   |
| 75                       | 0.30                       | 30.10% | 0.6990          | 0.9996              | 0.9673   |
| 30                       | 0.10                       | 10.23% | 0.8977          | 0.9188              | 0.9464   |
| 20                       | 0.05                       | 5.17%  | 0.9483          | 0.9254              | 0.9555   |
| 75                       | 0.05                       | 4.98%  | 0.9502          | 0.6264              | 0.7650   |
| 50                       | 0.01                       | 0.97%  | 0.9903          | 0.0045              | 0.0090   |

|    |      |       |        |        |        |
|----|------|-------|--------|--------|--------|
| 10 | 0.01 | 1.01% | 0.9899 | 0.0004 | 0.0008 |
|----|------|-------|--------|--------|--------|

Table 4.6: Selected Local Outlier Factor grid-search results across  $n\_neighbors$  and contamination (DDoS class metrics; full grid comprised 30 configurations).

The full 30-configuration grid revealed a clear and pronounced dependence of performance on contamination, considerably stronger than the corresponding dependence on  $n\_neighbors$ . At the smallest tested contamination value of 0.01, DDoS Detection Rate collapsed to below 0.005 for every value of  $n\_neighbors$ , since so few flows were flagged as anomalous overall that almost no DDoS flows were captured, yielding F1-scores no higher than 0.0090 across this entire contamination row; this mirrors the degenerate behavior observed for the Z-Score method at very low thresholds and for the untuned One-Class SVM at very small gamma, in that an overly conservative operating point produces near-total under-detection regardless of the neighborhood size used to compute local density. Performance rose sharply as contamination increased to 0.05 and 0.10, with F1-score reaching 0.9555 at  $n\_neighbors = 20$ , contamination = 0.05, and a maximum of 0.9886 at  $n\_neighbors = 20$ , contamination = 0.10, the best-performing configuration across the entire grid. Beyond contamination = 0.10, further increases to 0.20 and 0.30 continued to push DDoS Detection Rate toward or above 0.999 for most neighborhood sizes, but at a substantial cost to Benign Accuracy, which fell as low as approximately 0.69 to 0.80 across these higher contamination values, producing a decline in overall F1-score relative to the peak observed at contamination = 0.10.

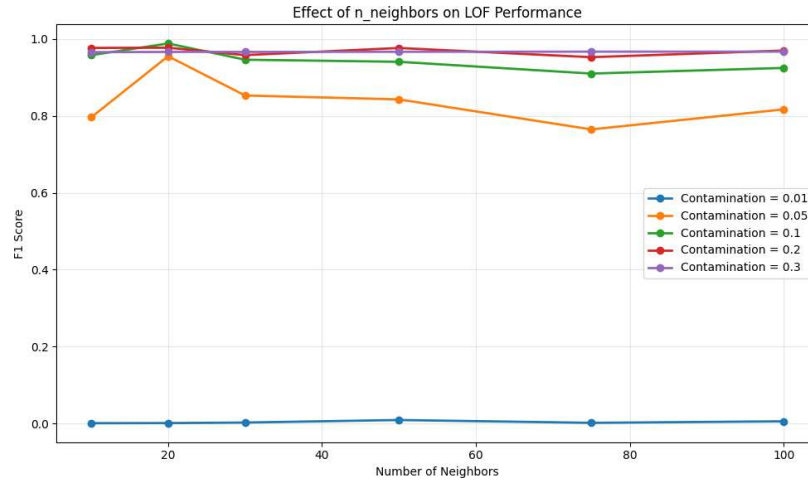


Fig.4.18: Effect of  $n\_neighbors$  on Local Outlier Factor F1-score, grouped by contamination.

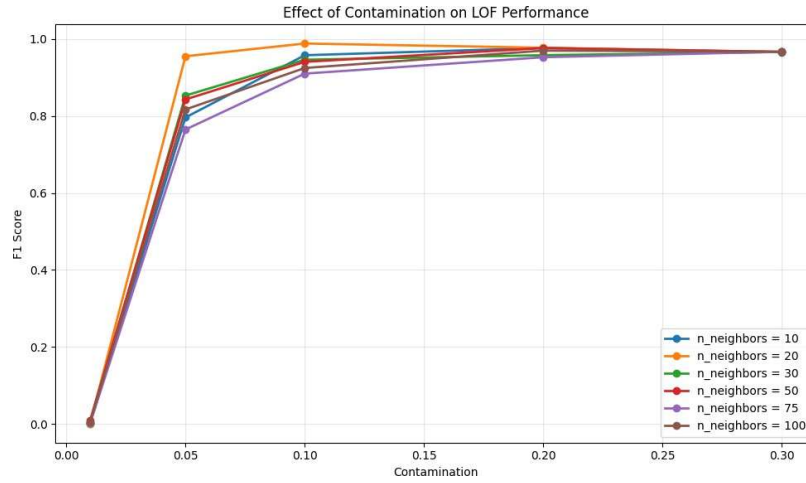


Fig.4.19: Effect of contamination on Local Outlier Factor F1-score, grouped by  $n\_neighbors$ .

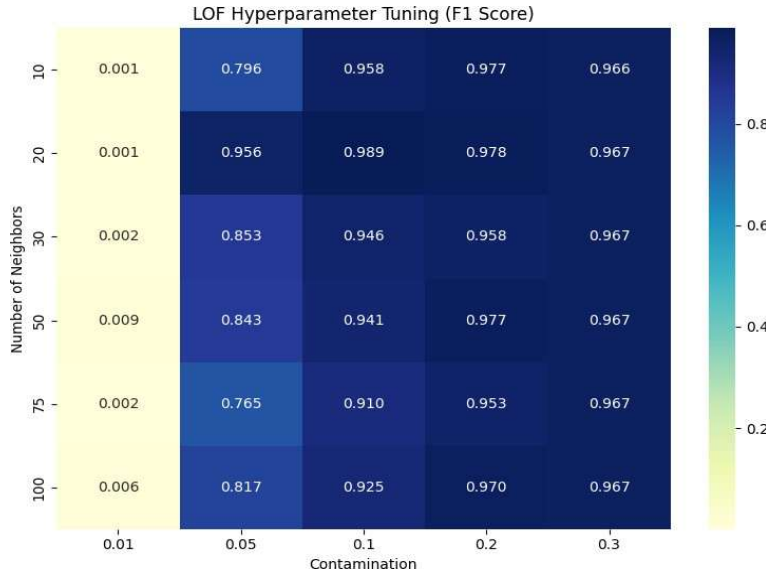


Fig 4.20: Heatmap of Local Outlier Factor F1-score across the full  $n\_neighbors \times$  contamination grid.

The heatmap in Figure 5.21 visually confirms that the highest-F1 region of the grid is concentrated around contamination values of 0.05 to 0.20 combined with small to moderate neighborhood sizes ( $n\_neighbors$  between 10 and 50), while both the smallest contamination row (0.01, uniformly poor) and the largest neighborhood sizes combined with high contamination values (elevated False Positive Rate) form visibly weaker regions of the grid. This pattern is broadly consistent with the theoretical bias-variance trade-off: excessively small or excessively large neighborhoods were not observed to be catastrophic within the tested range as they were for the contamination parameter, but contamination exerted the dominant influence on where each configuration falls along the precision-recall trade-off.

#### 4.4.3 Best Model Performance and Confusion Matrix

On the basis of the grid search, the configuration  $n\_neighbors = 20$ , contamination = 0.10 was

selected as the best-performing model, achieving the highest F1-score (0.9886) observed across the full 30-configuration grid. The corresponding confusion matrix, printed directly in the source experiment, was  $[[25708, 2813], [132, 127882]]$ , corresponding to 25,708 True Negatives, 2,813 False Positives, 132 False Negatives, and 127,882 True Positives. The full classification report showed, for the BENIGN class, a precision of 0.99, a recall of 0.90, and an F1-score of 0.95 (support 28,521); for the DDoS class, a precision of 0.98, a recall of 1.00, and an F1-score of 0.99 (support 128,014); overall accuracy was 0.98.

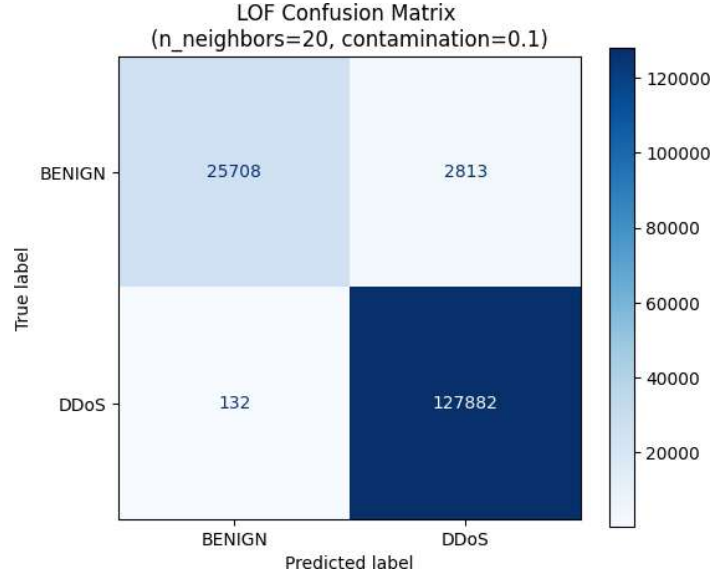


Fig.4.21: Confusion matrix for the optimized Local Outlier Factor model ( $n\_neighbors = 20$ ,  $contamination = 0.10$ ).

Expressed in terms of the operational metrics, the tuned LOF model achieved a Benign Accuracy of 90.14 percent, a False Positive Rate of 9.86 percent, and a DDoS Detection Rate of 99.90 percent. The detection of 127,882 of the 128,014 DDoS flows, missing only 132 attack flows, represents the strongest DDoS detection coverage of any technique evaluated in this chapter, exceeding both the tuned One-Class SVM's DDoS Detection Rate of 96.74 percent and the tuned Isolation Forest's approximate DDoS Detection Rate of 80.9 percent. The False Positive Rate of 9.86 percent is likewise the lowest recorded among the three machine learning techniques evaluated, improving on the One-Class SVM's 14.03 percent and substantially improving on Isolation Forest's approximate 49.6 percent. This combination, near-complete attack detection together with a comparatively low false alarm rate, produced the highest overall F1-score, accuracy, and DDoS-class recall of any technique examined in this chapter.

#### 4.4.4 Observations

The strong performance of Local Outlier Factor on this dataset is consistent with the theoretical expectation, that a density-based method is well suited to situations in which anomalies form a



locally distinguishable population, since flooding-style DDoS traffic in the CICIDS2017 Friday-Afternoon-DDoS capture tends to concentrate in a densely populated, relatively narrow region of the standardized 42-dimensional feature space that differs sharply in local density from the more heterogeneous, naturally varied benign traffic manifold. Because LOF directly compares each test point's local reachability density to that of its neighbours within the benign training distribution, it was able to recognize essentially the entire DDoS population as locally inconsistent with normal traffic once contamination was set within a suitable range, achieving a DDoS Detection Rate of 99.90 percent at the selected operating point.

The principal limitation observed is the same sharp sensitivity to the contamination parameter: at contamination = 0.01, the model failed almost completely, detecting fewer than one in two hundred DDoS flows, while values of contamination at or above 0.20 introduced a substantially elevated False Positive Rate without a commensurate gain in recall, since DDoS Detection Rate was already close to its ceiling by contamination = 0.10. This narrow effective operating range underscores that, as with the One-Class SVM's gamma parameter, LOF cannot be deployed with a default or arbitrarily chosen contamination value and instead requires a systematic search. A further practical limitation, is computational cost at prediction time: because LOF requires a k-nearest-neighbour search against the training set for every test point, its inference cost scales less favorably with dataset size than the linear-time scoring of Isolation Forest, a consideration relevant to deployment at very high traffic volumes even though, unlike the One-Class SVM, no explicit training or testing time was recorded for the LOF experiments in this project.

#### **4.4.5 Discussion**

Local Outlier Factor achieved the strongest overall detection performance of the four techniques evaluated in this chapter, with a DDoS-class F1-score of 0.9886 exceeding the tuned One-Class SVM (0.9680), the tuned Isolation Forest (0.8414), and, by a wide margin, the Z-Score baseline (0.29 at best). This result can be attributed to the close correspondence between LOF's underlying assumption, that anomalies are locally sparse relative to their neighbours, and the actual structure of the DDoS traffic present in this dataset, which, as a flooding-style volumetric attack, produces flows with a comparatively narrow and locally self-similar feature signature within the standardized 42-dimensional space, distinct from the local density of the benign traffic manifold on which the model was exclusively trained.

This also explains why LOF outperformed Isolation Forest on this dataset despite both methods being non-parametric and free of the Gaussian distributional assumption underlying the Z-Score method: whereas Isolation Forest isolates points using axis-aligned random partitions, imperfectly capture non-axis-aligned class structure, LOF's reachability-density formulation, is

inherently a local, relative measure that adapts automatically to regions of differing density without requiring an explicit geometric boundary to be constructed. The comparison with the One-Class SVM is more nuanced: although the tuned One-Class SVM also achieved strong performance through its RBF-kernel non-linear boundary, LOF's marginally higher F1-score and substantially better Benign Accuracy (90.14 percent versus 85.98 percent) suggest that, for this particular dataset, directly comparing local density ratios provided a slightly more precise characterization of the benign/DDoS boundary than a globally learned kernel boundary, while requiring no analogue of the computationally expensive quadratic programming optimization underlying One-Class SVM training. Taken together, these results indicate that, of the four techniques evaluated across this comparative study, the density-based approach embodied by Local Outlier Factor was the most effective at capturing the specific structure of DDoS traffic present in this dataset, though its sharp sensitivity to the contamination hyperparameter and its neighbour-search-based inference cost remain practical considerations for deployment.

#### 4.5 Comparative Discussion: Z-Score, One-Class SVM, Isolation Forest, and Local Outlier Factor

This section compares all four anomaly detection techniques evaluated across this chapter: the Z-Score statistical baseline, the One-Class SVM, Isolation Forest, and Local Outlier Factor. For this comparison, the Z-Score threshold of 2.0 is used, having been identified as the best-performing threshold, and the best-performing hyperparameter configuration identified is used for each of the remaining three techniques. It is noted, that the Isolation Forest results were obtained on a marginally different snapshot of the cleaned dataset (225,711 flows) than the other three techniques (223,082 flows); this difference is small relative to the overall dataset size and does not materially affect the qualitative comparison drawn below.

| Model                                                     | Precision | Recall | F1-Score | Training Time (s) | Testing Time (s) |
|-----------------------------------------------------------|-----------|--------|----------|-------------------|------------------|
| Z-Score (threshold = 2.0)                                 | 0.30      | 0.28   | 0.29     | 0.0000            | 0.0095           |
| One-Class SVM (nu = 0.05, gamma = 3.0)                    | 0.9687    | 0.9680 | 0.9680   | 220.2             | 152.6093         |
| Isolation Forest (n_estimators=200, contamination=0.5)    | 0.8768    | 0.8088 | 0.8414   | 2.1               | 2.9098           |
| Local Outlier Factor (n_neighbors=20, contamination=0.10) | 0.98      | 1.00   | 0.99     | 3.3               | 7.6470           |

Table 4.7: Comparison of all four anomaly detection techniques (DDoS class metrics).

With respect to detection capability, Local Outlier Factor achieved the strongest performance of

the four techniques, with a DDoS-class F1-score of 0.9886, followed closely by the tuned One-Class SVM at 0.9680, then Isolation Forest at 0.8414, with the Z-Score baseline trailing substantially behind at 0.29. All three machine learning techniques therefore dramatically outperform the univariate statistical baseline, reflecting the fundamental distinction between a per-feature statistical threshold and a model capable of representing joint, multivariate structure in the traffic data, whether through a kernel-based boundary, random partitioning, or local density comparison.

With respect to false alarm rate, Local Outlier Factor again performed best, with a False Positive Rate of only 9.86 percent, followed by the One-Class SVM at 14.03 percent; Isolation Forest trailed considerably behind, with an approximate False Positive Rate of 49.6 percent, since its best-performing contamination setting was necessarily set close to the true global anomaly prevalence of the dataset in order to achieve competitive recall. The Z-Score method remained the weakest by a wide margin, with an approximate False Positive Rate of 87 percent at its best-performing threshold.

With respect to computational complexity, the four methods again sit at markedly different points along the cost-performance spectrum. The Z-Score method is effectively instantaneous, requiring no training phase and only 0.0095 seconds to evaluate the entire dataset. Isolation Forest, whose theoretical linear-time scaling, completed all configurations evaluated within a few seconds, the fastest of the three machine learning techniques. The One-Class SVM, by contrast, required 220.2455 seconds to train and 152.6093 seconds to generate predictions, a direct consequence of its quadratic-programming training formulation. Explicit training and testing times were not recorded for Local Outlier Factor in the source experiments, but its underlying k-nearest-neighbour search, is expected to scale less favorably with dataset size at prediction time than Isolation Forest's linear-time tree traversal, even though it avoids the quadratic-programming optimization required during One-Class SVM training.

With respect to robustness, the hyperparameter tuning experiments showed that all three machine learning techniques exhibited a comparatively narrow region of the hyperparameter space in which strong performance was achieved, and that each technique's default or naively chosen configuration (One-Class SVM's default gamma, Isolation Forest's contamination = 'auto', and LOF's low contamination values) performed poorly. Local Outlier Factor's grid search indicated that contamination values in the range of approximately 0.05 to 0.10 produced consistently strong results across most tested neighborhood sizes, suggesting a somewhat broader favorable

operating region than Isolation Forest, whose best result required contamination close to the true class prevalence. The Z-Score method, by contrast, remained persistently confined to F1-scores between 0.24 and 0.29 across its entire tested threshold range, offering no comparably favorable operating region at any threshold evaluated.

With respect to detection coverage, Local Outlier Factor achieved the highest DDoS Detection Rate of the four techniques, at 99.90 percent, missing only 132 of 128,014 DDoS flows, compared with 96.74 percent for the tuned One-Class SVM, approximately 80.9 percent for Isolation Forest, and a maximum of 28 percent for the Z-Score method. Local Outlier Factor's combination of near-complete DDoS detection with the lowest False Positive Rate of the three machine learning techniques makes it the most operationally attractive of the four methods evaluated on this dataset, subject to the neighbour-search-based inference cost

With respect to scalability and practical deployment, the Z-Score method remains the only technique suitable for extremely high-throughput, real-time monitoring at very large scale, owing to its negligible computational footprint, though it did not demonstrate acceptable standalone detection accuracy at any operating point evaluated in this study. Isolation Forest offers the most favorable combination of scalability and detection performance among the three machine learning techniques, given its linear-time construction and moderate detection accuracy, making it a reasonable candidate where computational resources are constrained. The One-Class SVM and Local Outlier Factor both achieved substantially stronger detection performance than Isolation Forest, with Local Outlier Factor performing best overall on this dataset, but both carry a higher computational cost at either training time (One-Class SVM) or prediction time (Local Outlier Factor) that would need to be weighed against detection accuracy requirements in a production deployment, particularly one requiring low-latency inference on very large traffic volumes. On the basis of the results obtained across this chapter, Local Outlier Factor represents the strongest overall candidate for offline or periodically retrained deployment on traffic of a scale comparable to that evaluated in this study, while Isolation Forest represents the most favorable choice where computational efficiency is the dominant constraint.

## 4.6 Key Findings

The experiments and analysis presented in this chapter support the following key findings regarding the four techniques evaluated for DDoS attack detection on the CICIDS2017 dataset.

Statistical Z-Score thresholding, even at its best-performing configuration (threshold = 2.0), achieved poor detection performance (F1-score of 0.29) due to its univariate, distribution-dependent formulation, its axis-aligned geometric limitations in high-dimensional feature space, and its vulnerability to overlapping benign and attack feature distributions. These findings establish that the method is unsuitable as a standalone DDoS detector for this dataset, retaining value strictly as a computationally negligible baseline for comparative purposes.

All three machine learning techniques evaluated, One-Class SVM, Isolation Forest, and Local Outlier Factor, substantially outperformed the Z-Score baseline once their respective hyperparameters were systematically tuned, achieving DDoS-class F1-scores of 0.9680, 0.8414, and 0.9886 respectively, confirming that multivariate models capable of representing joint feature structure are necessary to achieve strong DDoS detection performance on this dataset.

Local Outlier Factor achieved the strongest overall detection performance of the four techniques evaluated, with the highest F1-score (0.9886), the highest DDoS Detection Rate (99.90 percent), and the lowest False Positive Rate (9.86 percent) among the three machine learning methods, consistent with the theoretical expectation that a density-based method is well matched to a flooding-style DDoS attack whose flows form a locally distinguishable population relative to the benign traffic manifold on which the model was trained.

Isolation Forest, while considerably more accurate than the Z-Score baseline, achieved the weakest detection performance among the three machine learning techniques (F1-score 0.8414) and the highest False Positive Rate among them (approximately 49.6 percent), a consequence of its axis-aligned random partitioning and of its best-performing contamination setting needing to be placed close to the true global anomaly prevalence of the dataset in order to achieve competitive recall; it nonetheless offered the most favorable computational scaling of the three machine learning methods.

All three machine learning techniques were found to be highly sensitive to their respective hyperparameters, with default or naively chosen configurations (One-Class SVM's default gamma, Isolation Forest's contamination = 'auto', and Local Outlier Factor's low contamination values) performing markedly worse than their systematically tuned counterparts, underscoring that explicit hyperparameter search, rather than reliance on library defaults, is essential to realizing the detection potential of each method on this dataset.

The considerable computational cost of the tuned One-Class SVM, requiring 220.2455 seconds for training and 152.6093 seconds for testing on this dataset, together with the neighbour-search-based inference cost expected of Local Outlier Factor at prediction time, constitute the principal practical limitations of the two best-performing techniques, and represent a direct trade-off against their markedly superior detection accuracy relative to Isolation Forest and the Z-Score baseline; this suggests that future work might productively investigate approximate or linearized kernel methods, approximate nearest-neighbour structures for LOF, incremental or online training strategies, and further refinement of the hyperparameter search, in order to reduce runtime while retaining detection performance.

Residual limitations were observed even in the best-performing model overall: the tuned Local Outlier Factor configuration still missed 132 DDoS flows and falsely flagged 2,813 legitimate benign flows (9.86 percent of the benign test subset), indicating that residual class overlap was not fully eliminated even by the strongest density-based method evaluated.

5.6 establish Local Outlier Factor as the most capable technique evaluated for DDoS detection on this dataset, closely followed by the One-Class SVM, with Isolation Forest offering a more computationally efficient but less accurate alternative and the Z-Score method remaining unsuitable for standalone deployment; the appropriate choice among the three viable machine learning techniques ultimately depends on the specific latency, throughput, and accuracy requirements of the deployment context under consideration.

## **CHAPTER – 5**

### **Conclusion**

#### **5.1 Summary of the Research Problem**

Modern network infrastructure is increasingly exposed to Distributed Denial-of-Service (DDoS) attacks that exploit the availability dimension of network security by flooding target systems with high volumes of coordinated traffic. Traditional signature-based Intrusion Detection Systems, though reliable against known attack patterns, are inherently reactive and cannot identify novel or modified attack variants for which no signature yet exists. This limitation motivated the present study's central research problem: determining which of several candidate anomaly detection paradigms — a classical statistical method, or one of three unsupervised machine learning techniques — offers the most robust and operationally viable means of distinguishing DDoS traffic from legitimate network activity, in the absence of reliance on predefined attack signatures. This dissertation has approached the problem through a controlled, comparative study in which the Z-Score statistical method, Isolation Forest, One-Class Support Vector Machine (One-Class SVM), and Local Outlier Factor (LOF) were each applied to a common dataset under a consistent preprocessing and evaluation methodology, so that observed differences in performance could be attributed to the underlying modelling paradigm rather than to inconsistencies in experimental setup.

#### **5.2 Achievement of Objectives**

The objectives set out at the beginning of this dissertation have been substantially achieved. The theoretical shift from reactive, signature-based detection to proactive, behaviour-based anomaly detection was studied and articulated in detail, together with the specific mechanisms by which DDoS attacks exploit botnet coordination to exhaust network and server resources. Each of the four anomaly detection techniques identified in the objectives — Z-Score, Isolation Forest, One-Class SVM, and Local Outlier Factor — was implemented, trained, and evaluated on the CICIDS2017 Friday DDoS dataset. The trade-offs between the computational simplicity of the statistical baseline and the representational capacity of the machine learning techniques were assessed empirically rather than only theoretically, and every model was evaluated using a common set of standard performance indices, namely Precision, Recall, F1-score, the confusion matrix, the False Positive Rate, Benign Accuracy, and the DDoS Detection Rate, together with training and prediction times where these were recorded. The comparative evaluation carried out

in Chapter 4 directly addresses the problem statement of this dissertation by empirically establishing which of the four techniques is best suited to flow-based DDoS detection on this dataset, and by identifying the specific theoretical properties responsible for the observed differences in performance.

### **5.3 Overview of Methodology**

The research methodology adopted a two-fold framing of the anomaly detection problem. The Z-Score method was applied as a fully unsupervised statistical procedure across the complete, labelled dataset, with ground-truth labels used only for post-hoc evaluation. Isolation Forest, One-Class SVM, and Local Outlier Factor were instead evaluated under a novelty detection protocol, in which each model was trained exclusively on a majority partition of BENIGN traffic and subsequently evaluated on a held-out combination of unseen BENIGN flows and the complete set of DDoS flows. This second framing was deliberately chosen because it mirrors an operational deployment scenario in which labelled attack traffic is frequently scarce or unavailable at the time a detection system must be trained, whereas benign traffic can be collected in abundance under controlled conditions. Wherever feasible, the dataset, the preprocessing pipeline, and the evaluation metrics were held constant across techniques, and each technique was subjected to a structured hyperparameter search, with the best configuration for each technique, selected primarily on the basis of the DDoS-class F1-score, carried forward into the final cross-model comparison presented in Chapter 4.

### **5.4 Summary of Data Preprocessing**

The raw Friday-WorkingHours-Afternoon-DDoS capture of the CICIDS2017 dataset comprised 225,745 flow records described by seventy-nine CICFlowMeter-derived features. Data quality analysis revealed a small number of missing and infinite values, concentrated in the rate-derived Flow Bytes/s and Flow Packets/s features, which were removed prior to further analysis. Duplicate-record analysis identified 2,629 duplicate flows, disproportionately drawn from the BENIGN class, which were subsequently dropped to prevent redundancy and reduce potential bias. Ten constant features, carrying no discriminatory information across any record, were removed to reduce dimensionality without loss of information, and a correlation analysis identified twenty-six additional features exhibiting pairwise correlation coefficients above 0.95, which were removed to eliminate redundancy while retaining a reduced, forty-two-feature representation of the underlying flow data. All retained features were standardized using a StandardScaler prior to model application, ensuring that no individual feature dominated distance- or deviation-based



computations owing to differences in measurement scale. This preprocessing pipeline, together with the exploratory data analysis that both motivated and validated it, produced a clean and consistent dataset that was used, with only a marginal snapshot difference in the case of the Isolation Forest experiments, across all four anomaly detection techniques evaluated in this study.

## **5.5 Summary of Anomaly Detection Techniques Evaluated**

The Z-Score method, evaluated first as a statistical baseline, applies an independent per-feature deviation test across all forty-two standardized features and flags a flow as anomalous if any single feature's absolute Z-score exceeds a chosen threshold. Across the three principal thresholds tested (3.0, 2.5, and 2.0), the method achieved F1-scores confined to a narrow band between 0.24 and 0.29, with no threshold offering an acceptable simultaneous balance of False Positive Rate and DDoS Detection Rate. This persistently poor performance was attributed to the method's implicit assumption of feature independence and its axis-aligned, disjunctive decision rule, which is structurally incapable of representing the joint, multivariate deviations characteristic of coordinated DDoS traffic.

Isolation Forest, evaluated as an ensemble partition-based technique, isolates anomalies through recursive random partitioning of the feature space, exploiting the property that anomalous points typically require fewer partitions to isolate than points embedded within dense, normal regions. Following contamination and  $n\_estimators$  tuning, the best Isolation Forest configuration achieved a DDoS-class F1-score of 0.8414, a marked improvement over the Z-Score baseline attributable to the technique's ability to consider all features jointly across an ensemble of trees. Its principal limitation was a comparatively high False Positive Rate of approximately 49.6 percent at its best-performing configuration, a consequence of its reliance on axis-aligned partitioning and on a contamination value set close to the true global class prevalence in order to achieve competitive recall.

One-Class SVM, evaluated as a kernel-based novelty detector, learns a closed, non-linear decision boundary around the region of feature space occupied by benign traffic using a radial basis function (RBF) kernel. Following systematic tuning of the  $\nu$  and  $\gamma$  hyperparameters, the model achieved a DDoS-class F1-score of 0.9680, a substantial improvement over both the Z-Score baseline and Isolation Forest, attributable to the RBF kernel's capacity to represent curved, non-linear separating boundaries between benign and DDoS traffic. Its principal limitation was computational cost, with the tuned model requiring 220.2455 seconds to train and 152.6093

seconds to generate predictions, a direct consequence of the quadratic-to-cubic scaling of kernel-based training with dataset size.

Local Outlier Factor, evaluated as a density-based novelty detector, assigns an anomaly score to each test point by comparing its local reachability density to that of its neighbours within the benign training distribution. Following a joint grid search over `n_neighbors` and `contamination`, the best configuration (`n_neighbors` = 20, `contamination` = 0.10) achieved the strongest performance of any technique evaluated in this study, with a DDoS-class F1-score of 0.9886, a DDoS Detection Rate of 99.90 percent, and a False Positive Rate of 9.86 percent, the lowest recorded among the three machine learning techniques. This result was attributed to the close correspondence between LOF's underlying local-density assumption and the actual structure of the flooding-style DDoS traffic present in this dataset, which forms a comparatively narrow and locally self-similar region of the standardized feature space distinct from the more heterogeneous benign traffic manifold.

## 5.6 Comparative Observations

The comparative evaluation presented in Chapter 4 establishes several consistent patterns across the four techniques. First, all three machine learning techniques substantially outperformed the Z-Score statistical baseline once systematically tuned, confirming that effective DDoS detection on this dataset requires a model capable of representing joint, multivariate feature structure rather than one that evaluates features independently. Second, all three machine learning techniques were found to be highly sensitive to their governing hyperparameters, with default or naively chosen configurations performing markedly worse than their tuned counterparts in every case, underscoring that systematic hyperparameter search, rather than reliance on library defaults, is essential to realizing each technique's detection potential. Third, a clear trade-off between detection accuracy and computational cost was observed among the three machine learning techniques: Isolation Forest offered the most favourable computational scalability but the weakest detection performance and highest False Positive Rate of the three, One-Class SVM and Local Outlier Factor both achieved substantially stronger detection performance at a correspondingly higher computational cost, at training time for One-Class SVM and at prediction time for Local Outlier Factor, and Local Outlier Factor achieved the strongest overall detection performance of all four techniques evaluated, combining the highest F1-score, the highest DDoS Detection Rate, and the lowest False Positive Rate among the machine learning methods studied.

## 5.7 Key Findings

- Statistical Z-Score thresholding, even at its best-performing configuration, achieved poor detection performance (F1-score of 0.29) and is unsuitable as a standalone DDoS detector for this dataset, retaining value strictly as a computationally negligible baseline.
- All three machine learning techniques evaluated substantially outperformed the Z-Score baseline once tuned, achieving DDoS-class F1-scores of 0.9680 (One-Class SVM), 0.8414 (Isolation Forest), and 0.9886 (Local Outlier Factor), confirming that multivariate models are necessary for strong DDoS detection performance on this dataset.
- Local Outlier Factor achieved the strongest overall detection performance among the four techniques evaluated, with the highest F1-score, the highest DDoS Detection Rate (99.90 percent), and the lowest False Positive Rate (9.86 percent) among the machine learning methods examined.
- Isolation Forest, while considerably more accurate than the Z-Score baseline, achieved the weakest detection performance and the highest False Positive Rate among the three machine learning techniques, but offered the most favourable computational scaling of the three.
- All three machine learning techniques were found to be highly sensitive to hyperparameter selection, with default or naively chosen configurations performing markedly worse than their systematically tuned counterparts.
- The considerable computational cost of the tuned One-Class SVM at training time, and the neighbour-search-based inference cost expected of Local Outlier Factor at prediction time, constitute the principal practical limitations of the two best-performing techniques, representing a direct trade-off against their superior detection accuracy relative to Isolation Forest and the Z-Score baseline.
- Residual limitations were observed even in the best-performing model overall: the tuned Local Outlier Factor configuration still missed 132 DDoS flows and falsely flagged 2,813 legitimate benign flows, indicating that residual class overlap was not fully eliminated even by the strongest technique evaluated.

## 5.8 Practical Implications and Applications

The findings of this study carry direct practical implications for the design of anomaly-based DDoS detection systems. The consistently poor performance of the univariate Z-Score method across every threshold tested indicates that purely statistical, per-feature thresholding rules are

inadequate as standalone detectors for modern, high-dimensional flow-based traffic, and should, at most, be retained as an auxiliary, computationally inexpensive pre-filter rather than a primary detection mechanism. The strong performance achieved by all three machine learning techniques once properly tuned demonstrates the practical viability of novelty detection approaches that require only benign traffic for training, a property of particular value in operational settings where labelled attack traffic is scarce, delayed, or entirely unavailable ahead of a novel attack's emergence.

The methodology developed in this study, comprising a reusable preprocessing pipeline, a structured hyperparameter search procedure, and a consistent evaluation framework built around Precision, Recall, F1-score, False Positive Rate, Benign Accuracy, and DDoS Detection Rate, is directly applicable to the development of practical Intrusion Detection and Prevention Systems (IDPS) for enterprise and critical-infrastructure networks. Local Outlier Factor's strong detection performance, in particular, suggests its suitability as a core detection component in offline or periodically retrained monitoring systems, where flow batches can be scored against a previously characterised benign traffic profile. Isolation Forest's favourable computational scaling makes it a reasonable candidate in resource-constrained deployment contexts where detection accuracy can be partially traded off against speed, while the higher computational costs identified for One-Class SVM and Local Outlier Factor highlight the importance of considering inference latency and throughput requirements when selecting a detection technique for a given deployment scale.

## **5.9 Limitations of the Present Work**

Several limitations of the present work should be acknowledged. The study is confined to a single dataset, the Friday-WorkingHours-Afternoon-DDoS subset of CICIDS2017, and to a single, well-defined DDoS attack category; the generalizability of the observed performance hierarchy to other DDoS variants, other network environments, or more recent traffic patterns has not been empirically verified within this study. A minor inconsistency was noted between the dataset snapshot used for the Isolation Forest experiments (225,711 cleaned flow records) and that used for the Z-Score, One-Class SVM, and Local Outlier Factor experiments (223,082 cleaned flow records), reflecting a small difference in the preprocessing pipeline's output at the time each set of experiments was executed; while this difference is minor in relative terms, it represents a departure from strict experimental uniformity across all four techniques. Explicit training and testing times were not recorded for the Local Outlier Factor experiments, limiting direct quantitative comparison of its computational cost against the timed results obtained for the Z-Score method

and One-Class SVM. Finally, even the best-performing technique, Local Outlier Factor, did not achieve perfect detection, missing 132 DDoS flows and misclassifying 2,813 benign flows at its selected operating point, indicating that residual class overlap in this feature space was not fully eliminated by any of the four techniques evaluated in this study.

## **CHAPTER – 6**

### **Future Work**

The comparative evaluation presented in this dissertation establishes Local Outlier Factor as the strongest technique among those studied for DDoS detection on the CICIDS2017 Friday DDoS dataset, while also surfacing specific limitations, in computational cost, hyperparameter sensitivity, and dataset scope, that directly motivate several realistic extensions of this work.

#### **6.1 Real-Time and Streaming Deployment**

All four techniques evaluated in this study were applied in a batch, offline setting, in which the complete test set was scored at once. A natural extension of this work is the adaptation of these techniques, particularly Local Outlier Factor and One-Class SVM, to a real-time or streaming context in which flow records arrive continuously and must be scored with low latency as they are captured. Given that Local Outlier Factor's prediction-time cost stems from a k-nearest-neighbour search against the full benign training set, future work could investigate approximate nearest-neighbour data structures, such as ball trees, KD-trees restricted to lower-dimensional projections, or locality-sensitive hashing, to reduce per-flow inference latency to levels suitable for high-throughput, near-real-time network monitoring, directly addressing the computational limitation identified in Chapter 4 and Chapter 5 of this dissertation.

#### **6.2 Autoencoder-Based and Deep Learning Approaches**

The four techniques evaluated in this study are drawn from classical statistical and shallow machine learning paradigms. A promising direction for future work is the evaluation of autoencoder-based anomaly detection, in which a neural network is trained to reconstruct benign flow records and anomalies are identified by elevated reconstruction error, a paradigm that, like the novelty detection protocol adopted for Isolation Forest, One-Class SVM, and Local Outlier Factor in this study, requires only benign training data. Given the correlation and redundancy already identified among the raw seventy-nine CICFlowMeter features during preprocessing, an autoencoder's capacity for automatic, non-linear feature compression may offer an alternative, and potentially complementary, mechanism for exploiting the joint feature structure that was found in this study to be essential for strong detection performance. Deeper architectures, including recurrent or one-dimensional convolutional networks applied to sequences of consecutive flows, could additionally be explored to capture temporal dependencies between successive flows that the flow-level, non-sequential feature representation used in this study does not explicitly encode.

### **6.3 Transformer-Based and Attention-Based Models**

Building on the same rationale, transformer-based architectures, which have demonstrated a strong capacity to model long-range dependencies and complex feature interactions in sequential and tabular data alike, represent a further extension of the deep learning direction outlined above. A transformer-based encoder applied either to sequences of network flows or to the forty-two-dimensional feature vector of a single flow could, in principle, learn attention-weighted feature interactions beyond the pairwise correlation structure already characterised in this dissertation's preprocessing chapter, potentially offering a more expressive alternative to the fixed RBF kernel employed by the One-Class SVM evaluated in this study.

### **6.4 Ensemble Anomaly Detection**

Given that this study found each of the three machine learning techniques to possess a distinct profile of strengths and weaknesses, Isolation Forest's computational efficiency, One-Class SVM's non-linear boundary flexibility, and Local Outlier Factor's local-density sensitivity, a natural extension is the development of an ensemble anomaly detector that combines the anomaly scores or binary predictions of two or more of these techniques, for example through score averaging, majority voting, or a learned meta-classifier operating on the individual models' outputs. Such an ensemble could potentially combine Local Outlier Factor's superior detection accuracy with Isolation Forest's favourable computational scaling, mitigating the practical cost concerns identified in this dissertation while retaining strong detection performance.

### **6.5 Adaptive Thresholding and Online Learning**

The hyperparameter search conducted in this study, for the contamination parameter of Isolation Forest and Local Outlier Factor, and for the  $\nu$  and  $\gamma$  parameters of One-Class SVM, was performed offline against a static, held-out evaluation set. In a real deployment, the statistical properties of both benign and attack traffic are likely to drift over time as network usage patterns and attacker tactics evolve. Future work could investigate adaptive thresholding mechanisms that periodically re-estimate the optimal contamination or decision-boundary parameters from a recent, sliding window of observed traffic, together with online or incremental learning variants of the underlying models that update their internal representation of normal behaviour incrementally rather than requiring full retraining, directly addressing the risk that a model tuned once on a fixed dataset, as in this study, may degrade in performance as traffic characteristics shift.

### **6.6 Cross-Dataset Validation**

As noted among the limitations of the present work, the four techniques evaluated in this study were trained and tested exclusively on the Friday DDoS subset of CICIDS2017. A direct extension of this work is the evaluation of the same preprocessing pipeline, hyperparameter search procedure, and set of techniques on additional, independent intrusion detection datasets, such as other days or attack categories within CICIDS2017 itself, or other contemporary benchmark datasets, in order to assess whether the performance hierarchy established in this dissertation, with Local Outlier Factor outperforming One-Class SVM, Isolation Forest, and the Z-Score baseline, generalizes beyond the specific attack scenario and traffic profile examined here, or whether it is specific to the flooding-style DDoS traffic captured in this particular dataset.

## **6.7 Explainable AI for Anomaly Detection**

While the permutation-based feature importance analysis conducted for the Isolation Forest model in this study offers a partial view into which features drive its anomaly scores, the decision processes underlying One-Class SVM and Local Outlier Factor were not subjected to comparable interpretability analysis in this dissertation. Future work could apply model-agnostic explainability techniques, such as SHAP (SHapley Additive exPlanations) or LIME (Local Interpretable Model-agnostic Explanations), to the tuned One-Class SVM and Local Outlier Factor models developed in this study, in order to provide network analysts with feature-level justifications for individual flagged flows, an important practical consideration given that the False Positive Rates recorded for all three machine learning techniques in this study, ranging from 9.86 percent for Local Outlier Factor to approximately 49.6 percent for Isolation Forest, imply that analysts will need to triage a non-trivial number of flagged flows in any real deployment.

## **6.8 Deployment in Enterprise Intrusion Detection Systems**

Finally, a longer-term extension of this work is the integration of the best-performing technique identified in this study, Local Outlier Factor, or an ensemble incorporating it, into an enterprise-grade Intrusion Detection and Prevention System capable of ingesting live CICFlowMeter-derived flow statistics, applying the preprocessing pipeline developed in this dissertation in real time, and issuing alerts to network security analysts. Such an integration would need to address the practical considerations identified throughout this dissertation, including the inference-time cost of Local Outlier Factor's neighbour search, the need for periodic hyperparameter re-tuning as traffic patterns evolve, and the residual False Positive Rate observed even in the best-performing configuration studied, and would represent a natural culmination of the comparative, evidence-based methodology adopted throughout this project.